

QMX-Panadapter

Shared via NAS

Network radio support for the M5Stack Tab5 QMX-Panadapter: using a QRP Labs QMX/QMX+ plugged into a Synology NAS over TCP/IP, including the radio's own menu terminal

Item	Value
Firmware base	Upstream SteffenLav/qmx-panadapter v1.16.11 (tag commit 7337eb5), branch network-radio
Project folder	~/src/qmx-panadapter-v1.16
NAS application	tools/nas-relay (Python 3, Linux usbfs or libusb; runs natively on DSM via Task Scheduler)
Tested on	Synology DS416j, DSM 7.4.1, with a QMX+ (firmware 1_03_002)
Protocol	QMXR/1 over TCP port 7355
Toolchain checked	ESP-IDF v5.4.4, target ESP32-P4
Document date	October 04, 2026

Contents

- 1. Summary of outcome
- 2. Background and findings
- 3. Architecture
- 4. QMXR/1 wire protocol
- 5. Firmware changes (driver side)
- 6. NAS application (relay)
- 7. Verification performed
- 8. Deployment steps
- 9. Known limitations and open items
- 10. Sources (references)
- 11. Driver listings (firmware)
- 12. Application listings (NAS relay)

1. Summary of outcome

The QMX-Panadapter firmware can now use a QMX that is plugged into a Synology NAS instead of the Tab5's own USB port. A small relay application runs **on the NAS itself** (no extra computer), owns the QMX from userspace (Linux usbfs, no DSM drivers), and serves the radio's 48 kHz I/Q stream, its CAT port and its optional second serial port (the radio's menu terminal) to the Tab5 over a single TCP connection.

- Firmware work is built on the current upstream release **v1.16.11** in a fresh clone (~src/qmx-panadapter-v1.16, branch network-radio).
- On the Tab5's touchscreen: **settings drawer → Radio source** shows where the radio is and whether it is connected, and switches between USB and the NAS (the last NAS address is remembered). The same setting is on the web page under **Radio menu → Radio source**. The setting persists, appears in the config file (relay_host, relay_port) and is exposed as GET/POST /api/relay.
- Panadapter, FT8/FT4, WSPR, CAT control and the Tab5's RX audio work over the network exactly as over USB: network I/Q enters the same sample ring and RX-audio path as the USB audio driver.
- **Radio menus** (the QMX terminal) work over the network on the radio's second serial port, carried as a separate channel so CAT is never disturbed.
- With the radio set to **3 USB serial ports**, the relay also serves the QMX's **third serial port** as a plain TCP serial port on the NAS (default 7356), so PC software (WSJT-X CAT, rigctld, logging programs, a terminal) can control the radio at the same time as the Tab5, each on its own port.
- All changed firmware files compile cleanly (0 errors, 0 warnings) against ESP-IDF 5.4.4 for ESP32-P4; the firmware network code and the relay were tested end-to-end (Section 7).
- **Installed and tested on the real hardware:** the relay runs on the Synology DS416j (10.0.0.137) with the QMX+ plugged in, starts at boot, and passed both its self-test and a network test from the NAS (I/Q 47,680 pairs/s over TCP, CAT, Radio menus port, PC on port 3 alongside the Tab5).
- **End to end on the Tab5:** the Tab5 on WiFi receives **47,700 of 48,000 I/Q pairs/s** from the NAS, sets the QMX's IQ mode and reads frequency and mode over the network, and stays responsive to touch. This needed three fixes in the Tab5's WiFi receive path (esp_hosted SDIO driver), added as standing patches - without them it received 10-20k pairs/s and touch input froze (Section 5).
- **Stays up:** a deadlock that froze every network connection on the Tab5 20-50 s after boot (the QMX clock update after network time) is fixed. Switching NAS → USB → NAS from the drawer and powering the QMX off and on at the NAS were tested; the stream resumes by itself.
- The firmware builds and flashes on macOS with tools/mac-build-flash.sh and reports itself as v1.16.11-NAS.

2. Background and findings

Starting point

- Base: upstream QMX-Panadapter (github.com/SteffenLav/qmx-panadapter) release **v1.16.11**, which has no network-radio support - it assumes the QMX is on the Tab5's own USB port.
- Its CAT layer (~3,300 lines) writes to the USB CDC device directly in 63 places, so the feature was implemented by routing every CAT write through one transport function rather than duplicating paths.

Chosen approach

Run the relay on the Synology. DSM does not ship USB-audio drivers, so instead of depending on kernel drivers the relay talks to the QMX from **userspace** through Linux usbfs (/dev/bus/usb), using ioctl() directly, or libusb where available. The DS416j is an ARMv7 model without Container Manager and without libusb-1.0, so the relay runs natively with DSM's own Python 3.8, started at boot by a root Task Scheduler task. It opens the QMX only while a Tab5 or a PC is connected, and releases it afterwards.

QMX USB facts used (from the QMX-Panadapter source and documentation)

Function	Interfaces / endpoints	Notes
CAT serial (CDC-ACM)	IF0 comm, IF1 data (bulk IN/OUT)	38400 8N1, DTR+RTS asserted; Kenwood-style CAT
I/Q audio (UAC)	IF3 alt1, iso IN EP 0x83, mps 300	48 kHz, stereo, 24-bit packed; firmware keeps s24 >> 8
TX audio (UAC)	IF4 alt1, iso OUT EP 0x03	not used by this feature
Second serial port	CDC function starting at IF5 (data IF6)	only when the radio is set to USB serial ports = 2 or 3; used by the Radio menus terminal
Third serial port	next CDC function after port 2 (e.g. IF7/IF8)	only when the radio is set to USB serial ports = 3; served to PC software on TCP 7356
USB IDs	VID 0x0483, PID 0xA34C	STM32-based composite device

3. Architecture

```

QMX / QMX+ --USB--> Synology NAS
                        Task Scheduler (root, boot-up): start-relay.sh
                        qmx_nas_relay.py -- usbfs ioctl / libusb
                        | CAT   : CDC #1 bulk IN/OUT
                        | I/Q   : UAC iso IN 0x83 -> int16 (s24>>8)
                        | TERM  : CDC #2 bulk IN/OUT (on request)
                        | PORT3 : CDC #3 bulk IN/OUT --> TCP 7356
                        |       raw serial for PC software
                        v
                        TCP 7355, QMXR/1 frames
                        v
M5Stack Tab5 QMX-Panadapter (net/radio_relay.c)
  IQ  -> audio_push_net_iq() -> same sample ring + RX audio
  CAT -> handle_rx() queue -> cat_rx_task (same parser as USB)
  TERM -> qmx_term.c screen model (Radio menus)

```

- **Session = radio present.** The relay answers the Tab5's HELLO only after it has opened the QMX and closes the socket when the QMX disappears, so the firmware treats link loss exactly like a USB unplug and retries every ~2 s.
- **One Tab5 at a time, one PC at a time.** On each listener a new connection replaces the old one, so a rebooted client cannot be locked out by its own stale session.
- **Shared radio.** The QMX is opened when the first user (the Tab5 or a PC on port 3) connects and released when the last one leaves; either can come and go without affecting the other.
- **Latency over completeness.** If WiFi cannot keep up, the relay drops I/Q frames (counted) and never drops CAT or terminal bytes. Throughput is about 192 kB/s of I/Q.
- **Transport switch.** In the firmware every CAT write goes through one `cat_tx()` that sends via USB CDC or the relay; `link_task` chooses the transport on every (re)connect, and `cat_reconnect()` forces a switch when the setting changes.

4. QMXR/1 wire protocol

Every frame is a 4-byte header followed by the payload: u8 type, u8 flags (0), u16 length (little-endian).

Type	Name	Direction	Payload
1	HELLO	both	Tab5: QMXR/1 tab5-qmx-panadapter; relay: QMXR/1 rate=48000 ch=2 fmt=s16le src=nas term=1
2	IQ	relay → Tab5	interleaved int16 LE I/Q pairs at 48 kHz, 480 pairs (1920 bytes) per frame

Type	Name	Direction	Payload
3	CAT	both	raw bytes of the QMX's first serial port
4	PING	relay → Tab5	empty, once per second (Tab5 treats 3 s of silence as link loss)
5	TERM_OPEN	Tab5 → relay	empty: open the QMX's second serial port
6	TERM_STATUS	relay → Tab5	u8 ok (1/0) + optional reason text; unsolicited 0 = port went away
7	TERM_DATA	both	raw bytes of the second serial port (terminal)
8	TERM_CLOSE	Tab5 → relay	empty: close the second serial port

- Feature negotiation: the relay advertises term=1. A newer firmware talking to an older relay reports “update tools/nas-relay” instead of opening the terminal; an older firmware ignores the extra token.
- I/Q scaling matches the USB path bit-for-bit: for 24-bit samples the relay keeps the upper two bytes, which equals the firmware's `s24 >> 8` (verified on 10,000 random samples).

5. Firmware changes (driver side)

All changes are in `~/src/qmx-panadapter-v1.16` on branch `network-radio` (local commits). Full listings are in Section 11.

File	Change
<code>main/net/radio_relay.c/.h</code> (new)	QMXR/1 client: non-blocking connect with timeout, handshake, RX task dispatching IQ/CAT/TERM, thread-safe send, link-drop detection, terminal open/send/close API
<code>main/cat/cat.c, cat.h</code>	<code>cat_tx()</code> transport switch (59 CDC writes rerouted), <code>CAT_LINK_UP()</code> guards, <code>try_open_net()</code> , net teardown in <code>link_task</code> , <code>cat_reconnect()</code> , <code>cat_get_transport()</code> , orderly shutdown returns a networked radio to RX
<code>main/audio/audio.c, audio.h</code>	<code>audio_push_net_iq()</code> : IQ balance, peak meters, sample ring, RX-audio forward; a locally plugged QMX is ignored while the network source is selected
<code>main/qmx_term.c</code>	Radio menus over the network: second serial port via the relay, network gone-callback, unchanged behaviour on USB
<code>main/storage/settings.c/.h, config_io.c</code>	<code>relay_host / relay_port</code> persisted in NVS (dirty bit 134), included in config export/import
<code>main/net/webserver.c</code>	GET/POST <code>/api/relay</code> ; <code>radio_link</code> field in <code>/api/status</code> ; HTTP handler table 55 → 64 - the two <code>/api/relay</code> handlers made 56 and the live-spectrum socket (<code>/ws</code> , registered last) was refused, so the web page's spectrum never connected
<code>main/net/www/index.html</code>	Radio menu → Radio source entry (prompt for NAS IP[:port]; blank = USB)
<code>main/main.c, main/CMakeLists.txt</code>	apply the stored target before <code>cat_init()</code> ; build <code>radio_relay.c</code>
<code>main/net/radio_relay.c</code> (fix)	no lwIP call until WiFi is connected: <code>getaddrinfo()</code> before the tcpip thread existed asserted in <code>cat_link</code> at 4 s uptime - a boot loop as soon as a NAS address was saved
<code>CMakeLists.txt</code>	<code>PROJECT_VER</code> = latest release tag + “-NAS” (e.g. v1.16.11-NAS); the update check compares only the numbers before the first ‘-’
<code>main/idf_component.yml</code>	pin <code>usb_host_cdc_acm</code> 2.4.0, <code>usb_host_hid</code> 1.2.0, <code>esp_codec_dev</code> 1.5.11, <code>mdns</code> 1.11.3 - the versions in the official v1.16.11 image; a fresh fetch of newer ones built without working touch input
<code>sdkconfig</code>	LWIP TCP receive window 2880 → 11520, receive mailbox 6 → 12 (send buffer stays 2880)

File	Change
tools/patches (2 new, 2 fixed), tools/check_patches.py	apply_esp_hosted_sdio_split_bundles.ps1 and apply_esp_hosted_sdio_rx_copy_psram.ps1 (below), both required by check_patches.py; apply_cdc_acm_close_tolerant.ps1 also accepts cdc_acm 2.4.1; apply_httpd_ws_dead_socket_close.ps1 matches LF as well as CRLF source trees (macOS/Linux)
tools/mac-build-flash.sh, mac-capture-serial.sh (new)	macOS build + flash with the standing patches (PowerShell), and a USB serial log recorder
main/ui/ui.c	settings drawer: Radio source button (live state, refreshed each second) and a panel with the NAS address, number keyboard, Use NAS / Use USB / Cancel - the same three calls as POST /api/relay; last NAS address kept in NVS
main/wifi/wifi.c	the SNTP callback runs on lwIP's tcpip thread; its QMX time push is a CAT write, which over the network is a socket send that waits on that same thread - every connection froze for good. The time work now runs on its own task
main/net/radio_relay.c (guard)	a send from the tcpip thread is refused with an error log instead of deadlocking lwIP
tools/patches/apply_esp_hosted_sdio_split_psram.ps1 (new)	received SDIO frames queued as PSRAM copies (below); required by check_patches.py (27 patches)

The Tab5's WiFi receive path (esp_hosted SDIO)

The Tab5 reaches WiFi through an ESP32-C6 co-processor on SDIO, driven by esp_hosted. The relay stream is a steady ~1.5 Mbit/s inbound - far more than the firmware had ever received - and it exposed three faults in the host driver, all fixed as standing patches on top of the project's existing SDIO recovery patch:

- **Multi-frame reads (QMX_SDIO_RX_SPLIT).** The C6 streams several frames per SDIO read when they queue up; the host is built for one frame per read (RX_NONE, 1536-byte mempool buffers). A normal read holding two frames lost the second, and a burst larger than 1536 bytes was drained and discarded whole by the oversize recovery - several times a second under the stream, so TCP kept retransmitting and the rate collapsed. The patch walks each read, copies every frame into its own mempool buffer and delivers it; a header that does not parse or an empty pool ends the walk without aborting. Verified off-target on the measured 17 x 1526-byte burst, padding, corrupt and truncated input (AddressSanitizer clean, no leaks).
- **Frame copies in internal RAM (QMX_SDIO_RX_COPY_PSRAM).** Each received frame is copied into a buffer that lwIP holds until TCP has delivered it. It came from malloc, and with SPIRAM_MALLOC_ALWAYSINTERNAL=16384 every frame-sized malloc lands in internal RAM - so internal and DMA-capable RAM ran out with the data in flight, and the touch controller (I2C) and the web server stopped until a reboot. The copy now goes to PSRAM, and an allocation failure drops the frame instead of asserting.
- **Frames parked in the DMA mempool (QMX_SDIO_SPLIT_PSRAM, QMX_SDIO_RXQ_PSRAM).** Every received frame waited in the RX queue (depth 20) in a block of the SDIO buffer pool, and that pool never returns memory to the heap. Under the stream it grew to about 20 blocks and DMA-capable RAM fell to under 1 KB (SD card and I/O-expander reads failing). Split frames, and then every frame, are now queued as PSRAM copies and the pool block is returned at once.

6. NAS application (relay)

- **qmx_nas_relay.py:** discovers the QMX's CDC functions (paired by order, since the QMX's interfaces are not contiguous) and its stereo UAC input; claims them with auto-detach of any kernel driver; sets line coding and DTR/RTS; selects the streaming alt setting and requests 48 kHz (UAC1 SET_CUR); runs 8 isochronous transfers of 8 x 1 ms plus bulk CAT reads.

- Two USB backends behind one interface: **usbfs** (Linux, Python ctypes + ioctl: SUBMITURB / REAPURBNDELAY for iso and bulk, CONTROL, SETINTERFACE, CLAIM/RELEASE, kernel-driver disconnect; a reaper thread completes URBs) and **libusb** (python-libusb1). libusb is used when it loads; otherwise usbfs. QMX_BACKEND=usbfs|libusb forces one.
- Up to three QMX serial ports: port 1 = CAT for the Tab5, port 2 = Tab5 Radio menus (opened on request), port 3 = raw TCP serial for PC software on 7356 (--serial-port N, 0 = off). A shared radio manager keeps the QMX open while either the Tab5 or a PC is connected.
- Clear diagnostics: QMX not found; QMX busy (claimed by another program on the NAS); no USB permission; radio set to too few serial ports for Radio menus or port 3. --probe lists USB devices, the QMX's interfaces and its number of serial ports.
- --fake provides a synthetic +3 kHz tone, CAT emulators on ports 1 and 3 and a small menu terminal on port 2 for bench tests without hardware.
- --selftest (at every start) and --selftest-only: read-only check of the QMX - I/Q rate and level, CAT ID; FA; MD; VN;, ports 2 and 3. Nothing that transmits; no CR is sent to port 2, so the radio never enters terminal mode.
- **DSM native install**: start-relay.sh (supervisor loop, restarts the relay, log rotation at 2 MB), stop-relay.sh, env.sh (finds Python), check.sh (system and USB report, probe, then nettest.py against the running relay or a one-off self-test), and get_libusb1.py (optional python-libusb1 fetch, SHA-256 pinned).
- **CAT before I/Q**: control frames (CAT, terminal, status, ping) always go out ahead of queued I/Q; the I/Q queue holds 3 frames (30 ms) and the kernel send buffer is 8 KB, so a slow WiFi link drops spectrum data instead of delaying the radio's replies (before this the Tab5's IQ-mode setup timed out behind seconds of queued I/Q).
- **Updates without a new root task**: a restart.request file plus a run of the check task restarts the relay with new code (found via /proc - DSM's ps does not list it).
- **Dockerfile** and **docker-compose.yml** for models with Container Manager.

7. Verification performed

Test	Result
Compile all 8 changed firmware files against ESP-IDF v5.4.4 (ESP32-P4) with the project's sdkconfig and real component headers	Pass: 0 errors, 0 warnings
Compile ui.c, wifi.c, radio_relay.c and the patched sdio_drv.c with the Mac build's exact flags	Pass: 0 errors; no new warnings (ui.c keeps its 4 upstream unused-function warnings)
Web page JavaScript syntax (node --check on all inline scripts)	Pass
Firmware radio_relay.c compiled on Linux (POSIX stand-ins for FreeRTOS/lwIP) against the real Python relay: handshake, I/Q rate, CAT round trip	Pass: 48,480 pairs/s; ID/FA replies correct
Same harness: Radio menus open, screen data, arrow keys, close, reopen; no terminal bytes in CAT	Pass
Same harness: relay killed during a terminal session	Pass: CAT and terminal both notified
One-serial-port radio	Pass: clear "set USB serial ports to 2"; CAT/I/Q unaffected
Relay libusb path against a mock QMX with the real interface layout (IF0/1 CAT, IF3 iso 0x83, IF5/6 port 2)	Pass: correct interfaces claimed/released, line coding, 48 kHz request, unplug detection, busy-device message

Test	Result
24-bit to int16 conversion vs firmware s24 >> 8 (10,000 random samples); 32-bit variant	Pass: bit-exact
Session replacement and “no QMX on the NAS” behaviour	Pass
Mock QMX with three serial ports (port 3 at IF7/IF8): port 3 claimed, line coding, data kept separate from CAT and port 2, close/reopen, release on close	Pass
Port 3 end-to-end: PC alone; PC and Tab5 together (PC frequency change seen by the Tab5); Tab5 leaves while PC continues; radio released only after both leave	Pass
Port 3 on a two-port radio	Pass: connection refused with “set USB serial ports to 3”; Tab5 unaffected
Patch applied to ~/src/qmx-panadapter-v1.16 identical to the compiled tree	Pass: checksums match
Real hardware , Synology DS416j + QMX+ via usbfs: relay self-test at start (I/Q, CAT ID/FA/MD/VN, 3 serial ports, port 2 opens, port 3 answers FA)	Pass: 47,993 pairs/s, -86.7 dBFS; ID020, VN1_03_002QMX
Real hardware , nettest.py on the NAS against the running relay: Tab5 handshake on 7355, I/Q over TCP, CAT, Radio menus port, PC on 7356 while the Tab5 is connected, Tab5 still answers after the PC leaves; QMX released afterwards	Pass: 47,680 pairs/s; all checks passed
Real hardware, end to end : Tab5 on WiFi ← NAS relay ← QMX+, stock esp_hosted RX path	10-20k pairs/s; SDIO bursts discarded several times a second; IQ mode not confirmed (CAT replies ~1 s late); touch input frozen
Same, with the frame-split patch only (TCP window 2880 / 11520)	31.5k / 47.1k pairs/s, 0 frames dropped; touch works only at 2880 (internal RAM low-water 1-4 KB)
Same, with both WiFi patches, window 11520, relay CAT-first sending	Pass: 47,700 pairs/s , IQ mode confirmed on the first attempt, frequency/mode read; touch works (about 0.5 s response)
Raw touch read instrumented during the stream (temporary diagnostic, since removed)	Controller reports the finger, 0 I2C errors, reads ≤ 10 ms; taps and edge swipes reach the UI
Patch scripts on macOS/Linux (PowerShell 7) vs the compile-tested patched source	Pass: byte-identical output; second run is a no-op; check_patches: all 26 present
Network on the Tab5 after SNTP sync, before the fix	Every connection dead within 1 s of “SNTP sync” (twice, 23 s and 49 s uptime)
Same, after the fix (time push on its own task)	Pass: “Tab5→QMX time push” succeeds; stream 48,000 pairs/s for minutes; web page answers
Radio source from the drawer: NAS → USB → NAS	Pass: target logged each time, NAS reconnects, touch works
QMX+ powered off (lead pulled) and on at the NAS while the Tab5 is connected	Pass: stream pauses and resumes by itself

Test	Result
Web page live spectrum (/ws) with the handler table at 64	Pass: socket registers, 1 live session, spectrum drawn from the NAS stream
Split-psram patch: both stages applied to the pre-patch file, stage 2 to a stage-1 file; second run	Pass: identical output; no-op; compiles clean; check_patches: all 27 present
Touch dead after a restart (raw reads OK, controller reports no finger, INT idle)	Cleared only by a full power-off of the Tab5; touch then worked with the stream running

Not yet done: a long (multi-hour) soak of the Tab5 on the network radio, and building the container image.

8. Deployment steps

1. On the radio: **System config** → **GPS & Ser. ports** → **USB serial ports** = **2** for Radio menus, or **3** to also give PC software its own port.
2. Copy tools/nas-relay to the NAS with File Station (here /volume1/File Share/qmx-relay). Control Panel → Task Scheduler → Create → Triggered Task → User-defined script: user **root**, event **Boot-up**, command `sh "/volume1/File Share/qmx-relay/start-relay.sh"`; then Run. qmx-relay.log shows SELFTEST PASSED and listening on 0.0.0.0:7355 / 7356. Optional: a second root task running check.sh writes check.log (NETTEST PASSED). Models with Container Manager can use the docker-compose.yml instead.
3. Build and flash the Tab5 from ~/src/qmx-panadapter-v1.16. On a Mac: `bash tools/mac-build-flash.sh` - it applies and checks all standing patches (PowerShell), builds with the ESP-IDF in ~/esp/esp-idf and does a normal flash (settings kept). Nothing may be plugged into the Tab5's own USB port while flashing.
4. Updating the relay later: copy the new files to the NAS, create an empty restart.request file in the same folder and run the check task - the relay restarts with the new code and the network test runs.
5. On the Tab5: settings drawer → **Radio source** → enter the NAS IP address → **Use NAS** (or on the web page: Radio → Radio source). The drawer button reads "NAS <address> ✓" once connected.
6. Optional, PC software: map a COM port to <NAS IP>:7356 (macOS/Linux: `socat pty,link=$HOME/qmx-port3,raw,echo=0 tcp:<NAS IP>:7356`; Windows: a virtual-serial-over-TCP tool) and configure the program as for a USB-connected QMX.

9. Known limitations and open items

- FT8/FT4 transmit sends CAT tone commands every 160 ms; these now cross WiFi and the NAS. Expected to be fine on a LAN but the first transmissions should be watched.
- The relay must be the only program on the NAS that claims the QMX's USB device.
- Port 3 carries serial data only, not audio: digital-mode software on a PC still needs the radio's audio by other means.
- Builds of this branch report the release they are based on plus -NAS (v1.16.11-NAS), not upstream's git-describe version, so two builds of the branch are not told apart by the version string.
- Diagnostic USB-only helpers (second-port probe, USB replug) do not apply to a networked radio.
- Touch responds about 0.5 s after a tap while the full stream runs; how this compares with a QMX on the Tab5's own USB port has not been measured.
- Internal RAM still dips to a low-water mark of about 2 KB (it recovers); a multi-hour soak on the network radio is the real test of stability.
- The C6 still delivers multi-frame bursts about 35 times a second under the stream. None are lost now, but splitting them costs CPU.

- The Tab5 offers upstream releases over the air. Installing one replaces this firmware and removes network-radio support until the branch is ported to it.
- Four components are pinned to the official v1.16.11 versions; unpinning them needs a retest of touch input.
- The touch controller can get stuck so that it reports no finger; a restart does not clear it, a full power-off does. What first put it in that state (one of the earlier deadlocks or crashes, or something else) is not known.
- After a flash the touch controller is stuck the same way, every time so far, although the firmware pulses its reset line at boot; powering the Tab5 off and on after flashing restores touch.

10. Sources (references)

Each source is kept on one page. “Verified” means it was read or executed during this work; “background knowledge” means it was relied on but not re-checked here.

[1] Upstream firmware: SteffenLav/qmx-panadapter v1.16.11

Location: <https://github.com/SteffenLav/qmx-panadapter> (tag v1.16.11, commit 7337eb5)

Used for: Base for the port. Cloned and read: main/cat/cat.c, audio/audio.c, qmx_term.c/.h, net/webserver.c, storage/settings.c, config_io.c, main.c, net/www/index.html, CMakeLists.txt, idf_component.yml, and the QMX USB descriptor notes in docs/ (interfaces, endpoints, sample format). Verified.

[2] QMX second serial port behaviour

Location: upstream main/qmx_term.h and cat.c (cat_probe_extra_cdc_ports)

Used for: Second CDC function at interface 5 when “USB serial ports” = 2; CAT stays on interface 0; terminal entered with a bare CR; 0x7F as delete. Verified in code comments (measurements by the upstream authors).

[3] Espressif ESP-IDF v5.4.4

Location: <https://github.com/espressif/esp-idf> (tag v5.4.4)

Used for: Toolchain (riscv32-esp-elf GCC 14.2) and headers used to compile-check the firmware for ESP32-P4. Verified.

[4] Espressif USB host class drivers (esp-usb)

Location: <https://github.com/espressif/esp-usb> (host/class/cdc/usb_host_cdc_acm, host/class/hid)

Used for: CDC-ACM host API headers (cdc_acm_host_open, data_tx_blocking, line coding) for the compile check. Verified.

[5] Espressif BSP, DSP and protocol components

Location: <https://github.com/espressif/esp-bsp>, <https://github.com/espressif/esp-dsp>, <https://github.com/espressif/esp-protocols>

Used for: esp_lvgl_port, esp_lcd_touch, esp-dsp and mdns headers needed to compile the touched files. Verified.

[6] LVGL v9.2.2

Location: <https://github.com/lvgl/lvgl> (tag v9.2.2)

Used for: Graphics library built as an ESP-IDF component for the compile check. Verified.

[7] libusb-1.0 and python-libusb1

Location: <https://libusb.info> ; <https://pypi.org/project/libusb1/>

Used for: Userspace USB access used by the relay (control, bulk and isochronous transfers, kernel-driver auto-detach). Package installed and exercised in tests. Verified (against a mock device). Not present on the DS416j, which uses the usbfs backend instead.

[8] Linux usbfs (USB device filesystem) ioctl interface

Location: Linux kernel include/uapi/linux/usbdevice_fs.h; Documentation/driver-api/usb/usb.rst

Used for: USBDEVFS_SUBMITURB / REAPURBNDELAY / DISCARDURB, CONTROL, SETINTERFACE, CLAIM/RELEASEINTERFACE and IOCTL_DISCONNECT/CONNECT, struct layouts on 32-bit ARM. Verified on the DS416j (kernel 3.10.108).

[9] Python official container image

Location: https://hub.docker.com/_/python (python:3.12-alpine)

Used for: Base image for the optional relay container. Not built in this environment (no Docker daemon). Background knowledge.

[10] USB CDC (PSTN) class requests

Location: USB Class Definitions for Communications Devices 1.2, PSTN subclass

Used for: SET_LINE_CODING (0x20) and SET_CONTROL_LINE_STATE (0x22) issued by the relay on each serial port. Background knowledge.

[11] USB Audio Class 1.0

Location: USB Device Class Definition for Audio Devices 1.0

Used for: Format Type I descriptor parsing and the endpoint SET_CUR sampling-frequency request (48 kHz). Background knowledge.

[12] Synology DSM 7.4 (DS416j): Task Scheduler and File Station API

Location: <https://www.synology.com> ; DSM Control Panel → Task Scheduler; SYNO.FileStation.Upload / Download

Used for: Native deployment: root boot-up task for the relay, files copied and logs read through File Station. DSM's lack of USB-audio drivers and of libusb-1.0 on this model. Verified on the NAS.

[13] Espressif esp_hosted (ESP-Hosted-MCU) host driver 1.4.0

Location: <https://github.com/espressif/esp-hosted-mcu> (host/drivers/transport/sdio/sdio_drv.c, mempool, os_wrapper)

Used for: SDIO receive path, streaming vs packet RX modes, the per-frame copy in sdio_process_rx_task(); the two network-radio patches are written against it. Verified (read, compiled, tested off-target).

[14] ESP-IDF heap capabilities and PSRAM malloc policy

Location: ESP-IDF v5.4 API reference: heap_caps_malloc, CONFIG_SPIRAM_MALLOC_ALWAYSINTERNAL

Used for: Why frame-sized mallocs land in internal RAM, and allocating the frame copies in PSRAM instead. Verified against the project's sdkconfig and on the Tab5.

[15] lwIP TCP receive window

Location: lwIP / ESP-IDF lwipopts.h (TCP_WND, ESP_PER_SOC_TCP_WND = 0)

Used for: Receive window sizing (global only in this IDF) and its effect on the stream rate. Verified.

[16] QRP Labs QMX / QMX+ operating manual

Location: <https://qrp-labs.com/qmx>

Used for: Menu path System config → GPS & Ser. ports → USB serial ports; terminal-mode warning about closing sessions. Referenced via the upstream code comments.

11. Driver listings (firmware)

Final firmware sources from `~/src/qmx-panadapter-v1.16`. The new network radio driver is listed in full; changes to existing upstream files are listed as unified diffs against tag v1.16.11. Long lines are wrapped (↵).

main/net/radio_relay.h (new)

```
#pragma once

/*
 * radio_relay - "Network radio" source for the QMX panadapter.
 *
 * Lets the Tab5 use a QMX that is NOT plugged into its own USB port but into
 * a Synology NAS. tools/nas-relay/qmx_nas_relay.py runs on the NAS (Docker /
 * Container Manager), owns the QMX through libusb, and streams the QMX's I/Q
 * audio and CAT bytes over one TCP connection using the small QMXR/1 framing:
 *
 *      u8 type | u8 flags | u16 len (LE) | payload
 *      1=HELLO 2=IQ 3=CAT 4=PING
 *      5=TERM_OPEN 6=TERM_STATUS 7=TERM_DATA 8=TERM_CLOSE   (relay says "term=1")
 *
 * IQ payloads are interleaved into 16 LE I/Q pairs at 48 kHz, already scaled
 * exactly like the USB path's (s24 >> 8), and are pushed into the same audio
 * ring buffer the UAC driver feeds, so DSP/FFT/FT8 see no difference.
 * CAT payloads go to/from the same Kenwood parser cat.c uses for USB CDC.
 *
 * An open TCP session == "radio present": when the NAS loses the QMX it
 * closes the socket and cat.c handles it exactly like a USB unplug.
 *
 * TERM_* carry the QMX's SECOND serial port (the one qmx_term.c uses for the
 * radio's own menus, interface 5 on USB) as a separate byte stream, so a
 * terminal session over the network still never touches the CAT port.
 */

#include <stdbool.h>
#include <stddef.h>
#include <stdint.h>
#include "esp_err.h"

#define RADIO_RELAY_DEFAULT_PORT 7355
#define RADIO_RELAY_HOST_LEN    64

/* CAT bytes from the radio. Same signature as the CDC-ACM data callback. */
typedef bool (*radio_relay_cat_rx_cb_t)(const uint8_t *data, size_t len, void *arg);
/* Called (from the relay RX task) when an established link drops on its own. */
typedef void (*radio_relay_gone_cb_t)(void);

/* Set the runtime relay target. Empty/NULL host disables the network source
 * (USB is used). Does not by itself drop an open link - see cat_reconnect(). */
void radio_relay_set_target(const char *host, uint16_t port);
void radio_relay_get_target(char *host, size_t cap, uint16_t *port);
bool radio_relay_enabled(void);

/* Connect + QMXR/1 handshake. Starts the RX task on success. */
esp_err_t radio_relay_connect(radio_relay_cat_rx_cb_t cat_cb,
                             radio_relay_gone_cb_t gone_cb,
                             uint32_t timeout_ms);
bool radio_relay_connected(void);

/* Send CAT bytes to the radio (thread-safe). */
esp_err_t radio_relay_send_cat(const uint8_t *data, size_t len, uint32_t timeout_ms);

/* Tear down the link (does NOT invoke gone_cb). Safe if not connected. */
void radio_relay_close(void);

/* ---- QMX second serial port (terminal) over the relay -----
 * true when the connected relay advertised "term=1" in its HELLO. */
bool radio_relay_term_supported(void);

/* Ask the relay to open the QMX's second serial port. data_cb gets the bytes
 * the radio sends on it (relay RX task context - must not block); gone_cb fires
 * if the port or the whole link goes away while open. Blocks up to timeout_ms
 * for the relay's answer. ESP_ERR_NOT_FOUND = the radio has no second port
 * (set to 1 USB serial port); radio_relay_term_error() has the relay's reason. */
esp_err_t radio_relay_term_open(radio_relay_cat_rx_cb_t data_cb,
                                radio_relay_gone_cb_t gone_cb,
                                uint32_t timeout_ms);
esp_err_t radio_relay_term_send(const uint8_t *data, size_t len, uint32_t timeout_ms);
/* Close the second port on the relay. Does NOT invoke gone_cb. */
void radio_relay_term_close(void);
const char *radio_relay_term_error(void);
```

```
/* Diagnostics for /api/relay. */
uint32_t radio_relay_iq_pairs_total(void);
const char *radio_relay_last_error(void);
```

main/net/radio_relay.c (new)

```
#include "radio_relay.h"

#include <string.h>
#include <stdio.h>
#include <errno.h>
#include <fcntl.h>

#include "esp_log.h"
#include "esp_timer.h"
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "freertos/semphr.h"
#include "lwip/sockets.h"
#include "lwip/netdb.h"

#include "audio.h"
#include "wifi.h" // wifi_is_connected(): no socket calls before the stack is up

static const char *TAG = "relay";

#define PROTO_HELLO 1
#define PROTO_IQ 2
#define PROTO_CAT 3
#define PROTO_PING 4
#define PROTO_TERM_OPEN 5 // Tab5 -> relay: open the QMX's second serial port
#define PROTO_TERM_STATUS 6 // relay -> Tab5: u8 ok (1/0) [+ reason text]
#define PROTO_TERM_DATA 7 // both ways: bytes on the second serial port
#define PROTO_TERM_CLOSE 8 // Tab5 -> relay: close it

#define RX_PAYLOAD_MAX 8192 // NAS relay sends <=1920-byte IQ frames; anything larger is skipped
#define RX_IDLE_TIMEOUT_S 3 // relay streams I/Q + 1 s PINGs; 3 s of silence == gone

static char s_host[RADIO_RELAY_HOST_LEN] = {0};
static uint16_t s_port = RADIO_RELAY_DEFAULT_PORT;
static portMUX_TYPE s_cfg_mux = portMUX_INITIALIZER_UNLOCKED;

static int s_fd = -1;
static SemaphoreHandle_t s_tx_mutex = NULL;
static TaskHandle_t s_rx_task = NULL;
static volatile bool s_connected = false;
static volatile bool s_closing = false;
static radio_relay_cat_rx_cb_t s_cat_cb = NULL;
static radio_relay_gone_cb_t s_gone_cb = NULL;
static volatile uint32_t s_iq_pairs_total = 0;
static char s_last_err[64] = "not configured";

// Second serial port (terminal) state.
static volatile bool s_term_supported = false;
static radio_relay_cat_rx_cb_t s_term_cb = NULL;
static radio_relay_gone_cb_t s_term_gone_cb = NULL;
static SemaphoreHandle_t s_term_sem = NULL;
static volatile int s_term_result = 0; // -1 waiting, 0 failed, 1 open
static char s_term_err[64] = "";
static int64_t s_last_fail_log_us = 0;

// ---- config -----

void radio_relay_set_target(const char *host, uint16_t port)
{
    portENTER_CRITICAL(&s_cfg_mux);
    // snprintf, not strncpy: IDF builds with -Werror=stringop-truncation.
    snprintf(s_host, sizeof(s_host), "%s", host ? host : "");
    s_port = port ? port : RADIO_RELAY_DEFAULT_PORT;
    portEXIT_CRITICAL(&s_cfg_mux);
    ESP_LOGI(TAG, "target: %s", s_host[0] ? s_host : "(none - USB)");
}

void radio_relay_get_target(char *host, size_t cap, uint16_t *port)
{
    portENTER_CRITICAL(&s_cfg_mux);
    if (host && cap) snprintf(host, cap, "%s", s_host);
    if (port) *port = s_port;
    portEXIT_CRITICAL(&s_cfg_mux);
}

bool radio_relay_enabled(void) { return s_host[0] != '\0'; }
bool radio_relay_connected(void) { return s_connected; }
```

```

uint32_t radio_relay_iq_pairs_total(void) { return s_iq_pairs_total; }
const char *radio_relay_last_error(void) { return s_last_err; }

// ---- socket helpers -----

static int recv_exact(int fd, void *buf, size_t n)
{
    uint8_t *p = (uint8_t *)buf;
    size_t got = 0;
    while (got < n) {
        int r = recv(fd, p + got, n - got, 0);
        if (r <= 0) return -1;
        got += (size_t)r;
    }
    return 0;
}

static esp_err_t send_frame(uint8_t type, const uint8_t *payload, size_t len, uint32_t timeout_ms)
{
    if (len > 0xFFFF) return ESP_ERR_INVALID_SIZE;
    /* [] Never from lwIP's own thread ("tiT"): a socket send there waits on the
     * tcpip thread, i.e. on itself, and the whole network stack stops for good
     * (the SNTP callback did exactly this with the QMX time push, 2026-10-04 -
     * see wifi.c sntp_sync_cb). Refuse loudly instead of hanging silently. */
    if (strcmp(pcTaskGetName(NULL), "tiT") == 0) {
        ESP_LOGE(TAG, "send from the tcpip thread refused (would deadlock lwIP)");
        return ESP_ERR_INVALID_STATE;
    }
    if (!s_tx_mutex || xSemaphoreTake(s_tx_mutex, pdMS_TO_TICKS(timeout_ms)) != pdTRUE) {
        return ESP_ERR_TIMEOUT;
    }
    esp_err_t ret = ESP_OK;
    int fd = s_fd;
    if (fd < 0 || s_closing) {
        ret = ESP_ERR_INVALID_STATE;
    } else {
        uint8_t hdr[4] = { type, 0, (uint8_t)(len & 0xFF), (uint8_t)(len >> 8) };
        const uint8_t *parts[2] = { hdr, payload };
        size_t lens[2] = { sizeof(hdr), len };
        for (int i = 0; i < 2 && ret == ESP_OK; i++) {
            size_t off = 0;
            while (off < lens[i]) {
                int n = send(fd, parts[i] + off, lens[i] - off, 0); // SO_SNDTIMEO bounds each
                if (n <= 0) { ret = (errno == EAGAIN) ? ESP_ERR_TIMEOUT : ESP_FAIL; break; }
                off += (size_t)n;
            }
        }
        xSemaphoreGive(s_tx_mutex);
        return ret;
    }
}

esp_err_t radio_relay_send_cat(const uint8_t *data, size_t len, uint32_t timeout_ms)
{
    if (!s_connected) return ESP_ERR_INVALID_STATE;
    return send_frame(PROTO_CAT, data, len, timeout_ms ? timeout_ms : 200);
}

// ---- RX task -----

static void rx_task(void *arg)
{
    (void)arg;
    // int16-aligned so IQ payloads can be handed to the audio ring directly.
    static int16_t payload16[RX_PAYLOAD_MAX / 2];
    uint8_t *payload = (uint8_t *)payload16;
    int fd = s_fd;

    while (!s_closing) {
        uint8_t hdr[4];
        if (recv_exact(fd, hdr, sizeof(hdr)) < 0) break;
        uint8_t type = hdr[0];
        size_t len = (size_t)hdr[2] | ((size_t)hdr[3] << 8);

        if (len > RX_PAYLOAD_MAX) {
            // Unknown/oversized frame: skip it in chunks to stay in sync.
            size_t left = len;
            bool ok = true;
            while (left && ok) {
                size_t n = left > RX_PAYLOAD_MAX ? RX_PAYLOAD_MAX : left;
                ok = recv_exact(fd, payload, n) == 0;
                left -= n;
            }
            if (!ok) break;
        }
    }
}

```

```

        continue;
    }
    if (len && recv_exact(fd, payload, len) < 0) break;

    switch (type) {
    case PROTO_IQ: {
        size_t pairs = len / 4;
        if (pairs) {
            audio_push_net_iq(payload16, pairs);
            s_iq_pairs_total += pairs;
        }
        break;
    }
    case PROTO_CAT:
        if (s_cat_cb && len) s_cat_cb(payload, len, NULL);
        break;
    case PROTO_TERM_DATA:
        if (s_term_cb && len) s_term_cb(payload, len, NULL);
        break;
    case PROTO_TERM_STATUS: {
        bool ok = len >= 1 && payload[0] == 1;
        if (len > 1) {
            size_t n = len - 1 < sizeof(s_term_err) - 1 ? len - 1 : sizeof(s_term_err) - 1;
            memcpy(s_term_err, payload + 1, n);
            s_term_err[n] = '\0';
        } else if (ok) {
            s_term_err[0] = '\0';
        }
        if (s_term_result == -1) {
            // The answer to our TERM_OPEN.
            s_term_result = ok ? 1 : 0;
            if (s_term_sem) xSemaphoreGive(s_term_sem);
        } else if (!ok && s_term_cb) {
            // Unsolicited "port gone" while a session is open.
            radio_relay_gone_cb_t g = s_term_gone_cb;
            s_term_cb = NULL;
            s_term_gone_cb = NULL;
            ESP_LOGW(TAG, "relay closed the terminal port: %s", s_term_err);
            if (g) g();
        }
        break;
    }
    default: // HELLO / PING / unknown: nothing to do
        break;
    }
}

bool unexpected = !s_closing;
s_connected = false;
if (s_term_result == -1 && s_term_sem) { // a TERM_OPEN still waiting
    s_term_result = 0;
    snprintf(s_term_err, sizeof(s_term_err), "network link dropped");
    xSemaphoreGive(s_term_sem);
}
if (s_term_cb) { // an open terminal loses its port too
    radio_relay_gone_cb_t g = s_term_gone_cb;
    s_term_cb = NULL;
    s_term_gone_cb = NULL;
    if (g) g();
}
if (unexpected) {
    snprintf(s_last_err, sizeof(s_last_err), "link dropped (errno %d)", errno);
    ESP_LOGW(TAG, "relay link dropped (errno %d)", errno);
}
s_rx_task = NULL;
if (unexpected && s_gone_cb) s_gone_cb();
vTaskDelete(NULL);
}

// ---- connect / close -----

static void note_fail(const char *what, int code)
{
    snprintf(s_last_err, sizeof(s_last_err), "%s (%d)", what, code);
    // link_task retries every ~2 s; don't flood the log.
    int64_t now = esp_timer_get_time();
    if (now - s_last_fail_log_us > 30LL * 1000 * 1000) {
        s_last_fail_log_us = now;
        ESP_LOGW(TAG, "connect to %s:%u failed: %s", s_host, s_port, s_last_err);
    }
}

esp_err_t radio_relay_connect(radio_relay_cat_rx_cb_t cat_cb,
                             radio_relay_gone_cb_t gone_cb,

```

```

        uint32_t timeout_ms)
{
    if (s_connected || s_rx_task) return ESP_ERR_INVALID_STATE;
    if (!s_tx_mutex) {
        s_tx_mutex = xSemaphoreCreateMutex();
        if (!s_tx_mutex) return ESP_ERR_NO_MEM;
    }

    char host[RADIO_RELAY_HOST_LEN];
    uint16_t port;
    radio_relay_get_target(host, sizeof(host), &port);
    if (!host[0]) return ESP_ERR_INVALID_STATE;

    /* [] No lwIP call until WiFi is up. link_task starts inside cat_init(),
     * seconds before wifi_init() has created the tcpip thread, and getaddrinfo()
     * on a stack that does not exist yet asserts in tcpip_send_msg_wait_sem
     * ("Invalid mbox") - a boot loop as soon as a NAS address is saved
     * (hardware-observed 2026-10-04, cat_link at 4.0 s uptime). Not connected
     * yet is an ordinary "try again": link_task retries every ~2 s. */
    if (!wifi_is_connected()) {
        note_fail("waiting for WiFi", 0);
        return ESP_ERR_INVALID_STATE;
    }

    char port_str[8];
    snprintf(port_str, sizeof(port_str), "%u", port);
    struct addrinfo hints = { .ai_family = AF_INET, .ai_socktype = SOCK_STREAM };
    struct addrinfo *res = NULL;
    int gai = getaddrinfo(host, port_str, &hints, &res);
    if (gai != 0 || !res) {
        note_fail("DNS/WiFi not ready", gai);
        if (res) freeaddrinfo(res);
        return ESP_ERR_NOT_FOUND;
    }

    int fd = socket(res->ai_family, res->ai_socktype, res->ai_protocol);
    if (fd < 0) {
        freeaddrinfo(res);
        note_fail("no free socket", errno);
        return ESP_ERR_NO_MEM;
    }

    // Non-blocking connect bounded by timeout_ms.
    int fl = fcntl(fd, F_GETFL, 0);
    fcntl(fd, F_SETFL, fl | O_NONBLOCK);
    int rc = connect(fd, res->ai_addr, res->ai_addrlen);
    freeaddrinfo(res);
    if (rc != 0 && errno != EINPROGRESS) {
        note_fail("connect", errno);
        close(fd);
        return ESP_FAIL;
    }
    if (rc != 0) {
        fd_set wfds;
        FD_ZERO(&wfds);
        FD_SET(fd, &wfds);
        struct timeval tv = { .tv_sec = timeout_ms / 1000, .tv_usec = (timeout_ms % 1000) * 1000 };
        int sel = select(fd + 1, NULL, &wfds, NULL, &tv);
        int soerr = 0;
        socklen_t sl = sizeof(soerr);
        if (sel <= 0 || getsockopt(fd, SOL_SOCKET, SO_ERROR, &soerr, &sl) != 0 || soerr != 0) {
            note_fail(sel <= 0 ? "connect timeout" : "connect refused", soerr);
            close(fd);
            return ESP_ERR_TIMEOUT;
        }
    }
    fcntl(fd, F_SETFL, fl & ~O_NONBLOCK);

    int one = 1;
    setsockopt(fd, IPPROTO_TCP, TCP_NODELAY, &one, sizeof(one));
    setsockopt(fd, SOL_SOCKET, SO_KEEPALIVE, &one, sizeof(one));
    struct timeval rto = { .tv_sec = RX_IDLE_TIMEOUT_S, .tv_usec = 0 };
    setsockopt(fd, SOL_SOCKET, SO_RCVTIMEO, &rto, sizeof(rto));
    struct timeval sto = { .tv_sec = 0, .tv_usec = 200 * 1000 };
    setsockopt(fd, SOL_SOCKET, SO_SNDTIMEO, &sto, sizeof(sto));

    s_fd = fd;
    s_closing = false;

    // Handshake: say hello, expect "QMXR/1 ..." back as the first frame. The
    // NAS relay only answers once it has actually opened the QMX, so a
    // completed handshake means the radio is there.
    static const char hello[] = "QMXR/1 tab5-qmx-panadapter";
    uint8_t hdr[4];

```

```

char resp[96];
size_t len;
if (send_frame(PROTO_HELLO, (const uint8_t *)hello, sizeof(hello) - 1, 500) != ESP_OK) {
    note_fail("hello send", errno);
    goto fail;
}
if (recv_exact(fd, hdr, sizeof(hdr)) < 0) { note_fail("no hello (QMX not on NAS?)", errno); goto fail; }
len = (size_t)hdr[2] | ((size_t)hdr[3] << 8);
if (hdr[0] != PROTO_HELLO || len == 0 || len >= sizeof(resp)) {
    note_fail("not a QMXR/1 relay", hdr[0]);
    goto fail;
}
if (recv_exact(fd, resp, len) < 0) { note_fail("hello recv", errno); goto fail; }
resp[len] = '\0';
if (strncmp(resp, "QMXR/1", 6) != 0) { note_fail("bad protocol", 0); goto fail; }
s_term_supported = strstr(resp, "term=1") != NULL;
if (!strstr(resp, "rate=48000")) {
    ESP_LOGW(TAG, "relay rate is not 48000 Hz (%s) - spectrum scale will be off", resp);
}

s_cat_cb = cat_cb;
s_gone_cb = gone_cb;
s_connected = true;
if (xTaskCreatePinnedToCore(rx_task, "relay_rx", 4096, NULL, 5, &s_rx_task, 1) != pdPASS) {
    s_connected = false;
    note_fail("rx task", 0);
    goto fail;
}
audio_request_reset();
snprintf(s_last_err, sizeof(s_last_err), "ok");
ESP_LOGI(TAG, "connected to %s:%u (%s)", host, port, resp);
return ESP_OK;

fail:
s_fd = -1;
close(fd);
return ESP_FAIL;
}

void radio_relay_close(void)
{
    s_closing = true;
    int fd = s_fd;
    if (fd >= 0) shutdown(fd, SHUT_RDWR);
    // Let the RX task notice and exit before the fd is released.
    for (int i = 0; i < 100 && s_rx_task != NULL; i++) vTaskDelay(pdMS_TO_TICKS(20));
    if (s_tx_mutex) xSemaphoreTake(s_tx_mutex, pdMS_TO_TICKS(500));
    if (s_fd >= 0) { close(s_fd); s_fd = -1; }
    if (s_tx_mutex) xSemaphoreGive(s_tx_mutex);
    s_connected = false;
    s_cat_cb = NULL;
    s_gone_cb = NULL;
    s_term_cb = NULL;
    s_term_gone_cb = NULL;
    s_term_supported = false;
    ESP_LOGI(TAG, "relay link closed");
}

// ---- second serial port (terminal) -----

bool radio_relay_term_supported(void) { return s_connected && s_term_supported; }
const char *radio_relay_term_error(void) { return s_term_err; }

esp_err_t radio_relay_term_open(radio_relay_cat_rx_cb_t data_cb,
                                radio_relay_gone_cb_t gone_cb,
                                uint32_t timeout_ms)
{
    if (!s_connected) return ESP_ERR_INVALID_STATE;
    if (!s_term_supported) {
        snprintf(s_term_err, sizeof(s_term_err), "NAS relay too old (no terminal support)");
        return ESP_ERR_NOT_SUPPORTED;
    }
    if (!s_term_sem) {
        s_term_sem = xSemaphoreCreateBinary();
        if (!s_term_sem) return ESP_ERR_NO_MEM;
    }
    xSemaphoreTake(s_term_sem, 0); // clear any stale give
    // Install the data path FIRST: the radio repaints as soon as it is open.
    s_term_cb = data_cb;
    s_term_gone_cb = gone_cb;
    s_term_result = -1;
    esp_err_t e = send_frame(PROTO_TERM_OPEN, NULL, 0, 500);
    if (e == ESP_OK && xSemaphoreTake(s_term_sem, pdMS_TO_TICKS(timeout_ms)) != pdTRUE) {
        snprintf(s_term_err, sizeof(s_term_err), "no answer from the NAS relay");
    }
}

```

```

    e = ESP_ERR_TIMEOUT;
}
if (e == ESP_OK && s_term_result != 1) e = ESP_ERR_NOT_FOUND;
if (e != ESP_OK) {
    s_term_cb = NULL;
    s_term_gone_cb = NULL;
    if (s_term_result == -1) s_term_result = 0;
    ESP_LOGW(TAG, "terminal port over the network not available: %s", s_term_err);
    return e;
}
ESP_LOGI(TAG, "QMX second serial port open on the NAS relay");
return ESP_OK;
}

esp_err_t radio_relay_term_send(const uint8_t *data, size_t len, uint32_t timeout_ms)
{
    if (!s_connected || !s_term_cb) return ESP_ERR_INVALID_STATE;
    return send_frame(PROTO_TERM_DATA, data, len, timeout_ms ? timeout_ms : 300);
}

void radio_relay_term_close(void)
{
    bool was_open = s_term_cb != NULL;
    s_term_cb = NULL;
    s_term_gone_cb = NULL;
    if (was_open && s_connected) send_frame(PROTO_TERM_CLOSE, NULL, 0, 300);
}

```

main/cat/cat.c (diff vs v1.16.11)

```

diff --git a/main/cat/cat.c b/main/cat/cat.c
--- a/main/cat/cat.c
+++ b/main/cat/cat.c
@@ -18,6 +18,8 @@
#include "usb/usb_host.h"
#include "usb/cdc_acm_host.h"
#include "radio_relay.h"
#include "audio.h" // audio_request_reset - network link teardown
#include "bsp/m5stack_tab5.h"

#include "wspr_rx.h" // wspr_pa_guard_release_pending - see the VN; handler
@@ -54,6 +56,21 @@ static const char *TAG = "cat";
static TaskHandle_t s_poll_task = NULL;
static EventGroupHandle_t s_evt_group = NULL;
static cdc_acm_dev_hdl_t s_cdc_dev = NULL;
// Network radio source (QMX plugged into a NAS, QMXR/1 relay - see
// net/radio_relay.h and tools/nas-relay). When true the CAT link is a TCP
// session to the NAS instead of the USB CDC-ACM device; every command path
// goes through cat_tx() so it doesn't care which. The QMX's optional SECOND
// serial port (terminal, qmx_term.c) is USB-only and simply isn't available
// over the network.
+static volatile bool s_net_link = false;
+#define CAT_LINK_UP() (s_cdc_dev != NULL || s_net_link)
+static esp_err_t cat_tx(const uint8_t *buf, size_t len, uint32_t timeout_ms)
+{
+    if (s_net_link) return radio_relay_send_cat(buf, len, timeout_ms);
+    cdc_acm_dev_hdl_t dev = s_cdc_dev;
+    if (!dev) return ESP_ERR_INVALID_STATE;
+    return cdc_acm_host_data_tx_blocking(dev, buf, len, timeout_ms);
+}
+static volatile bool s_cat_ready = false;

bool cat_is_ready(void)
@@ -448,7 +465,7 @@ static volatile bool s_tune_poll_active = false;
void cat_tune_poll_set_active(bool active) { s_tune_poll_active = active; }
esp_err_t cat_send_raw_cmd(const char *fmt, ...)
{
    if (!s_cdc_dev) return ESP_ERR_INVALID_STATE;
    if (!CAT_LINK_UP()) return ESP_ERR_INVALID_STATE;
    char buf[64];
    va_list ap;
    va_start(ap, fmt);
@@ -463,17 +480,17 @@ esp_err_t cat_send_raw_cmd(const char *fmt, ...)
    * 2026-08-29: a burst at a hand-set 7.5 V announced PA=15.0 V. A label that
    * can be stale is worse than one that admits it does not know. */
    if (strstr(buf, "Max. PA voltage")) s_pa_voltage_x10 = -1;
    return cdc_acm_host_data_tx_blocking(s_cdc_dev, (const uint8_t *)buf, len, 200);
+    return cat_tx((const uint8_t *)buf, len, 200);
}

esp_err_t cat_query_power_swr(float *power_w, float *swr)
{

```

```

-   if (!s_cdc_dev) return ESP_ERR_INVALID_STATE;
+   if (!CAT_LINK_UP()) return ESP_ERR_INVALID_STATE;

    s_pc_resp_len = 0;
    s_sw_resp_len = 0;

-   esp_err_t err = cdc_acm_host_data_tx_blocking(s_cdc_dev, (const uint8_t *) "PC;SW;", 6, 200);
+   esp_err_t err = cat_tx((const uint8_t *) "PC;SW;", 6, 200);
    if (err != ESP_OK) return err;

    for (int wi = 0; wi < 20 && (s_pc_resp_len == 0 || s_sw_resp_len == 0); wi++) {
@@ -512,10 +529,10 @@ esp_err_t cat_query_power_swr(float *power_w, float *swr)
    // keyed, same as the unchanged end-of-burst query.
    esp_err_t cat_pwr_swr_async_send(void)
    {
-   if (!s_cdc_dev) return ESP_ERR_INVALID_STATE;
+   if (!CAT_LINK_UP()) return ESP_ERR_INVALID_STATE;
        s_pc_resp_len = 0;
        s_sw_resp_len = 0;
-   return cdc_acm_host_data_tx_blocking(s_cdc_dev, (const uint8_t *) "PC;SW;", 6, 50);
+   return cat_tx((const uint8_t *) "PC;SW;", 6, 50);
    }

    // Parse whatever response has arrived since the last_send(). Returns ESP_OK
@@ -1259,7 +1276,7 @@ static esp_err_t try_open_qmx(void)
    cdc_acm_host_set_control_line_state(s_cdc_dev, true, true);
    ESP_LOGI(TAG, "QMX configured: %d baud, 8N1", CAT_BAUD_RATE);

-   cdc_acm_host_data_tx_blocking(s_cdc_dev, (const uint8_t *) "ID;", 3, 500);
+   cat_tx((const uint8_t *) "ID;", 3, 500);

    // Phase 3.1 – dump audio descriptors ONCE per session
    if (!s_audio_dumped) {
@@ -1404,7 +1421,7 @@ static void cw_filters_read(void)
    char q[40];
    int n = snprintf(q, sizeof(q), "MMCW|Choose filters|d;", i);
    s_mm_resp_len = 0;
-   if (n <= 0 || cdc_acm_host_data_tx_blocking(s_cdc_dev, (const uint8_t *) q,
+   if (n <= 0 || cat_tx((const uint8_t *) q,
                                (size_t)n, 200) != ESP_OK) {
        ESP_LOGW(TAG, "CW filters: write failed at index %d - offering all widths", i);
        s_cw_filter_mask = 0;
@@ -1441,7 +1458,7 @@ static bool iq_mode_handshake(int max_attempts);

bool cat_apply_cw_profile(uint16_t centre_hz, uint8_t mask)
{
-   if (!s_cdc_dev || !centre_hz) return false;
+   if (!CAT_LINK_UP() || !centre_hz) return false;
    s_pending_prof_mask = mask;
    s_pending_prof_centre = centre_hz; /* set LAST - it is the "go" flag */
    return true;
@@ -1472,12 +1489,12 @@ static bool cw_center_write_confirmed(uint16_t centre_hz)
    char cmd[40];
    int n = snprintf(cmd, sizeof(cmd), "MMCW|CW center=%u;", (unsigned)centre_hz);
    if (n > 0)
-       cdc_acm_host_data_tx_blocking(s_cdc_dev, (const uint8_t *) cmd, (size_t)n, 200);
+       cat_tx((const uint8_t *) cmd, (size_t)n, 200);
    vTaskDelay(pdMS_TO_TICKS(200)); /* let the menu redraw finish */

    s_mm_resp_len = 0;
    const char *q = "MMCW|CW center;";
-   if (cdc_acm_host_data_tx_blocking(s_cdc_dev, (const uint8_t *) q,
+   if (cat_tx((const uint8_t *) q,
                                strlen(q), 200) != ESP_OK) continue;
    for (int wi = 0; wi < 30 && s_mm_resp_len == 0; wi++) vTaskDelay(pdMS_TO_TICKS(10));
    if (s_mm_resp_len >= 4 && strcmp(s_mm_resp, "MM", 2) == 0 &&
@@ -1520,7 +1537,7 @@ static bool cw_profile_apply_pending(void)
    int n = snprintf(cmd, sizeof(cmd), "MMCW|Choose filters|d=%s;",
                                i, (mask & (1u << i)) ? "ENABLED" : "DISABLED");
    if (n > 0)
-       cdc_acm_host_data_tx_blocking(s_cdc_dev, (const uint8_t *) cmd, (size_t)n, 200);
+       cat_tx((const uint8_t *) cmd, (size_t)n, 200);
    /* 120 ms, not 40: an MM write sprays a menu redraw back at us and the
     * next command must not arrive into it. 40 ms was measured refusing the
     * write that followed the eighth row. */
@@ -1535,7 +1552,7 @@ static bool cw_profile_apply_pending(void)
    * on the WSPR PA guard, where the read-back agreed and the radio still ran
    * at full power. */
    const char *mu = "MU;";
-   cdc_acm_host_data_tx_blocking(s_cdc_dev, (const uint8_t *) mu, 3, 200);
+   cat_tx((const uint8_t *) mu, 3, 200);
    vTaskDelay(pdMS_TO_TICKS(250));

    /* □ AND MU; DROPS IQ MODE. Q9 is session state, so the reload leaves the

```

```

@@ -1570,7 +1587,7 @@ static bool cw_offset_refresh(void)

    const char *q = "MMCW|CW offset;";
    s_mm_resp_len = 0;
-   if (cdc_acm_host_data_tx_blocking(s_cdc_dev, (const uint8_t *)q, strlen(q), 200) != ESP_OK) {
+   if (cat_tx((const uint8_t *)q, strlen(q), 200) != ESP_OK){
        return true;    // transient; the poll's own error handling owns the link
    }
    for (int wi = 0; wi < 10 && s_mm_resp_len == 0; wi++) vTaskDelay(pdMS_TO_TICKS(10));
@@ -1638,11 +1655,11 @@ static bool cw_split_maintain(void)
    if (base != 0) {
        char fb[20];
        int n = snprintf(fb, sizeof fb, "FB%011lu;", (unsigned long)base);
-       efb = cdc_acm_host_data_tx_blocking(s_cdc_dev, (const uint8_t *)fb, (size_t)n, 200);
+       efb = cat_tx((const uint8_t *)fb, (size_t)n, 200);
        vTaskDelay(pdMS_TO_TICKS(30));
    }
    // Now stand down: back to simplex, transmitting where we listen.
-   esp_err_t e = cdc_acm_host_data_tx_blocking(s_cdc_dev, (const uint8_t *)"SP0;", 4, 200);
+   esp_err_t e = cat_tx((const uint8_t *)"SP0;", 4, 200);
    // Δ AND PUT THE VFO MODE BACK, which SP0; does NOT do. The QMX has a single
    // three-state VFO Mode - A / B / Split - and the CAT manual is explicit that
    // it is FR/FT that select it: "0, 1, 2 correspond to VFO A, VFO B or Split
@@ -1653,7 +1670,7 @@ static bool cw_split_maintain(void)
    // been "fixed" twice: B matched A, split read off, and the display was still
    // A/B. FR0; is what actually returns the radio to plain VFO A.
    vTaskDelay(pdMS_TO_TICKS(30));
-   esp_err_t efr = cdc_acm_host_data_tx_blocking(s_cdc_dev, (const uint8_t *)"FR0;", 4, 200);
+   esp_err_t efr = cat_tx((const uint8_t *)"FR0;", 4, 200);
    // The manual says FR and FT both select the same three-state VFO Mode, so send
    // BOTH - and then ASK, because FR0; alone demonstrably did not clear the A/B
    // display and I am not going to guess a fourth time. The answers arrive as
@@ -1661,16 +1678,16 @@ static bool cw_split_maintain(void)
    // for "FA"), which is enough to tell whether the mode really is 0 and the
    // display is driven by something else, or the write is simply not landing.
    vTaskDelay(pdMS_TO_TICKS(30));
-   cdc_acm_host_data_tx_blocking(s_cdc_dev, (const uint8_t *)"FT0;", 4, 200);
+   cat_tx((const uint8_t *)"FT0;", 4, 200);
    vTaskDelay(pdMS_TO_TICKS(30));
-   cdc_acm_host_data_tx_blocking(s_cdc_dev, (const uint8_t *)"FR;", 3, 200);
+   cat_tx((const uint8_t *)"FR;", 3, 200);
    vTaskDelay(pdMS_TO_TICKS(30));
-   cdc_acm_host_data_tx_blocking(s_cdc_dev, (const uint8_t *)"FT;", 3, 200);
+   cat_tx((const uint8_t *)"FT;", 3, 200);
    // Ask the radio to confirm simplex, judged on a later cycle above.
    vTaskDelay(pdMS_TO_TICKS(30));
    s_split_readback = -1;
    s_split_verify_pending = true;
-   cdc_acm_host_data_tx_blocking(s_cdc_dev, (const uint8_t *)"SP;", 3, 200);
+   cat_tx((const uint8_t *)"SP;", 3, 200);
    s_split_engaged = false;
    s_split_warned = false;
    ESP_LOGI(TAG, "CW TX offset off - VFO B -> %lu (%s), split off (%s), VFO mode A (%s)",
@@ -1699,20 +1716,20 @@ static bool cw_split_maintain(void)
    if (tx < 0) tx = 0;
    char cmd[20];
    int n = snprintf(cmd, sizeof cmd, "FB%011lld;", (long long)tx);
-   esp_err_t e1 = cdc_acm_host_data_tx_blocking(s_cdc_dev, (const uint8_t *)cmd, (size_t)n, 200);
+   esp_err_t e1 = cat_tx((const uint8_t *)cmd, (size_t)n, 200);
    vTaskDelay(pdMS_TO_TICKS(30));
    // SP1 is re-sent with every FB, not just on the first one: it is one short
    // command, and it is the difference between "the offset is applied" and
    // "we quietly transmitted on top of the station" if the radio dropped split
    // while we were not looking.
-   esp_err_t e2 = cdc_acm_host_data_tx_blocking(s_cdc_dev, (const uint8_t *)"SP1;", 4, 200);
+   esp_err_t e2 = cat_tx((const uint8_t *)"SP1;", 4, 200);
    // Ask what actually happened. The answer lands asynchronously in
    // s_split_readback and is judged on a later cycle, so this costs one short
    // write and no waiting.
    vTaskDelay(pdMS_TO_TICKS(30));
    s_split_readback = -1;
    s_split_warned = false;
-   cdc_acm_host_data_tx_blocking(s_cdc_dev, (const uint8_t *)"SP;", 3, 200);
+   cat_tx((const uint8_t *)"SP;", 3, 200);
    s_split_engaged = true;
    s_split_base_hz = base;
    s_split_last_us = now;
@@ -1739,8 +1756,7 @@ static bool iq_mode_handshake(int max_attempts)
    s_iq_mode_confirmed = false;
    for (int attempt = 1; attempt <= max_attempts; attempt++) {
        const char *iq_on = "Q9 1;";
-       esp_err_t terr = cdc_acm_host_data_tx_blocking(
-           s_cdc_dev, (const uint8_t *)iq_on, 5, 200);
+       esp_err_t terr = cat_tx((const uint8_t *)iq_on, 5, 200);
    }

```

```

    if (terr == ESP_OK) {
        ESP_LOGI(TAG, "QMX IQ mode enabled (Q9 1;) attempt %d/%d",
            attempt, max_attempts);
    }
@@ -1762,8 +1778,7 @@ static bool iq_mode_handshake(int max_attempts)
    vTaskDelay(pdMS_TO_TICKS(150));
    s_q9_resp_len = 0;
    const char *iq_q = "Q9;";
    esp_err_t qerr = cdc_acm_host_data_tx_blocking(
-    s_cdc_dev, (const uint8_t *)iq_q, strlen(iq_q), 200);
+    esp_err_t qerr = cat_tx((const uint8_t *)iq_q, strlen(iq_q), 200);
    if (qerr == ESP_OK) {
        for (int wi = 0; wi < 20 && s_q9_resp_len == 0; wi++) {
            vTaskDelay(pdMS_TO_TICKS(20));
        }
    }
@@ -1829,7 +1844,7 @@ static void poll_task(void *arg)

    int phase = 0;
    int poll_fail = 0; // consecutive poll-TX failures; one transient timeout must not kill the poll
-    while (s_cdc_dev != NULL) {
+    while (CAT_LINK_UP()) {
        /* RX watchdog - see the note above the declaration. Checked once a
         * second so the arithmetic costs nothing at the 50 ms poll rate, and
         * only while we are actually polling: a paused link (FT8 burst, the
@@ -1913,7 +1928,7 @@ static void poll_task(void *arg)
        s_pending_mode_digit = 0;
        char cmd[8];
        cmd[0] = 'M'; cmd[1] = 'D'; cmd[2] = md; cmd[3] = ';'; cmd[4] = 0;
-        esp_err_t err = cdc_acm_host_data_tx_blocking(s_cdc_dev, (const uint8_t *)cmd, 4, 200);
+        esp_err_t err = cat_tx((const uint8_t *)cmd, 4, 200);
        ESP_LOGI(TAG, "mode -> MD%c; (%s)", md, err == ESP_OK ? "ok" : "fail");
        vTaskDelay(pdMS_TO_TICKS(CAT_POLL_INTERVAL_MS));
        continue;
    }
@@ -1921,7 +1936,7 @@ static void poll_task(void *arg)
    if (s_af_gain_query_pending) {
        s_af_gain_query_pending = false;
        static const char q[] = "AG;";
-        esp_err_t err = cdc_acm_host_data_tx_blocking(s_cdc_dev, (const uint8_t *)q, 3, 200);
+        esp_err_t err = cat_tx((const uint8_t *)q, 3, 200);
        ESP_LOGI(TAG, "AF gain query AG; (%s)", err == ESP_OK ? "sent" : "fail");
        vTaskDelay(pdMS_TO_TICKS(CAT_POLL_INTERVAL_MS));
        continue;
    }
@@ -1936,7 +1951,7 @@ static void poll_task(void *arg)
    // is in 0.25 dB steps, range 0-799 per the manual.
    char cmd[16];
    int n = snprintf(cmd, sizeof cmd, "AG%03u;", ag);
-    esp_err_t err = cdc_acm_host_data_tx_blocking(s_cdc_dev, (const uint8_t *)cmd,
+    esp_err_t err = cat_tx((const uint8_t *)cmd,
        (size_t)n, 200);
    ESP_LOGI(TAG, "AF gain -> %s (%.2f dB) (%s)", cmd, ag * 0.25,
        err == ESP_OK ? "ok" : "fail");
    }
@@ -1947,7 +1962,7 @@ static void poll_task(void *arg)
    if (s_rf_gain_query_pending) {
        s_rf_gain_query_pending = false;
        static const char q[] = "RG;";
-        esp_err_t err = cdc_acm_host_data_tx_blocking(s_cdc_dev, (const uint8_t *)q, 3, 200);
+        esp_err_t err = cat_tx((const uint8_t *)q, 3, 200);
        ESP_LOGI(TAG, "RF gain query RG; (%s)", err == ESP_OK ? "sent" : "fail");
        vTaskDelay(pdMS_TO_TICKS(CAT_POLL_INTERVAL_MS));
        continue;
    }
@@ -1959,21 +1974,21 @@ static void poll_task(void *arg)
    // covers hz == 0 on its own, which is how RIT gets switched off.
    char cmd[24];
    int n = snprintf(cmd, sizeof cmd, "RC;");
-    esp_err_t err = cdc_acm_host_data_tx_blocking(s_cdc_dev, (const uint8_t *)cmd,
+    esp_err_t err = cat_tx((const uint8_t *)cmd,
        (size_t)n, 200);

    if (err == ESP_OK && hz != 0) {
        vTaskDelay(pdMS_TO_TICKS(20));
        unsigned mag = (unsigned)(hz < 0 ? -hz : hz);
        n = snprintf(cmd, sizeof cmd, "%s%03u;", hz > 0 ? "RU" : "RD", mag);
-        err = cdc_acm_host_data_tx_blocking(s_cdc_dev, (const uint8_t *)cmd,
+        err = cat_tx((const uint8_t *)cmd,
            (size_t)n, 200);
        vTaskDelay(pdMS_TO_TICKS(20));
        n = snprintf(cmd, sizeof cmd, "RT1;");
-        cdc_acm_host_data_tx_blocking(s_cdc_dev, (const uint8_t *)cmd, (size_t)n, 200);
+        cat_tx((const uint8_t *)cmd, (size_t)n, 200);
    } else if (err == ESP_OK) {
        vTaskDelay(pdMS_TO_TICKS(20));
        n = snprintf(cmd, sizeof cmd, "RT0;");
-        cdc_acm_host_data_tx_blocking(s_cdc_dev, (const uint8_t *)cmd, (size_t)n, 200);
+        cat_tx((const uint8_t *)cmd, (size_t)n, 200);
    }
    if (err == ESP_OK) s_rit_hz = hz;
    ESP_LOGI(TAG, "RIT -> %d Hz (%s)", hz, err == ESP_OK ? "ok" : "fail");

```

```

@@ -1991,7 +2006,7 @@ static void poll_task(void *arg)
    // re-reads on open rather than replaying a stored number).
    char cmd[16];
    int n = snprintf(cmd, sizeof cmd, "RG%03u;", rg);
    esp_err_t err = cdc_acm_host_data_tx_blocking(s_cdc_dev, (const uint8_t *)cmd,
+
    esp_err_t err = cat_tx((const uint8_t *)cmd,
                                (size_t)n, 200);
    ESP_LOGI(TAG, "RF gain -> %s (%s)", cmd, err == ESP_OK ? "ok" : "fail");
    // Confirm it from HERE, after the value has gone out. Queued from the
@@ -2012,7 +2027,7 @@ static void poll_task(void *arg)
    s_split_query_pending = false;
    if (!s_split_engaged) {
        s_split_readback = -1;
        cdc_acm_host_data_tx_blocking(s_cdc_dev, (const uint8_t *)"SP;", 3, 200);
+
        cat_tx((const uint8_t *)"SP;", 3, 200);
        vTaskDelay(pdMS_TO_TICKS(CAT_POLL_INTERVAL_MS));
        continue;
    }
@@ -2021,7 +2036,7 @@ static void poll_task(void *arg)
    s_pa_query_pending = false;
    const char *q = "MMPProtection|Max. PA voltage;";
    s_pa_awaiting_reply = true;
    if (cdc_acm_host_data_tx_blocking(s_cdc_dev, (const uint8_t *)q,
+
    if (cat_tx((const uint8_t *)q,
                                strlen(q), 200) != ESP_OK) {
        s_pa_awaiting_reply = false; /* nothing is coming */
    }
@@ -2034,7 +2049,7 @@ static void poll_task(void *arg)
    char mm[48];
    int n = snprintf(mm, sizeof(mm), "MMPProtection|Max. PA voltage=%u.%u;",
                                (unsigned)(pav / 10), (unsigned)(pav % 10));
    esp_err_t e = cdc_acm_host_data_tx_blocking(s_cdc_dev, (const uint8_t *)mm, n, 200);
+
    esp_err_t e = cat_tx((const uint8_t *)mm, n, 200);
    ESP_LOGW(TAG, "PA voltage -> %u.%u V (%s)", (unsigned)(pav / 10),
                                (unsigned)(pav % 10), e == ESP_OK ? "ok" : "fail");
    /* □ AND NOW MAKE THE RADIO ACTUALLY USE IT.
@@ -2059,7 +2074,7 @@ static void poll_task(void *arg)
    if (e == ESP_OK) {
        vTaskDelay(pdMS_TO_TICKS(120));
        const char *mu = "MU;";
        if (cdc_acm_host_data_tx_blocking(s_cdc_dev, (const uint8_t *)mu, 3, 200) == ESP_OK) {
+
            if (cat_tx((const uint8_t *)mu, 3, 200) == ESP_OK) {
                ESP_LOGW(TAG, "PA voltage: MU; sent - config reloaded so the "
                                "new limit takes effect (MM Effect may be 'On demand')");
                vTaskDelay(pdMS_TO_TICKS(250));
@@ -2086,10 +2101,10 @@ static void poll_task(void *arg)
    // Write both to the same value: live applies AND there's nothing
    // for the FW; poll to revert to. Result sticks and persists.
    int n = snprintf(mm, sizeof(mm), "MMSB|Filter RX=%lu;", (unsigned long)bw);
    esp_err_t e1 = cdc_acm_host_data_tx_blocking(s_cdc_dev, (const uint8_t *)mm, n, 200);
+
    esp_err_t e1 = cat_tx((const uint8_t *)mm, n, 200);
    vTaskDelay(pdMS_TO_TICKS(40));
    n = snprintf(mm, sizeof(mm), "MMSB|Bandwidth=%lu;", (unsigned long)bw);
    esp_err_t e2 = cdc_acm_host_data_tx_blocking(s_cdc_dev, (const uint8_t *)mm, n, 200);
+
    esp_err_t e2 = cat_tx((const uint8_t *)mm, n, 200);
    ESP_LOGI(TAG, "SSB filter -> %lu Hz (RX=%s, BW=%s)", (unsigned long)bw,
                                e1 == ESP_OK ? "ok" : "fail", e2 == ESP_OK ? "ok" : "fail");
    vTaskDelay(pdMS_TO_TICKS(CAT_POLL_INTERVAL_MS));
@@ -2100,7 +2115,7 @@ static void poll_task(void *arg)
    s_pending_cw_passband = 0;
    char mm[32];
    int n = snprintf(mm, sizeof(mm), "MMCW|CW passband=%lu;", (unsigned long)cwbw);
    esp_err_t e = cdc_acm_host_data_tx_blocking(s_cdc_dev, (const uint8_t *)mm, n, 200);
+
    esp_err_t e = cat_tx((const uint8_t *)mm, n, 200);
    ESP_LOGI(TAG, "CW passband -> %lu Hz (%s)", (unsigned long)cwbw,
                                e == ESP_OK ? "ok" : "fail");
    vTaskDelay(pdMS_TO_TICKS(CAT_POLL_INTERVAL_MS));
@@ -2189,8 +2204,7 @@ static void poll_task(void *arg)
    CAT_POLL_INTERVAL_MS, (unsigned long)s_last_freq_hz, cat_get_mode_str());
}
}
esp_err_t err = cdc_acm_host_data_tx_blocking(
    s_cdc_dev, (const uint8_t *)cmd, cmd_len, 200);
+
esp_err_t err = cat_tx((const uint8_t *)cmd, cmd_len, 200);
if (err != ESP_OK) {
    // A CDC TX failure burst must NEVER permanently kill the poll
    // task. Root-caused 2026-07-10 (serial-log proven): opening any
@@ -2285,7 +2299,7 @@ static void band_scan_restore(uint32_t pre_freq, char pre_mode_digit)
    snprintf(fa, sizeof(fa), "FA%011lu;", (unsigned long)plan.freq_hz);
    ESP_LOGW(TAG, "the band scan left the radio on %lu Hz - restoring %lu Hz",
                                (unsigned long)s_last_freq_hz, (unsigned long)plan.freq_hz);
    if (cdc_acm_host_data_tx_blocking(s_cdc_dev, (const uint8_t *)fa,
+
    if (cat_tx((const uint8_t *)fa,
                                strlen(fa), 200) == ESP_OK) {

```

```

        s_last_freq_hz = plan.freq_hz;
        ui_update_frequency(plan.freq_hz);
@@ -2299,7 +2313,7 @@ static void band_scan_restore(uint32_t pre_freq, char pre_mode_digit)
    snprintf(md, sizeof(md), "MD%c;", plan.mode_digit);
    ESP_LOGW(TAG, "the band scan left the radio in mode '%c' - restoring '%c'",
        s_last_mode_digit ? s_last_mode_digit : '?', plan.mode_digit);
-    if (cdc_acm_host_data_tx_blocking(s_cdc_dev, (const uint8_t *)md,
+    if (cat_tx((const uint8_t *)md,
        strlen(md), 200) != ESP_OK)
        ESP_LOGW(TAG, "restore of mode '%c' could not be sent", plan.mode_digit);
    }
@@ -2319,7 +2333,7 @@ static void band_scan_restore(uint32_t pre_freq, char pre_mode_digit)
    * and the mode. */
    void cat_band_scan_restore_test(uint32_t pre_freq, char pre_mode_digit)
    {
-    if (!s_cdc_dev) {
+    if (!CAT_LINK_UP()) {
        ESP_LOGW(TAG, "restore test: no CAT link");
        return;
    }
@@ -2332,10 +2346,44 @@ void cat_band_scan_restore_test(uint32_t pre_freq, char pre_mode_digit)
    band_scan_restore(pre_freq, pre_mode_digit);
}

+static void net_gone_cb(void)
+{
+    ESP_LOGW(TAG, "QMX network link dropped");
+    xEventGroupSetBits(s_evt_group, EVT_DEV_GONE);
+}
+
+// Network counterpart of try_open_qmx(): connect to the QMXR/1 relay running
+// on the NAS the QMX is plugged into. CAT bytes go through the same handle_rx()
+// queue the CDC driver feeds, so cat_rx_task parses them identically.
+static esp_err_t try_open_net(void)
+{
+    xEventGroupClearBits(s_evt_group, EVT_DEV_GONE);
+    esp_err_t err = radio_relay_connect(handle_rx, net_gone_cb, 3000);
+    if (err != ESP_OK) return err;
+    s_net_link = true;
+    ESP_LOGI(TAG, "QMX CAT link via network (NAS relay)");
+    cat_tx((const uint8_t *)"ID;", 3, 500);
+    return ESP_OK;
+}
+
+void cat_reconnect(void)
+{
+    // Only meaningful while a link is up; a stale GONE bit would otherwise
+    // kill the next session the moment it opened.
+    if (s_evt_group && CAT_LINK_UP()) xEventGroupSetBits(s_evt_group, EVT_DEV_GONE);
+}
+
+const char *cat_get_transport(void)
+{
+    if (s_net_link) return "net";
+    if (s_cdc_dev) return "usb";
+    return "none";
+}
+
+static void link_task(void *arg)
+{
+    while (1) {
-        esp_err_t err = try_open_qmx();
+        esp_err_t err = radio_relay_enabled() ? try_open_net() : try_open_qmx();
+        if (err == ESP_OK) {
+            s_rx_len = 0;
+            s_last_freq_hz = 0;
@@ -2410,8 +2458,7 @@ static void link_task(void *arg)
            const int VOX_MAX_ATTEMPTS = 3;
            for (int attempt = 1; attempt <= VOX_MAX_ATTEMPTS; attempt++) {
                const char *vox_off = "Q3 0;";
-                esp_err_t terr = cdc_acm_host_data_tx_blocking(
-                    s_cdc_dev, (const uint8_t *)vox_off, 5, 200);
+                esp_err_t terr = cat_tx((const uint8_t *)vox_off, 5, 200);
+                if (terr != ESP_OK) {
                    ESP_LOGW(TAG, "Failed to send VOX-off (Q3 0;) attempt %d/%d: 0x%x",
                        attempt, VOX_MAX_ATTEMPTS, terr);
@@ -2423,8 +2470,7 @@ static void link_task(void *arg)
                vTaskDelay(pdMS_TO_TICKS(150));
                s_q3_resp_len = 0;
                const char *vox_q = "Q3;";
-                esp_err_t qerr = cdc_acm_host_data_tx_blocking(
-                    s_cdc_dev, (const uint8_t *)vox_q, strlen(vox_q), 200);
+                esp_err_t qerr = cat_tx((const uint8_t *)vox_q, strlen(vox_q), 200);
+                if (qerr == ESP_OK) {

```

```

        for (int w = 0; w < 20 && s_q3_resp_len == 0; w++) {
            vTaskDelay(pdMS_TO_TICKS(20));
@@ -2464,8 +2510,7 @@ static void link_task(void *arg)
    static const char *const warmup_cmds[] = { "FA;", "MD;", "FW;" };
    for (size_t wi = 0; wi < sizeof(warmup_cmds) / sizeof(warmup_cmds[0]); wi++) {
        const char *cmd = warmup_cmds[wi];
-       esp_err_t werr = cdc_acm_host_data_tx_blocking(
-       s_cdc_dev, (const uint8_t *)cmd, strlen(cmd), 200);
+       esp_err_t werr = cat_tx((const uint8_t *)cmd, strlen(cmd), 200);
        if (werr != ESP_OK) {
            ESP_LOGW(TAG, "Warmup %s send failed: 0x%x", cmd, werr);
            break;
@@ -2479,8 +2524,7 @@ static void link_task(void *arg)
    // differently (IQ output, TA TX, MMSB filter tokens).
    {
        const char *vn_q = "VN;";
-       esp_err_t verr = cdc_acm_host_data_tx_blocking(
-       s_cdc_dev, (const uint8_t *)vn_q, strlen(vn_q), 200);
+       esp_err_t verr = cat_tx((const uint8_t *)vn_q, strlen(vn_q), 200);
        if (verr == ESP_OK) {
            vTaskDelay(pdMS_TO_TICKS(100)); // response handled in process_cat_message
        } else {
@@ -2500,8 +2544,7 @@ static void link_task(void *arg)
    * - an unidentifiable radio keeps today's behaviour. */
    if (cat_qmx_fw_at_least(1, 4, 3)) {
        const char *ui_q = "UI;";
-       esp_err_t uerr = cdc_acm_host_data_tx_blocking(
-       s_cdc_dev, (const uint8_t *)ui_q, strlen(ui_q), 200);
+       esp_err_t uerr = cat_tx((const uint8_t *)ui_q, strlen(ui_q), 200);
        if (uerr == ESP_OK) vTaskDelay(pdMS_TO_TICKS(100)); /* handled in process_cat_message */
        else ESP_LOGW(TAG, "Failed to query unique id (UI): 0x%x", uerr);
    } else {
@@ -2512,8 +2555,7 @@ static void link_task(void *arg)
    // Read CW offset from QMX menu (session value, EEPROM-persisted on QMX side)
    {
        const char *cw_q = "MMCW|CW offset;";
-       esp_err_t cerr = cdc_acm_host_data_tx_blocking(
-       s_cdc_dev, (const uint8_t *)cw_q, strlen(cw_q), 200);
+       esp_err_t cerr = cat_tx((const uint8_t *)cw_q, strlen(cw_q), 200);
        if (cerr == ESP_OK) {
            vTaskDelay(pdMS_TO_TICKS(100));
            // Response is in s_mm_resp - parse MMnnn;
@@ -2572,7 +2614,7 @@ static void link_task(void *arg)
    * undo a split we did not engage. */
    {
        s_rx_vfo_mode = -1;
-       cdc_acm_host_data_tx_blocking(s_cdc_dev, (const uint8_t *)"FR;", 3, 200);
+       cat_tx((const uint8_t *)"FR;", 3, 200);
        for (int wi = 0; wi < 10 && s_rx_vfo_mode < 0; wi++) {
            vTaskDelay(pdMS_TO_TICKS(20));
        }
@@ -2585,10 +2627,10 @@ static void link_task(void *arg)
    ESP_LOGW(TAG, "VFO mode: radio was receiving on %s - the Tab5 only "
        "reads and writes VFO A, so nothing it did would have "
        "been audible. Switching to VFO A.", was);
-       cdc_acm_host_data_tx_blocking(s_cdc_dev, (const uint8_t *)"FR0;", 4, 200);
+       cat_tx((const uint8_t *)"FR0;", 4, 200);
        vTaskDelay(pdMS_TO_TICKS(60));
        s_rx_vfo_mode = -1;
-       cdc_acm_host_data_tx_blocking(s_cdc_dev, (const uint8_t *)"FR;", 3, 200);
+       cat_tx((const uint8_t *)"FR;", 3, 200);
        for (int wi = 0; wi < 10 && s_rx_vfo_mode < 0; wi++) {
            vTaskDelay(pdMS_TO_TICKS(20));
        }
@@ -2635,8 +2677,7 @@ static void link_task(void *arg)
    * value. */
    const char *gq = "MMSystem config|GPS & Ser. ports|GPS source;";
    s_mm_resp_len = 0;
-       esp_err_t gerr = cdc_acm_host_data_tx_blocking(
-       s_cdc_dev, (const uint8_t *)gq, strlen(gq), 200);
+       esp_err_t gerr = cat_tx((const uint8_t *)gq, strlen(gq), 200);
    if (gerr == ESP_OK) {
        for (int wi = 0; wi < 15 && s_mm_resp_len == 0; wi++) {
            vTaskDelay(pdMS_TO_TICKS(20));
@@ -2701,7 +2742,7 @@ static void link_task(void *arg)
    char pre_mode_digit = 0;
    {
        const char *q = "FA;MD;";
-       if (cdc_acm_host_data_tx_blocking(s_cdc_dev, (const uint8_t *)q,
+       if (cat_tx((const uint8_t *)q,
            strlen(q), 200) == ESP_OK) {
            for (int wi = 0; wi < 25 && !s_last_freq_hz; wi++)
                vTaskDelay(pdMS_TO_TICKS(20));
@@ -2739,8 +2780,7 @@ static void link_task(void *arg)

```

```

        snprintf(qname, sizeof(qname), "MMBAnd config.|Band name (m)[%d];", bi);
        s_rx_len = 0;
        s_mm_resp_len = 0;
-       esp_err_t be = cdc_acm_host_data_tx_blocking(
-           s_cdc_dev, (const uint8_t *)qname, strlen(qname), 200);
+       esp_err_t be = cat_tx((const uint8_t *)qname, strlen(qname), 200);
        if (be != ESP_OK) { transport_error = true; break; }
        for (int wi = 0; wi < 20 && s_mm_resp_len == 0; wi++) {
            vTaskDelay(pdMS_TO_TICKS(20));
@@ -2763,8 +2803,7 @@ static void link_task(void *arg)
    // Query center frequency
    char qfreq[56];
    snprintf(qfreq, sizeof(qfreq), "MMBAnd config.|Frequency center[%d];", bi);
-    be = cdc_acm_host_data_tx_blocking(
-        s_cdc_dev, (const uint8_t *)qfreq, strlen(qfreq), 200);
+    be = cat_tx((const uint8_t *)qfreq, strlen(qfreq), 200);
    if (be != ESP_OK) { transport_error = true; break; }
    for (int wi = 0; wi < 20 && s_mm_resp_len == 0; wi++) {
        vTaskDelay(pdMS_TO_TICKS(20));
@@ -2816,6 +2855,19 @@ static void link_task(void *arg)
    xEventGroupWaitBits(s_evt_group, EVT_DEV_GONE,
        pdTRUE, pdFALSE, portMAX_DELAY);
    ESP_LOGW(TAG, "QMX gone, cleaning up");
+    if (s_net_link) {
+        // Same ordering rules as the USB branch below: drop the link
+        // flag first so poll_task exits, wait for it, then close.
+        s_net_link = false;
+        s_cat_ready = false;
+        int wait_ms = 0;
+        while (s_poll_task != NULL && wait_ms < 4000) {
+            vTaskDelay(pdMS_TO_TICKS(20));
+            wait_ms += 20;
+        }
+        radio_relay_close();
+        audio_request_reset();
+    }
    if (s_cdc_dev) {
        cdc_acm_dev_hdl_t dev = s_cdc_dev;
        // Clear the handle FIRST so poll_task's "while (s_cdc_dev !=
@@ -2853,7 +2905,7 @@ static void link_task(void *arg)

    esp_err_t cat_set_frequency(uint32_t freq_hz)
    {
-        if (s_cdc_dev == NULL) {
+        if (!CAT_LINK_UP()) {
            return ESP_ERR_INVALID_STATE; // QMX not connected
        }

@@ -2912,7 +2964,7 @@ esp_err_t cat_set_frequency(uint32_t freq_hz)
    ESP_LOGW(TAG, "cat_set_frequency: snprintf produced %d chars (expected 14)", n);
    return ESP_FAIL;
}
-    esp_err_t err = cdc_acm_host_data_tx_blocking(s_cdc_dev, (const uint8_t *)cmd, 14, 200);
+    esp_err_t err = cat_tx((const uint8_t *)cmd, 14, 200);
    if (err != ESP_OK) {
        ESP_LOGW(TAG, "FA TX failed: 0x%x", err);
        return err;
@@ -3033,7 +3085,7 @@ esp_err_t cat_set_mode(const char *mode)
    ESP_LOGW(TAG, "cat_set_mode: unknown mode '%s'", mode ? mode : "(null)");
    return ESP_ERR_INVALID_ARG;
}
-    if (s_cdc_dev == NULL) return ESP_ERR_INVALID_STATE;
+    if (!CAT_LINK_UP()) return ESP_ERR_INVALID_STATE;
    // □ WAS THE ONE BLOCKING CDC WRITER IN THIS FILE WITH NO s_poll_paused
    // GUARD (found 2026-09-16, chasing Randy N40PI's "web Apply hangs and
    // loses the QMX, only during an active QSO/CQ exchange, needs a power
@@ -3056,7 +3108,7 @@ esp_err_t cat_set_mode(const char *mode)

    char cmd[8];
    cmd[0] = 'M'; cmd[1] = 'D'; cmd[2] = digit; cmd[3] = ';'; cmd[4] = 0;
-    esp_err_t err = cdc_acm_host_data_tx_blocking(s_cdc_dev, (const uint8_t *)cmd, 4, 200);
+    esp_err_t err = cat_tx((const uint8_t *)cmd, 4, 200);
    if (err != ESP_OK) {
        ESP_LOGW(TAG, "MD TX failed: 0x%x", err);
        return err;
@@ -3068,7 +3120,7 @@ esp_err_t cat_set_mode(const char *mode)
    esp_err_t cat_set_passband_hz(uint32_t hz)
    {
        if (hz < 50 || hz > 9999) return ESP_ERR_INVALID_ARG;
-        if (s_cdc_dev == NULL) return ESP_ERR_INVALID_STATE;
+        if (!CAT_LINK_UP()) return ESP_ERR_INVALID_STATE;

        uint64_t now = esp_timer_get_time();
        if (now - s_last_tx_us < 200000) return ESP_ERR_TIMEOUT;

```

```

@@ -3081,7 +3133,7 @@ esp_err_t cat_set_passband_hz(uint32_t hz)
    ESP_LOGW(TAG, "cat_set_passband_hz: snprintf produced %d chars (expected 7)", n);
    return ESP_FAIL;
}
-   esp_err_t err = cdc_acm_host_data_tx_blocking(s_cdc_dev, (const uint8_t *)cmd, 7, 200);
+   esp_err_t err = cat_tx((const uint8_t *)cmd, 7, 200);
    if (err != ESP_OK) {
        ESP_LOGW(TAG, "FW TX failed: 0x%x", err);
        return err;
    }
@@ -3100,7 +3152,7 @@ esp_err_t cat_set_qmx_time(int hour, int min, int sec)

    esp_err_t cat_query_qmx_time(int *out_hour, int *out_min, int *out_sec)
    {
-       if (!s_cdc_dev || !s_cat_ready) return ESP_ERR_INVALID_STATE;
+       if (!CAT_LINK_UP() || !s_cat_ready) return ESP_ERR_INVALID_STATE;

        /* NOT WHILE A BURST IS KEYED. This was missing, and it is what let the
         * RTC read clear a WSPR burst's hold 30 s into a 110.6 s transmission -
@@ -3117,7 +3169,7 @@ esp_err_t cat_query_qmx_time(int *out_hour, int *out_min, int *out_sec)

        cat_poll_hold(CAT_HOLD_TIME_SYNC);
        s_tm_resp_len = 0;
-       esp_err_t err = cdc_acm_host_data_tx_blocking(s_cdc_dev, (const uint8_t *)"TM;", 3, 200);
+       esp_err_t err = cat_tx((const uint8_t *)"TM;", 3, 200);
        if (err == ESP_OK) {
            for (int i = 0; i < 10 && s_tm_resp_len == 0; i++) {
                vTaskDelay(pdMS_TO_TICKS(20));
@@ -3171,7 +3223,7 @@ static bool parse_tm_resp(int *h, int *m, int *s)
// already owns the CDC pipe (FT8 TX pauses the same poll flag).
esp_err_t cat_gps_tick_sync(int *out_hour, int *out_min, int *out_sec, int64_t *out_flip_us)
{
-   if (!s_cdc_dev || !s_cat_ready) return ESP_ERR_INVALID_STATE;
+   if (!CAT_LINK_UP() || !s_cat_ready) return ESP_ERR_INVALID_STATE;
    if (cat_poll_held_by_other(CAT_HOLD_TIME_SYNC)) {
        ESP_LOGI(TAG, "GPS tick skipped - %s holds the link", hold_name(s_poll_holds));
        return ESP_ERR_INVALID_STATE; // a burst owns the pipe
@@ -3186,7 +3238,7 @@ esp_err_t cat_gps_tick_sync(int *out_hour, int *out_min, int *out_sec, int64_t *

    while (esp_timer_get_time() - start < 1300000) { // ~1.3 s cap: spans any 1 s boundary
        s_tm_resp_len = 0;
-       if (cdc_acm_host_data_tx_blocking(s_cdc_dev, (const uint8_t *)"TM;", 3, 100) != ESP_OK)
+       if (cat_tx((const uint8_t *)"TM;", 3, 100) != ESP_OK)
            break;
        // Wait briefly for the async RX handler to stamp + fill the response.
        for (int i = 0; i < 25 && s_tm_resp_len == 0; i++) vTaskDelay(pdMS_TO_TICKS(2));
@@ -3248,6 +3300,20 @@ esp_err_t cat_gps_tick_sync(int *out_hour, int *out_min, int *out_sec, int64_t *
// while (s_cdc_dev != NULL) lets it exit, wait for it, and only then close.
void cat_usb_shutdown(void)
{
+   if (s_net_link) {
+       ESP_LOGI(TAG, "shutdown: returning the networked radio to RX");
+       const char *rx_net = "TA0:RX;";
+       radio_relay_send_cat((const uint8_t *)rx_net, strlen(rx_net), 200);
+       s_net_link = false;
+       s_cat_ready = false;
+       int wait_ms = 0;
+       while (s_poll_task != NULL && wait_ms < 1500) {
+           vTaskDelay(pdMS_TO_TICKS(20));
+           wait_ms += 20;
+       }
+       radio_relay_close();
+       return;
+   }
    cdc_acm_dev_hdl_t dev = s_cdc_dev;
    if (!dev) {
        ESP_LOGI(TAG, "shutdown: no CAT device open");

```

main/cat/cat.h (diff vs v1.16.11)

```

diff --git a/main/cat/cat.h b/main/cat/cat.h
--- a/main/cat/cat.h
+++ b/main/cat/cat.h
@@ -563,3 +563,15 @@ bool cat_force_rx_pending(void);
 * Reached from POST /api/cmd {"action":"band_scan_restore_test", ...}. Nothing
 * calls it in normal operation; it sends no TX. */
void cat_band_scan_restore_test(uint32_t pre_freq, char pre_mode_digit);
+
+/**
+ * @brief Drop the current CAT/IQ link so link_task re-selects the transport.
+ *
+ * Call after changing the network-radio target (radio_relay_set_target()):
+ * link_task uses the QMX on USB when no NAS host is set, otherwise the QMX on
+ * the NAS (tools/nas-relay). No-op when no link is up.

```

```
+ */
+void cat_reconnect(void);
+
+/** @brief "usb", "net" or "none" - which transport the CAT link is using. */
+const char *cat_get_transport(void);
```

main/audio/audio.c (diff vs v1.16.11)

```
diff --git a/main/audio/audio.c b/main/audio/audio.c
--- a/main/audio/audio.c
+++ b/main/audio/audio.c
@@ -19,6 +19,7 @@
#include "iq_balance.h"
#include "psram_task.h"
#include "ui.h"
+#include "radio_relay.h"

static const char *TAG = "audio";

@@ -645,6 +646,48 @@ static bool process_rx(void)
}
}

+// Network radio source (net/radio_relay.c): the same post-decode path as
+// process_rx(), minus the 24-bit unpack the NAS relay already did.
+void audio_push_net_iq(const int16_t *iq, size_t pairs)
+{
+    static int16_t buf[2048 * 2];
+    if (!s_ring || !iq || pairs == 0) return;
+
+    if (s_flat_reset_pending) {
+        s_flat_reset_pending = false;
+        ui_flat_mode_reset();
+    }
+
+    int16_t local_peak_L = s_peak_left;
+    int16_t local_peak_R = s_peak_right;
+    while (pairs) {
+        size_t n = pairs > 2048 ? 2048 : pairs;
+        for (size_t i = 0; i < n; i++) {
+            int16_t Is = iq[2*i];
+            int16_t Qs = iq[2*i + 1];
+            int16_t aL = (Is < 0) ? (int16_t)(Is == -32768 ? 32767 : -Is) : Is;
+            int16_t aR = (Qs < 0) ? (int16_t)(Qs == -32768 ? 32767 : -Qs) : Qs;
+            if (aL > local_peak_L) local_peak_L = aL;
+            if (aR > local_peak_R) local_peak_R = aR;
+            iq_balance_apply(&Is, &Qs);
+            buf[2*i] = Is;
+            buf[2*i + 1] = Qs;
+        }
+        s_samples_this_period += n;
+        if (xRingbufferSend(s_ring, buf, n * sizeof(int16_t) * 2, 0) == pdTRUE) {
+            s_pairs_written += n;
+        } else {
+            s_dropped_this_period += n;
+            s_dropped_total += n;
+        }
+        dsp_rxaudio_forward(buf, n); // RX audio demodulator, as process_rx() does
+        iq += 2 * n;
+        pairs -= n;
+    }
+    s_peak_left = local_peak_L;
+    s_peak_right = local_peak_R;
+}

static esp_err_t open_and_start(uint8_t addr, uint8_t iface_num)
{
    const uac_host_device_config_t dev_cfg = {
@@ -716,7 +759,11 @@ static void audio_task(void *arg)
if (xQueueReceive(s_evt_queue, &e, 0) == pdTRUE) {
    switch (e.kind) {
        case AE_RX_CONNECTED:
-            if (s_uac_dev == NULL) {
+            if (radio_relay_enabled()) {
+                // Network radio (QMX on the NAS) selected: a locally plugged
+                // QMX would double-feed the sample ring, so leave it idle.
+                ESP_LOGW(TAG, "USB audio device ignored - network radio source is selected");
            } else if (s_uac_dev == NULL) {
                // A new UAC session starts on a frame boundary; anything
                // held from the old one belongs to a stream that is gone.
                audio_reset_frame_alignment();
```

main/audio/audio.h (diff vs v1.16.11)

```
diff --git a/main/audio/audio.h b/main/audio/audio.h
--- a/main/audio/audio.h
+++ b/main/audio/audio.h
@@ -52,6 +52,18 @@ bool audio_uac_active(void);
 */
void audio_request_reset(void);

+/**
+ * @brief Push already-decoded I/Q pairs from the network radio source
+ * (QMX on a NAS, see net/radio_relay.h).
+ *
+ * Input is interleaved int16 I/Q, scaled exactly like the USB path's
+ * (s24 >> 8). Applies IQ balance and feeds the same ring buffer (and the RX
+ * audio demodulator) the UAC path feeds, so FFT/FT8/WSPR/RX audio can't tell
+ * the difference. Never blocks: drops on ring-full like the USB path.
+ * Called from the relay RX task.
+ */
+void audio_push_net_iq(const int16_t *iq, size_t pairs);
+
// Drop any partial 6-byte I/Q frame held from a previous UAC session. Called
// when a session starts or ends - a new stream begins on a frame boundary, and
// carrying a fragment across would misalign the interleave from the first
```

main/qmx_term.c (diff vs v1.16.11)

```
diff --git a/main/qmx_term.c b/main/qmx_term.c
--- a/main/qmx_term.c
+++ b/main/qmx_term.c
@@ -9,6 +9,7 @@
#include "freertos/semphr.h"
#include "util/psram_task.h"
#include "cat.h" /* cat_request_iq_reassert - see close_locked */
+#include "net/radio_relay.h" /* the second port over the network (QMX on a NAS) */
#include "ui.h" /* ui_flat_mode_reset */
#include <string.h>
#include <stdio.h>
@@ -58,6 +59,10 @@ static uint32_t s_pre_freq_hz;
static char s_pre_mode[12];

static cdc_acm_dev_hdl_t s_dev;
+/* True when the session runs over the NAS relay instead of USB: the QMX is
+ * plugged into a NAS (tools/nas-relay), and the relay opens the radio's second
+ * serial port for us and carries it as its own stream, separate from CAT. */
+static volatile bool s_net;
static ansi_term_t *s_scr; /* PSRAM: ~4 KB of cells */
static bool s_open;
static int64_t s_last_activity_us;
@@ -111,6 +116,7 @@ static bool on_rx(const uint8_t *data, size_t len, void *arg)

static bool tx(const char *s, int len)
{
+ if (s_net) return radio_relay_term_send((const uint8_t *)s, (size_t)len, 300) == ESP_OK;
+ if (!s_dev) return false;
+ return cdc_acm_host_data_tx_blocking(s_dev, (const uint8_t *)s, len, 300) == ESP_OK;
}
@@ -264,6 +270,7 @@ static void close_locked(void)
} else {
s_open = false;
}
+ if (s_net) { radio_relay_term_close(); s_net = false; }
+ if (s_dev) { cdc_acm_host_close(s_dev); s_dev = NULL; }

/* □ Hand the radio back the way the operator-pause resume does, because a
@@ -342,6 +349,15 @@ static void on_cdc_event(const cdc_acm_host_dev_event_data_t *ev, void *arg)
if (s_dev) { cdc_acm_host_close(s_dev); s_dev = NULL; }
}

+/* The network counterpart of the DISCONNECTED event: the relay lost the port or
+ * the whole link. Runs on the relay RX task - flags only, nothing that blocks. */
+static void on_net_gone(void)
+{
+ ESP_LOGW(TAG, "the network radio went away with a terminal session open");
+ s_open = false;
+ s_net = false;
+}
+
static void idle_task(void *arg)
{
(void)arg;
@@ -390,6 +406,26 @@ bool qmx_term_open(void)
snprintf(s_pre_mode, sizeof(s_pre_mode), "%s", (m && m[0]) ? m : "");
}
+ const char *transport = cat_get_transport();
```

```

+   if (transport && strcmp(transport, "net") == 0) {
+       /* The QMX is on a NAS: the relay owns the USB device and opens port 2
+        * for us. Same retry reasoning as below does not apply - the relay
+        * answers definitively (open, or "the radio has one serial port"). */
+       if (!radio_relay_term_supported()) {
+           ESP_LOGW(TAG, "network radio: the NAS relay is too old to carry the "
+               "terminal port - update tools/nas-relay on the NAS");
+           xSemaphoreGive(s_lock);
+           return false;
+       }
+       esp_err_t ne = radio_relay_term_open(on_rx, on_net_gone, 3000);
+       if (ne != ESP_OK) {
+           ESP_LOGW(TAG, "network radio: port 2 would not open: %s - is the radio "
+               "set to 2 USB serial ports?", radio_relay_term_error());
+           xSemaphoreGive(s_lock);
+           return false;
+       }
+       s_net = true;
+   } else {
+       const cdc_acm_host_device_config_t cfg = {
+           .connection_timeout_ms = 1000,
+           .out_buffer_size = 64,
@@ -441,6 +477,7 @@ bool qmx_term_open(void)
+       };
+       cdc_acm_host_line_coding_set(s_dev, &lc);
+       cdc_acm_host_set_control_line_state(s_dev, true, true);
+   } /* USB */

+   s_open = true;
+   s_last_activity_us = esp_timer_get_time();
@@ -506,7 +543,8 @@ bool qmx_term_open(void)
+   s_idle_task = psram_task_create(idle_task, "qmx_term_idle", 3072, NULL, 2,
+       tskNO_AFFINITY);
+   }
+   ESP_LOGI(TAG, "terminal session open on interface %d", QMX_TERM_INTERFACE);
+   if (s_net) ESP_LOGI(TAG, "terminal session open on port 2 via the NAS relay");
+   else ESP_LOGI(TAG, "terminal session open on interface %d", QMX_TERM_INTERFACE);
+   xSemaphoreGive(s_lock);
+   return true;
+ }

```

main/storage/settings.c (diff vs v1.16.11)

```

diff --git a/main/storage/settings.c b/main/storage/settings.c
--- a/main/storage/settings.c
+++ b/main/storage/settings.c
@@ -69,6 +69,8 @@ static const char *TAG = "settings";
+ #define KEY_QRZ_UPLOADED "qrz_upl_n"
+ #define KEY_EQSL_USER "eqsl_user"
+ #define KEY_EQSL_PSWD "eqsl_pswd"
+ #define KEY_RELAY_HOST "relay_host"
+ #define KEY_RELAY_PORT "relay_port"
+ #define KEY_QRZ_LU_USER "qrz_lu_user"
+ #define KEY_QRZ_LU_PASS "qrz_lu_pass"
+ #define KEY_EQSL_UPLOADED "eqsl_upl_n"
@@ -415,7 +417,8 @@ static inline bool dirty_test_any(const dirty_t *d, const uint8_t *bits, size_t
+ #define DIRTY_RXA_RELEASE 130 /* AGC release time, ms */
+ #define DIRTY_RXA_AGCOFF 131 /* AGC bypass */
+ #define DIRTY_HP_MUTE 132 /* speaker auto-mute on headphones */
+ #define DIRTY_WIFI_STATIC_SSID 133 /* which SSID the static IP is for; 26 spare */
+ #define DIRTY_WIFI_STATIC_SSID 133 /* which SSID the static IP is for */
+ #define DIRTY_RELAY 134 /* network radio: relay_host + relay_port; 25 spare */

+ // Bits that actually affect config_io_export()'s output (storage/config_io.c).
+ // Bookkeeping bits like DIRTY_LAST_TIME (rewritten every FT8 slot by the
@@ -436,6 +439,7 @@ static const uint8_t s_config_export_bits[] = {
+   DIRTY_CQ_MSG2, DIRTY_CQ_SEL, DIRTY_ONBOARDED, DIRTY_FT8_FILT,
+   DIRTY_WIFI_ENABLED, DIRTY_QMX_GPS, DIRTY_FREQ_KP_CALC,
+   DIRTY_QRZ_KEY, DIRTY_QRZ_LU_USER, DIRTY_QRZ_LU_PASS, DIRTY_EQSL_USER, DIRTY_EQSL_PSWD,
+   DIRTY_RELAY,
+   DIRTY_CL_URL, DIRTY_CL_KEY, DIRTY_CL_STATION,
+   DIRTY_WF_BLACK, DIRTY_WF_CONTRAST, DIRTY_WF_BLEND, DIRTY_WF_WINDOW,
+   DIRTY_DISP_FLIP, DIRTY_QMX_VOL, DIRTY_RX_AUD_VOL, DIRTY_CHARGE_LIM_EN,
@@ -598,6 +602,10 @@ static void flush_task(void *arg)
+   if (dirty_test(&dirty_local, DIRTY_QRZ_UPLOADED)) nvs_set_u32(s_nvs, KEY_QRZ_UPLOADED, snap.qrz_uploaded_n);
+   if (dirty_test(&dirty_local, DIRTY_EQSL_USER)) nvs_set_str(s_nvs, KEY_EQSL_USER, snap.eqsl_user);
+   if (dirty_test(&dirty_local, DIRTY_EQSL_PSWD)) nvs_set_str(s_nvs, KEY_EQSL_PSWD, snap.eqsl_pswd);
+   if (dirty_test(&dirty_local, DIRTY_RELAY)) {
+       nvs_set_str(s_nvs, KEY_RELAY_HOST, snap.relay_host);
+       nvs_set_u16(s_nvs, KEY_RELAY_PORT, snap.relay_port);
+   }
+   if (dirty_test(&dirty_local, DIRTY_QRZ_LU_USER)) nvs_set_str(s_nvs, KEY_QRZ_LU_USER, snap.qrz_lookup_user);
+   if (dirty_test(&dirty_local, DIRTY_QRZ_LU_PASS)) nvs_set_str(s_nvs, KEY_QRZ_LU_PASS, snap.qrz_lookup_pass);

```

```

        if (dirty_test(&dirty_local, DIRTY_EQSL_UPLOADED)) nvs_set_u32(s_nvs, KEY_EQSL_UPLOADED, snap.eqsl_uploaded_n
    );
@@ -866,6 +874,8 @@ static void load_from_nvs(qmx_settings_t *out)
    out->qrz_lookup_pass[0] = '\0';
    out->eqsl_user[0] = '\0';
    out->eqsl_pswd[0] = '\0';
+   out->relay_host[0] = '\0';
+   out->relay_port = 7355;
    out->eqsl_uploaded_n = 0;
    out->cloudlog_url[0] = '\0';
    out->cloudlog_key[0] = '\0';
@@ -1073,6 +1083,12 @@ static void load_from_nvs(qmx_settings_t *out)
    out->eqsl_pswd[0] = '\0';
    sz = sizeof(out->eqsl_pswd);
    nvs_get_str(s_nvs, KEY_EQSL_PSWD, out->eqsl_pswd, &sz);
+   {
+       size_t rsz = sizeof(out->relay_host);
+       if (nvs_get_str(s_nvs, KEY_RELAY_HOST, out->relay_host, &rsz) != ESP_OK) out->relay_host[0] = '\0';
+       uint16_t rp = 0;
+       if (nvs_get_u16(s_nvs, KEY_RELAY_PORT, &rp) == ESP_OK && rp != 0) out->relay_port = rp;
+   }
    nvs_get_u32(s_nvs, KEY_EQSL_UPLOADED, &out->eqsl_uploaded_n);
    out->cloudlog_url[0] = '\0';
    sz = sizeof(out->cloudlog_url);
@@ -3542,3 +3558,13 @@ void settings_set_lotw_uploaded_n(uint32_t n)
    xSemaphoreGive(s_mutex);
    mark_dirty(DIRTY_LOTW_UPLOADED);
}
+
+void settings_set_relay(const char *host, uint16_t port)
+{
+   if (!s_ready) return;
+   xSemaphoreTake(s_mutex, portMAX_DELAY);
+   snprintf(s_pending.relay_host, sizeof(s_pending.relay_host), "%s", host ? host : "");
+   s_pending.relay_port = port ? port : 7355;
+   xSemaphoreGive(s_mutex);
+   mark_dirty(DIRTY_RELAY);
+}

```

main/storage/settings.h (diff vs v1.16.11)

```

diff --git a/main/storage/settings.h b/main/storage/settings.h
--- a/main/storage/settings.h
+++ b/main/storage/settings.h
@@ -601,6 +601,10 @@ typedef struct {
    char    lotw_state[4];    // 2-letter US state, e.g. "VA"
    char    lotw_county[40]; // county name only, e.g. "Arlington"
    uint32_t lotw_uploaded_n; // count of ADIF records already uploaded to LoTW
+   // Network radio: IP/host of the NAS the QMX is plugged into, running
+   // tools/nas-relay. Empty = QMX on the Tab5's own USB port (default).
+   char    relay_host[64];
+   uint16_t relay_port;    // network radio TCP port (default 7355)
} qmx_settings_t;

// Initialise the settings module. Opens an NVS handle. Safe to call
@@ -1004,6 +1008,10 @@ void settings_set_qrz_uploaded_n(uint32_t n);
void settings_set_eqsl_user(const char *user);
void settings_set_eqsl_pswd(const char *pswd);

+// Network radio (QMX on a NAS via tools/nas-relay instead of USB). Persists
+// only; callers also call radio_relay_set_target() + cat_reconnect().
+void settings_set_relay(const char *host, uint16_t port);
+
+// QRZ.com Callbook (XML) lookup credentials, set via the web UI (debounced
+// flush, one field per call like the eQSL setters above). Pass NULL or empty
+// to clear. See the struct field comment in this header for why this is

```

main/storage/config_io.c (diff vs v1.16.11)

```

diff --git a/main/storage/config_io.c b/main/storage/config_io.c
--- a/main/storage/config_io.c
+++ b/main/storage/config_io.c
@@ -116,6 +116,8 @@ char *config_io_export(size_t *out_len)
    APP("qrz_lookup_pass    = %s\n", c.qrz_lookup_pass);
    APP("eqsl_user          = %s\n", c.eqsl_user);
    APP("eqsl_pass           = %s\n", c.eqsl_pswd);
+   APP("relay_host         = %s\n", c.relay_host);    // QMX on a NAS (tools/nas-relay); empty = QMX on USB
+   APP("relay_port        = %u\n", (unsigned)c.relay_port);
    APP("cloudlog_url       = %s\n", c.cloudlog_url);
    APP("cloudlog_key       = %s\n", c.cloudlog_key);
    APP("cloudlog_station   = %s\n", c.cloudlog_station);
@@ -454,6 +456,15 @@ int config_io_import(char *text)
    else if (!strcasecmp(key, "qrz_lookup_pass")) settings_set_qrz_lookup_pass(val);

```

```

        else if (!strcasecmp(key, "eqsl_user"))          settings_set_eqsl_user(val);
        else if (!strcasecmp(key, "eqsl_pass"))          settings_set_eqsl_pswd(val);
+       else if (!strcasecmp(key, "relay_host") || !strcasecmp(key, "relay_port")) {
+       qmx_settings_t *rc = heap_caps_malloc(sizeof(*rc), MALLOC_CAP_SPIRAM | MALLOC_CAP_8BIT);
+       if (rc) {
+       settings_load_all(rc);
+       if (!strcasecmp(key, "relay_host")) settings_set_relay(val, rc->relay_port);
+       else
+       settings_set_relay(rc->relay_host, (uint16_t)atoi(val));
+       heap_caps_free(rc);
+       }
+       else if (!strcasecmp(key, "cloudlog_url"))        settings_set_cloudlog_url(val);
+       else if (!strcasecmp(key, "cloudlog_key"))        settings_set_cloudlog_key(val);
+       else if (!strcasecmp(key, "cloudlog_station"))    settings_set_cloudlog_station(val);

```

main/main.c (diff vs v1.16.11)

```

diff --git a/main/main.c b/main/main.c
--- a/main/main.c
+++ b/main/main.c
@@ -17,6 +17,7 @@
#include "bsp_info.h"
#include "gpio_relay.h"
#include "cat.h"
+include "radio_relay.h"
#include "cw_decode.h"
#include "audio.h"
#include "rx_audio.h"
@@ -477,6 +478,17 @@ void app_main(void)
}
cw_decode_init();
/* Network radio: if a NAS relay host is configured, cat.c/audio.c use the
QMX plugged into the NAS (tools/nas-relay) instead of the Tab5's USB
port. Heap, not stack - same reasoning as the cfg block above. */
{
qmx_settings_t *rs = heap_caps_malloc(sizeof(*rs), MALLOC_CAP_SPIRAM | MALLOC_CAP_8BIT);
if (rs) {
settings_load_all(rs);
radio_relay_set_target(rs->relay_host, rs->relay_port);
heap_caps_free(rs);
}
}
ESP_ERROR_CHECK(cat_init());
MEM_LEDGER("cat_init (CDC)");

```

main/net/webserver.c (diff vs v1.16.11)

```

diff --git a/main/net/webserver.c b/main/net/webserver.c
--- a/main/net/webserver.c
+++ b/main/net/webserver.c
@@ -79,6 +79,7 @@
#include <ctype.h>
#include "net/manual_embed.h" // manual_embed_get - /api/manual serves the built-in manual
#include "config_io.h" // config_io_export / config_io_import
+include "radio_relay.h" // network radio source (QMX on a NAS)
#include "usb_replug.h" // usb_replug (hidden /api/cmd recovery action)
#include "gpio_relay.h" // gpio_pulse - remote relay for a QMX power cycle
#include "util/usb_shutdown.h" // usb_shutdown_graceful - "prepare for flashing"
@@ -515,6 +516,7 @@ static esp_err_t status_handler(httpd_req_t *req)
}

cJSON_AddNumberToObject(root, "freq_hz", (double)cat_get_frequency());
+ cJSON_AddStringToObject(root, "radio_link", cat_get_transport()); // "usb" / "net" / "none"
+ cJSON_AddStringToObject(root, "qmx_fw", cat_get_qmx_fw());
+ cJSON_AddStringToObject(root, "mode", ui_get_mode_str());
// Decoded CW, EXACTLY the line the Tab5 is drawing - same grid, same wrap
@@ -3695,6 +3697,7 @@ static esp_err_t eqsl_upload_handler(httpd_req_t *req)
return httpd_resp_sendstr(req, "{\"status\":\"uploading\"}");
}

+// GET /api/relay -> {"host","port","enabled","connected","transport","iq_pairs","last_error"}
+// POST /api/relay {"host":"192.168.1.20","port":7355} (host "" = back to USB)
+// Network radio: the QMX plugged into a NAS running tools/nas-relay. Applies
+// immediately: persists, retargets, and drops the current link so cat.c's
+// link_task reconnects over the newly selected transport.
+static esp_err_t relay_get_handler(httpd_req_t *req)
+{
+ char host[RADIO_RELAY_HOST_LEN];
+ uint16_t port;
+ radio_relay_get_target(host, sizeof(host), &port);
+ cJSON *root = cJSON_CreateObject();
+ if (!root) return httpd_resp_send_500(req);

```

```

+ cJSON_AddStringToObject(root, "host", host);
+ cJSON_AddNumberToObject(root, "port", port);
+ cJSON_AddBoolToObject(root, "enabled", radio_relay_enabled());
+ cJSON_AddBoolToObject(root, "connected", radio_relay_connected());
+ cJSON_AddStringToObject(root, "transport", cat_get_transport());
+ cJSON_AddNumberToObject(root, "iq_pairs", (double)radio_relay_iq_pairs_total());
+ cJSON_AddStringToObject(root, "last_error", radio_relay_last_error());
+ char *out = cJSON_PrintUnformatted(root);
+ cJSON_Delete(root);
+ if (!out) return httpd_resp_send_500(req);
+ httpd_resp_set_type(req, "application/json");
+ esp_err_t err = httpd_resp_sendstr(req, out);
+ cJSON_free(out);
+ return err;
+}
+
+static esp_err_t relay_post_handler(httpd_req_t *req)
+{
+    char buf[160];
+    int len = httpd_req_recv(req, buf, sizeof(buf) - 1);
+    if (len < 0) {
+        httpd_resp_send_err(req, HTTPD_400_BAD_REQUEST, "no body");
+        return ESP_FAIL;
+    }
+    buf[len] = '\0';
+    cJSON *root = cJSON_Parse(buf);
+    if (!root) {
+        httpd_resp_send_err(req, HTTPD_400_BAD_REQUEST, "bad json");
+        return ESP_FAIL;
+    }
+    const char *host = cJSON_GetStringValue(cJSON_GetObjectItem(root, "host"));
+    cJSON *jp = cJSON_GetObjectItem(root, "port");
+    int port = cJSON_IsNumber(jp) ? jp->valueint : RADIO_RELAY_DEFAULT_PORT;
+    if (port <= 0 || port > 65535 || (host && strlen(host) >= RADIO_RELAY_HOST_LEN)) {
+        cJSON_Delete(root);
+        httpd_resp_send_err(req, HTTPD_400_BAD_REQUEST, "bad host/port");
+        return ESP_FAIL;
+    }
+    settings_set_relay(host ? host : "", (uint16_t)port);
+    radio_relay_set_target(host ? host : "", (uint16_t)port);
+    cJSON_Delete(root);
+    cat_reconnect();
+    return relay_get_handler(req);
+}
+
+static const httpd_uri_t uri_relay_get = {
+    .uri = "/api/relay", .method = HTTP_GET, .handler = relay_get_handler,
+};
+
+static const httpd_uri_t uri_relay_post = {
+    .uri = "/api/relay", .method = HTTP_POST, .handler = relay_post_handler,
+};
+
+static const httpd_uri_t uri_eqsl_creds = {
+    .uri = "/api/eqsl_creds", .method = HTTP_POST, .handler = eqsl_creds_handler,
+};
+
+@@ -6507,7 +6573,12 @@ esp_err_t webserver_start(void)
+    // silently from the endpoint's point of view, so the symptom would have been
+    // "the shortcuts page 404s" with nothing obviously wrong. Counted, not
+    // guessed: grep -c httpd_register_uri_handler in both files.
+    config.max_uri_handlers = 55; // 48 API (incl. rxaudio capture) + 5 file-browser + headroom
+    // □ AND AGAIN, 2026-10-04: the network radio's GET + POST /api/relay made
+    // it 50 here + 5 in filebrowser.c + 1 for /ws in webserver_ws.c = 56
+    // against 55. /ws registers LAST, so it was the one refused
+    // (ESP_ERR_HTTPD_HANDLERS_FULL) and the web page's live spectrum never
+    // connected. Count all three files. 64 leaves room for a few more.
+    config.max_uri_handlers = 64; // 50 here + 5 file-browser + 1 /ws (= 56) + headroom
+    config.lru_purge_enable = true;
+    // LWIP_MAX_SOCKETS is 16; httpd reserves 3, so up to 13 sessions are safe.
+    // Give the browser headroom (WS + /api polls + reconnect bursts) so a stale
+@@ -6636,6 +6707,8 @@ esp_err_t webserver_start(void)
+    httpd_register_uri_handler(s_server, &uri_help);
+    httpd_register_uri_handler(s_server, &uri_manual);
+    httpd_register_uri_handler(s_server, &uri_eqsl_creds);
+    httpd_register_uri_handler(s_server, &uri_relay_get); // network radio (QMX on a NAS)
+    httpd_register_uri_handler(s_server, &uri_relay_post);
+    httpd_register_uri_handler(s_server, &uri_qrz_lookup_creds);
+    httpd_register_uri_handler(s_server, &uri_eqsl_upload);
+    httpd_register_uri_handler(s_server, &uri_cloudlog_creds);

```

main/net/www/index.html (diff vs v1.16.11)

```

diff --git a/main/net/www/index.html b/main/net/www/index.html
--- a/main/net/www/index.html

```

```

+++ b/main/net/www/index.html
@@ -1310,6 +1310,7 @@
    <button type="button" class="bot-btn" id="radio-group-btn">Radio &#9650;</button>
    <div class="bot-menu" id="radio-menu">
      <a href="#" id="screen-toggle-link" title="Switch the Tab5 between the panadapter and the FT8/FT4 screen">Swi
< tch to FT8/FT4</a>
+      <a href="#" id="relay-link" title="Where the QMX is: on the Tab5's own USB port, or plugged into your NAS and
+ reached over the network (tools/nas-relay). Click to change.">Radio source: --</a>
      <a href="#" id="memory-btn" title="Your 32 memory channels - click one to tune to it">Memory channels</a>
      <a href="#" id="pause-btn" title="Stops the Tab5 talking to the radio, so its own menus and the Band Configur
+ ation terminal app behave normally. Nothing is decoded until you hand it back">Let me use the QMX menus</a>
      <a href="#" id="tune-ant-btn" style="display:none" title="Antenna Tune (QMX 1.04+): keys the radio with a ste
+ ady carrier and shows live power and SWR. Auto-stops after 60 s.">Antenna Tune</a>
@@ -6035,6 +6036,39 @@ function paintPaused(on) {
    bar.remove();
  }
}
+
+/* Network radio - the QMX plugged into a NAS, reached over TCP through
+ tools/nas-relay on the NAS. Blank host = QMX on the Tab5's USB port. */
+document.getElementById("relay-link").addEventListener("click", async (e) => {
+ e.preventDefault();
+ let cur = { host: "", port: 7355, last_error: "" };
+ try { cur = await fetch("/api/relay", { cache: "no-store" }).json(); } catch (_) {}
+ const shown = cur.host ? (cur.port && cur.port !== 7355 ? cur.host + ":" + cur.port : cur.host) : "";
+ const msg = "Radio source: QMX on a NAS (network).\n\n" +
+   "Enter the NAS IP address (optionally IP:port, default port 7355),\n" +
+   "or leave blank to use the QMX plugged into the Tab5's USB port.\n\n" +
+   "Current: " + (cur.host ? shown + " (" + (cur.connected ? "connected" : (cur.last_error || "not connec
+ ted)) + ") : "USB");
+ const v = prompt(msg, shown);
+ if (v === null) return;
+ let host = v.trim(), port = 7355;
+ const m = host.match(/^(.*):(\d{1,5})$/);
+ if (m) { host = m[1].trim(); port = parseInt(m[2], 10); }
+ if (port < 1 || port > 65535) { alert("Bad port"); return; }
+ try {
+   const r = await fetch("/api/relay", { method: "POST", body: JSON.stringify({ host, port }) });
+   if (!r.ok) throw new Error("HTTP " + r.status + " " + (await r.text()));
+ } catch (err) {
+   alert("Setting the radio source failed: " + err);
+ }
+ });
+
+function updateRelayLink(link) {
+ const el = document.getElementById("relay-link");
+ if (!el) return;
+ el.textContent = link === "net" ? "Radio source: NAS (network)"
+   : link === "usb" ? "Radio source: USB"
+   : "Radio source: none connected";
+}
+
+document.getElementById("pause-btn").addEventListener("click", async () => {
+   await sendCmd({ action: pausedNow ? "resume" : "pause" });
+   paintPaused(!pausedNow); // optimistic; the next status poll is authoritative
@@ -6293,6 +6327,7 @@ async function tick() {
    * one poll has had time to return carrying it. That window is the debounce
    * plus a round trip; anything shorter and the last snap-back still shows. */
    radioFreqHz = s.freq_hz || 0; /* always - see its declaration */
+   updateRelayLink(s.radio_link);
    if (Date.now() >= freqHoldUntil) lastFreqHz = s.freq_hz || 0;
    lastPassbandHz = s.passband_hz || 0;
    lastCwFilterMask = (typeof s.cw_filter_mask === "number") ? s.cw_filter_mask : 0;

```

main/CMakeLists.txt (diff vs v1.16.11)

```

diff --git a/main/CMakeLists.txt b/main/CMakeLists.txt
--- a/main/CMakeLists.txt
+++ b/main/CMakeLists.txt
@@ -95,6 +95,7 @@ idf_component_register(
    "net/filebrowser.c"
    "usb_hid_mouse.c"
    "net/rigctld_server.c"
+   "net/radio_relay.c"
    "net/reader_net.c"
    "net/manual_embed.c"
    "net/update_check.c"

```

CMakeLists.txt (diff vs v1.16.11)

```

diff --git a/CMakeLists.txt b/CMakeLists.txt
--- a/CMakeLists.txt
+++ b/CMakeLists.txt
@@ -13,4 +13,16 @@ include($ENV{IDF_PATH}/tools/cmake/project.cmake)

```

```
# build/compile_commands.json for lv_mem_core_builtin.c).
idf_build_set_property(COMPILE_DEFINITIONS "-DLV_ATTRIBUTE_LARGE_RAM_ARRAY=EXT_RAM_BSS_ATTR" APPEND)

+# Network-radio build: report the release it is based on plus "-NAS"
+# (e.g. v1.16.11-NAS) instead of git describe's v1.16.11-3-g<hash>-dirty, which
+# is too long for the UI. The update check compares only the numbers before the
+# first '.', so v1.16.11-NAS counts as v1.16.11.
+execute_process(COMMAND git describe --tags --abbrev=0
+                WORKING_DIRECTORY ${CMAKE_CURRENT_SOURCE_DIR}
+                OUTPUT_VARIABLE QMX_BASE_TAG OUTPUT_STRIP_TRAILING_WHITESPACE ERROR_QUIET)
+if(NOT QMX_BASE_TAG)
+  set(QMX_BASE_TAG "v1.16.11")
+endif()
+set(PROJECT_VER "${QMX_BASE_TAG}-NAS")
+
project(qmx_panadapter)
```

main/idf_component.yml (diff vs v1.16.11)

```
diff --git a/main/idf_component.yml b/main/idf_component.yml
--- a/main/idf_component.yml
+++ b/main/idf_component.yml
@@ -1,3 +1,8 @@
+# network-radio (2026-10-04): four components are pinned to the versions the
+# official v1.16.11 release was built with (identified from its firmware image).
+# A fresh fetch resolved newer ones (cdc_acm 2.4.1, usb_host_hid 1.2.1,
+# esp_codec_dev 1.6.2, mdns 1.14.0) and that build had NO working touch input on
+# the Tab5, while the official binary of the same source had it.
dependencies:
  idf:
    version: ">=5.4.0"
@@ -5,19 +10,19 @@ dependencies:
  espressif/esp_lcd_ili9881c: "^1.1.0"
  espressif/esp_lcd_touch_gt911: "^1.1.1~2"
  espressif/esp_lcd_touch_st7123: "^1.0.0"
- espressif/esp_codec_dev: "^1.3.0"
+ espressif/esp_codec_dev: "1.5.11"      # network-radio: pinned, see note above
  espressif/esp_lvgl_port: "~2.5.0"

- espressif/usb_host_cdc_acm: "^2.0.0"
+ espressif/usb_host_cdc_acm: "2.4.0"    # network-radio: pinned
  espressif/usb_host_uac: "^1.4.0"
- espressif/usb_host_hid: "^1.0.0"
+ espressif/usb_host_hid: "1.2.0"        # network-radio: pinned

  espressif/esp-dsp: "^1.6.0"

  # mDNS: reach the Tab5 as qmx.local instead of hunting an IP that changes with
  # the network. Added after an evening lost to exactly that (roamed onto a hotel
  # WiFi, new lease, no way to find it).
- espressif/mdns: "^1.2.0"
+ espressif/mdns: "1.11.3"              # network-radio: pinned

  lvgl/lvgl: "~9.2.0"
  espressif/esp_wifi_remote:
```

sdkconfig (diff vs v1.16.11)

```
diff --git a/sdkconfig b/sdkconfig
--- a/sdkconfig
+++ b/sdkconfig
@@ -1999,8 +1999,8 @@ CONFIG_LWIP_TCP_TMR_INTERVAL=250
 CONFIG_LWIP_TCP_MSL=10000
 CONFIG_LWIP_TCP_FIN_WAIT_TIMEOUT=20000
 CONFIG_LWIP_TCP_SND_BUF_DEFAULT=2880
-CONFIG_LWIP_TCP_WND_DEFAULT=2880
-CONFIG_LWIP_TCP_RECVMBOX_SIZE=6
+CONFIG_LWIP_TCP_WND_DEFAULT=11520
+CONFIG_LWIP_TCP_RECVMBOX_SIZE=12
 CONFIG_LWIP_TCP_ACCEPTMBOX_SIZE=6
 CONFIG_LWIP_TCP_QUEUE_OOSEQ=y
 CONFIG_LWIP_TCP_OOSEQ_TIMEOUT=6
@@ -3406,8 +3406,8 @@ CONFIG_TCP_SYNMAXRTX=12
 CONFIG_TCP_MSS=1440
 CONFIG_TCP_MSL=10000
 CONFIG_TCP_SND_BUF_DEFAULT=2880
-CONFIG_TCP_WND_DEFAULT=2880
-CONFIG_TCP_RECVMBOX_SIZE=6
+CONFIG_TCP_WND_DEFAULT=11520
+CONFIG_TCP_RECVMBOX_SIZE=12
 CONFIG_TCP_QUEUE_OOSEQ=y
 CONFIG_TCP_OVERSIZE_MSS=y
 # CONFIG_TCP_OVERSIZE_QUARTER_MSS is not set
```

tools/check_patches.py (diff vs v1.16.11)

```
diff --git a/tools/check_patches.py b/tools/check_patches.py
--- a/tools/check_patches.py
+++ b/tools/check_patches.py
@@ -77,6 +77,29 @@ PATCHES = [
    # Same FILE as the row above, different marker. sdio_drv.c now carries
    # several of our fixes and one apply script restores all of them, so a row
    # per marker is the only way a partially-restored copy shows up as missing.
+   ("apply_esp_hosted_sdio_rx_copy_psrn.ps1", "repo",
+    "managed_components/espressif_esp_hosted/host/drivers/transport/sdio/sdio_drv.c",
+    "QMX_SDIO_RX_COPY_PSRAM",
+    "every received WiFi frame is copied into INTERNAL RAM (malloc under "
+    "SPIRAM_MALLOC_ALWAYSINTERNAL) and held until TCP delivers it, so a "
+    "sustained inbound stream exhausts internal/DMA RAM - touch (I2C) and the "
+    "web server stop until reboot - and an allocation failure asserts"),
+   ("apply_esp_hosted_sdio_split_bundles.ps1", "repo",
+    "managed_components/espressif_esp_hosted/host/drivers/transport/sdio/sdio_drv.c",
+    "QMX_SDIO_RX_SPLIT",
+    "the C6 streams several frames per SDIO read; without the split only the "
+    "first survives a normal read and a burst is drained and discarded whole - "
+    "a sustained inbound stream (the NAS radio relay) collapses to a fraction "
+    "of its rate under TCP retransmits and starves the UI"),
+   ("apply_esp_hosted_sdio_split_psrn.ps1", "repo",
+    "managed_components/espressif_esp_hosted/host/drivers/transport/sdio/sdio_drv.c",
+    "QMX_SDIO_RXQ_PSRAM",
+    "received SDIO frames sat in the RX queue in DMA mempool blocks, and that "
+    "pool never returns memory to the heap - the NAS radio stream grew it until "
+    "DMA RAM was gone and the touchscreen's I2C reads failed (no tap, no swipe). "
+    "Stage 2 marker; stage 1 (QMX_SDIO_SPLIT_PSRAM) moved only the split frames."),
    ("apply_esp_hosted_sdio_recovery.ps1", "repo",
     "managed_components/espressif_esp_hosted/host/drivers/transport/sdio/sdio_drv.c",
     "QMX_SDIO_INIT_FAIL_TOLERANT",
```

tools/patches/apply_cdc_acm_close_tolerant.ps1 (diff vs v1.16.11)

```
diff --git a/tools/patches/apply_cdc_acm_close_tolerant.ps1 b/tools/patches/apply_cdc_acm_close_tolerant.ps1
--- a/tools/patches/apply_cdc_acm_close_tolerant.ps1
+++ b/tools/patches/apply_cdc_acm_close_tolerant.ps1
@@ -149,13 +149,30 @@ $patched = @'
$originalLf = $original -replace "`r`n", "`n"
$contentLf = $content -replace "`r`n", "`n"

+$patchedLf = $patched -replace "`r`n", "`n"
+
+# usb_host_cdc_acm 2.4.1 (2026-08-25, "Fixed submitting transfer poll race
+# condition") brought the two polling flags back, so its close() still clears
+# them before the endpoint resets. Accept that layout too and keep the two
+# lines, so the upstream race fix stays intact; the patch is otherwise identical.
+$pollLines = "    cdc_dev->data.in_polling = false;`n    cdc_dev->notif.xfer_polling = false;`n"
+$firstIf = "    if (cdc_dev->data.in_xfer) {`n"
+$originalPollLf = $originalLf.Replace("    // Cancel polling of BULK IN and INTERRUPT IN`n",
+    "    // Cancel polling of BULK IN and INTERRUPT IN`n" + $pollLines)
+if (-not $contentLf.Contains($originalLf) -and $contentLf.Contains($originalPollLf)) {
+    Write-Host "usb_host_cdc_acm with polling flags (2.4.1+) - keeping them." -ForegroundColor Yellow
+    $originalLf = $originalPollLf
+    $i = $patchedLf.Index($firstIf)
+    $patchedLf = $patchedLf.Substring(0, $i) + $pollLines + $patchedLf.Substring($i)
+    $patchedLf = $patchedLf.Replace("    // unchanged.`n", "    // unchanged. 2026-10-04: usb_host_cdc_acm 2.4.1 rest
+ored both flags (its`n    // poll race fix), so with 2.4.1+ they are cleared here again.`n")
+}
+
+if (-not $contentLf.Contains($originalLf)) {
+    Write-Host "Could not find the stock cdc_acm_host_close() block - the component may have changed." -ForegroundColor
+red
+    Write-Host "Patch by hand and update this script."
+    exit 1
+}

-$patchedLf = $patched -replace "`r`n", "`n"
$result = $contentLf.Replace($originalLf, $patchedLf)
[System.IO.File]::WriteAllText($target, $result, (New-Object System.Text.UTF8Encoding $false))
```

tools/patches/apply_httpd_ws_dead_socket_close.ps1 (diff vs v1.16.11)

```
diff --git a/tools/patches/apply_httpd_ws_dead_socket_close.ps1 b/tools/patches/apply_httpd_ws_dead_socket_close.ps1
--- a/tools/patches/apply_httpd_ws_dead_socket_close.ps1
+++ b/tools/patches/apply_httpd_ws_dead_socket_close.ps1
@@ -67,6 +67,11 @@ $old = @'
    return ESP_OK;
"@
```

```

+ # Match the file's own line endings: this script is CRLF (so $old is too), an
+ # IDF tree checked out on Windows is CRLF, but on macOS/Linux it is LF.
+ $eol = if ($text.Contains("`r`n")) { "`r`n" } else { "`n" }
+ $old = ($old -replace "`r`n", "`n") -replace "`n", $eol
+
+ if ($text -notmatch [regex]::Escape($old)) {
+     Write-Error "httpd_ws.c does not contain the expected block - IDF version drift? Patch NOT applied."
+ }
@@ -87,6 +92,7 @@ $new = @"
+     return ESP_FAIL;
+
+ "@

+ $new = ($new -replace "`r`n", "`n") -replace "`n", $eol
+ $text = $text.Replace($old, $new)
+ Set-Content -LiteralPath $file -Value $text -NoNewline -Encoding UTF8
+ Write-Output "Patched httpd_ws.c: a dead WS socket now closes instead of spinning."

```

main/ui/ui.c (diff vs v1.16.11)

```

diff --git a/main/ui/ui.c b/main/ui/ui.c
--- a/main/ui/ui.c
+++ b/main/ui/ui.c
@@ -85,6 +85,7 @@ LV_FONT_DECLARE(qmx_mono_25); /* shared with the radio-menus screen */
#include "ft8_test.h"
#include "esp_lcd_touch.h"
#include "esp_system.h" // esp_restart - the drawer's "Restart now" (#270)
+ #include "radio_relay.h" // Radio source: QMX on the Tab5 USB port or on a NAS
#include "bt_hid_mouse.h" // is the BLE radio actually up, and was it at boot

static const char *TAG = "ui";
@@ -3005,7 +3006,14 @@ static bool s_drawer_swipe_vertical = false; /* this drag went vertical */
// lv_obj_has_flag_any <- lv_obj_is_layout_positioned
//
// A comment cannot stop this on its own, so the assert below now does.
- #define N_DRAWER_SECTIONS 47
+ // "Radio source" - which QMX the Tab5 talks to: the one on its own USB port,
+ // or one plugged into a NAS running tools/nas-relay. Same effect as the web
+ // page's POST /api/relay (settings_set_relay + radio_relay_set_target +
+ // cat_reconnect), so the switch no longer needs a computer.
+ #define DRAWER_SEC_RADIOSRC 47
+ // IDs are 0..47 now. The next one raises N_DRAWER_SECTIONS AND gets an
+ // assert below - see the 2026-10-03 note above.
+ #define N_DRAWER_SECTIONS 48

/* Catches the mistake above at COMPILE time instead of as a crash minutes into
* a session. Every id must be a valid index; raise N_DRAWER_SECTIONS when you
@@ -3013,6 +3021,8 @@ static bool s_drawer_swipe_vertical = false; /* this drag went vertical */
_Static_assert(DRAWER_SEC_REBOOT < N_DRAWER_SECTIONS, "raise N_DRAWER_SECTIONS");
_Static_assert(DRAWER_SEC_SDEJECT < N_DRAWER_SECTIONS, "raise N_DRAWER_SECTIONS");
_Static_assert(DRAWER_SEC_OUTPWR < N_DRAWER_SECTIONS, "raise N_DRAWER_SECTIONS");
+ _Static_assert(DRAWER_SEC_RADIOSRC < N_DRAWER_SECTIONS, "raise N_DRAWER_SECTIONS");
+ _Static_assert(N_DRAWER_SECTIONS <= 64, "drawer masks are uint64_t");
static lv_obj_t *s_drawer_sections[N_DRAWER_SECTIONS];
static int s_drawer_section_y[N_DRAWER_SECTIONS];
static int s_drawer_section_h[N_DRAWER_SECTIONS];
@@ -3045,6 +3055,7 @@ static const drawer_item_t GRP_STATION[] = {
{ DRAWER_SEC_BPREGION, "Band-plan region", false },
};
static const drawer_item_t GRP_RADIO[] = {
+ { DRAWER_SEC_RADIOSRC, "Radio source (USB / NAS)", true },
{ DRAWER_SEC_QMXVOL, "QMX volume", true },
{ DRAWER_SEC_QMXRF, "RF gain", true },
/* Above CW centre (operator, 2026-09-07). It was filed under Spectrum with
@@ -3593,6 +3604,9 @@ static void gain_resolve_start(void); // repaint a read-back that answers late
static void gain_resolve_stop(void);
static void drawer_pause_btn_cb(lv_event_t *e);
static void drawer_term_btn_cb(lv_event_t *e);
+ static void drawer_radiosrc_btn_cb(lv_event_t *e);
+ static void radiosrc_label_timer_cb(lv_timer_t *t);
+ static lv_obj_t *s_radiosrc_lbl;
static void topbar_reconcile_cb(lv_timer_t *t);
static void drawer_slider_cwtxoff_cb(lv_event_t *e);
static void ui_set_cw_tx_offset_label(int hz);
@@ -12488,6 +12502,29 @@ static void drawer_build(void)
//
// Directly under the pause button on purpose: an operator reaching for
// "let me use the QMX menus" on a radio with no front panel wanted this.
+ // "Radio source" - USB or NAS, with the link state on the button itself so
+ // the drawer answers "which radio am I on, and is it there" at a glance.
+ // The label is refreshed once a second by radiosrc_label_timer_cb.
+ {
+     lv_obj_t *sec = drawer_section(DRAWER_SEC_RADIOSRC, y, 72);
+     lv_obj_t *btn = lv_btn_create(sec);

```

```

+     lv_obj_set_size(btn, DRAWER_W - 32, 56);
+     lv_obj_align(btn, LV_ALIGN_TOP_LEFT, 0, 0);
+     lv_obj_set_style_bg_color(btn, lv_color_hex(0x2a3138), 0);
+     lv_obj_set_style_border_color(btn, lv_color_hex(UI_COLOR_PRIMARY), 0);
+     lv_obj_set_style_border_width(btn, 2, 0);
+     lv_obj_add_event_cb(btn, drawer_radiosrc_btn_cb, LV_EVENT_CLICKED, NULL);
+     s_radiosrc_lbl = lv_label_create(btn);
+     lv_label_set_text(s_radiosrc_lbl, "Radio source");
+     lv_obj_set_style_text_font(s_radiosrc_lbl, &lv_font_montserrat_24, 0);
+     lv_obj_set_style_text_color(s_radiosrc_lbl, lv_color_hex(0xffffffff), 0);
+     lv_obj_center(s_radiosrc_lbl);
+     static lv_timer_t *s_radiosrc_timer;
+     if (!s_radiosrc_timer) s_radiosrc_timer = lv_timer_create(radiosrc_label_timer_cb, 1000, NULL);
+     radiosrc_label_timer_cb(NULL);
+     y += 72;
+ }
+
+ {
+     lv_obj_t *sec = drawer_section(DRAWER_SEC_TERM, y, 72);
+     lv_obj_t *btn = lv_btn_create(sec);
@@ -15289,6 +15326,230 @@ static void drawer_bt_restart_cb(lv_event_t *e)
+     * physically swap a card - which is exactly when it disappears. It also has a
+     * second instruction after the swap, so it has to still be there afterwards.
+     * A window waits. */
+/* ---- Radio source (USB / NAS) -----
+ * Built once and hidden/shown, not rebuilt per open: ui_kbd_set_buttons()
+ * keeps a bounded registry keyed by the button pointers, and a modal rebuild
+ * on every open would use up a slot each time. */
+#define RADIOSRC_NS "radiosrc"          /* remembers the last NAS address, so
+                                       * USB -> NAS is not a retype */
+static lv_obj_t *s_radiosrc_modal;
+static lv_obj_t *s_radiosrc_ta;
+static lv_obj_t *s_radiosrc_state;
+static lv_obj_t *s_radiosrc_kb;
+static uint16_t s_radiosrc_port = RADIO_RELAY_DEFAULT_PORT;
+
+static void radiosrc_label_timer_cb(lv_timer_t *t)
+{
+    (void)t;
+    if (!s_radiosrc_lbl) return;
+    char host[RADIO_RELAY_HOST_LEN] = "";
+    uint16_t port = 0;
+    radio_relay_get_target(host, sizeof(host), &port);
+    char txt[RADIO_RELAY_HOST_LEN + 48];
+    if (!radio_relay_enabled() || !host[0]) {
+        const bool up = (strcmp(cat_get_transport(), "usb") == 0);
+        snprintf(txt, sizeof(txt), LV_SYMBOL_USB " Radio: USB%s", up ? "" : " (no QMX)");
+    } else {
+        snprintf(txt, sizeof(txt), LV_SYMBOL_WIFI " NAS %s%s", host,
+            radio_relay_connected() ? "" : LV_SYMBOL_OK : " (offline)");
+    }
+    const char *cur = lv_label_get_text(s_radiosrc_lbl);
+    if (!cur || strcmp(cur, txt) != 0) lv_label_set_text(s_radiosrc_lbl, txt);
+}
+
+static void radiosrc_last_get(char *buf, size_t cap)
+{
+    buf[0] = '\0';
+    nvs_handle_t h;
+    if (nvs_open(RADIOSRC_NS, NVS_READONLY, &h) != ESP_OK) return;
+    size_t n = cap;
+    if (nvs_get_str(h, "last", buf, &n) != ESP_OK) buf[0] = '\0';
+    nvs_close(h);
+}
+
+static void radiosrc_last_set(const char *host)
+{
+    nvs_handle_t h;
+    if (nvs_open(RADIOSRC_NS, NVS_READWRITE, &h) != ESP_OK) return;
+    nvs_set_str(h, "last", host);
+    nvs_commit(h);
+    nvs_close(h);
+}
+
+static void radiosrc_close(void)
+{
+    if (s_radiosrc_ta) lv_obj_clear_state(s_radiosrc_ta, LV_STATE_FOCUSED);
+    if (s_radiosrc_modal) lv_obj_add_flag(s_radiosrc_modal, LV_OBJ_FLAG_HIDDEN);
+}
+
+/* The same three calls POST /api/relay makes (webserver.c relay_post_handler):
+ * persist, retarget, and drop the current link so cat's link_task reopens on
+ * the new source. Empty host = USB. */
+static void radiosrc_apply(const char *host)

```

```

+{
+    settings_set_relay(host, s_radiosrc_port);
+    radio_relay_set_target(host, s_radiosrc_port);
+    cat_reconnect();
+    radiosrc_label_timer_cb(NULL);
+    radiosrc_close();
+    drawer_close();
+    char msg[RADIO_RELAY_HOST_LEN + 48];
+    if (host[0]) snprintf(msg, sizeof(msg), "Radio source: NAS %s - connecting...", host);
+    else             snprintf(msg, sizeof(msg), "Radio source: USB - plug the QMX into the Tab5");
+    ui_toast_ms(msg, 3000);
+}
+
+static void radiosrc_use_nas_cb(lv_event_t *e)
+{
+    (void)e;
+    char host[RADIO_RELAY_HOST_LEN];
+    const char *t = lv_textarea_get_text(s_radiosrc_ta);
+    while (*t == ' ') t++;
+    snprintf(host, sizeof(host), "%s", t);
+    for (int i = (int)strlen(host) - 1; i >= 0 && host[i] == ' '; i--) host[i] = '\0';
+    if (!host[0]) {
+        lv_label_set_text(s_radiosrc_state, "Type the NAS address first (e.g. 192.168.1.20).");
+        lv_obj_set_style_text_color(s_radiosrc_state, lv_color_hex(0xE0A040), 0);
+        return;
+    }
+    radiosrc_last_set(host);
+    radiosrc_apply(host);
+}
+
+static void radiosrc_use_usb_cb(lv_event_t *e)
+{
+    (void)e;
+    /* Keep what is typed as the remembered NAS, so switching back is one tap. */
+    const char *t = lv_textarea_get_text(s_radiosrc_ta);
+    if (t && t[0]) radiosrc_last_set(t);
+    radiosrc_apply("");
+}
+
+static void radiosrc_cancel_cb(lv_event_t *e) { (void)e; radiosrc_close(); }
+static void radiosrc_swallow_cb(lv_event_t *e) { (void)e; }
+
+static void radiosrc_kb_cb(lv_event_t *e)
+{
+    lv_event_code_t c = lv_event_get_code(e);
+    if (c == LV_EVENT_READY) radiosrc_use_nas_cb(e);    /* keyboard's OK key */
+    if (c == LV_EVENT_CANCEL) radiosrc_close();
+}
+
+static lv_obj_t *radiosrc_btn(lv_obj_t *p, const char *txt, uint32_t bg, int x,
+                               lv_event_cb_t cb)
+{
+    lv_obj_t *b = lv_btn_create(p);
+    lv_obj_set_size(b, 250, 64);
+    lv_obj_align(b, LV_ALIGN_BOTTOM_MID, x, -12);
+    lv_obj_set_style_bg_color(b, lv_color_hex(bg), 0);
+    lv_obj_set_style_radius(b, 8, 0);
+    lv_obj_add_event_cb(b, cb, LV_EVENT_CLICKED, NULL);
+    lv_obj_t *l = lv_label_create(b);
+    lv_label_set_text(l, txt);
+    lv_obj_set_style_text_font(l, &lv_font_montserrat_28, 0);
+    lv_obj_set_style_text_color(l, lv_color_hex(0xffffffff), 0);
+    lv_obj_center(l);
+    return b;
+}
+
+static void radiosrc_modal_build(void)
+{
+    if (s_radiosrc_modal) return;
+    s_radiosrc_modal = lv_obj_create(lv_screen_active());
+    lv_obj_set_size(s_radiosrc_modal, LV_PCT(100), LV_PCT(100));
+    lv_obj_set_pos(s_radiosrc_modal, 0, 0);
+    lv_obj_set_style_bg_color(s_radiosrc_modal, lv_color_hex(0x000000), 0);
+    lv_obj_set_style_bg_opa(s_radiosrc_modal, UI_OPA_MODAL_SCRIM, 0);
+    lv_obj_set_style_border_width(s_radiosrc_modal, 0, 0);
+    lv_obj_set_style_pad_all(s_radiosrc_modal, 0, 0);
+    lv_obj_clear_flag(s_radiosrc_modal, LV_OBJ_FLAG_SCROLLABLE);
+    lv_obj_add_flag(s_radiosrc_modal, LV_OBJ_FLAG_CLICKABLE);
+    lv_obj_add_event_cb(s_radiosrc_modal, radiosrc_swallow_cb, LV_EVENT_CLICKED, NULL);
+
+    lv_obj_t *p = lv_obj_create(s_radiosrc_modal);
+    lv_obj_set_size(p, 900, 380);
+    lv_obj_align(p, LV_ALIGN_TOP_MID, 0, 20);
+    lv_obj_set_style_bg_color(p, lv_color_hex(0x1c2128), 0);

```

```

+ lv_obj_set_style_bg_opa(p, LV_OPA_COVER, 0);
+ lv_obj_set_style_border_color(p, lv_color_hex(UI_COLOR_PRIMARY), 0);
+ lv_obj_set_style_border_width(p, 2, 0);
+ lv_obj_set_style_radius(p, 10, 0);
+ lv_obj_clear_flag(p, LV_OBJ_FLAG_SCROLLABLE);
+
+ lv_obj_t *t = lv_label_create(p);
+ lv_label_set_text(t, "Radio source");
+ lv_obj_set_style_text_font(t, &lv_font_montserrat_28, 0);
+ lv_obj_set_style_text_color(t, lv_color_hex(0xffffffff), 0);
+ lv_obj_align(t, LV_ALIGN_TOP_MID, 0, 4);
+
+ s_radiosrc_state = lv_label_create(p);
+ lv_label_set_long_mode(s_radiosrc_state, LV_LABEL_LONG_WRAP);
+ lv_obj_set_width(s_radiosrc_state, 840);
+ lv_obj_set_style_text_font(s_radiosrc_state, &lv_font_montserrat_24, 0);
+ lv_obj_align(s_radiosrc_state, LV_ALIGN_TOP_LEFT, 10, 50);
+
+ lv_obj_t *hl = lv_label_create(p);
+ lv_label_set_text(hl, "NAS address:");
+ lv_obj_set_style_text_font(hl, &lv_font_montserrat_24, 0);
+ lv_obj_set_style_text_color(hl, lv_color_hex(0xE6E6E6), 0);
+ lv_obj_align(hl, LV_ALIGN_TOP_LEFT, 10, 158);
+
+ s_radiosrc_ta = lv_textarea_create(p);
+ lv_textarea_set_one_line(s_radiosrc_ta, true);
+ lv_textarea_set_max_length(s_radiosrc_ta, RADIO_RELAY_HOST_LEN - 1);
+ lv_textarea_set_placeholder_text(s_radiosrc_ta, "e.g. 192.168.1.20");
+ lv_obj_set_size(s_radiosrc_ta, 600, 60);
+ lv_obj_align(s_radiosrc_ta, LV_ALIGN_TOP_LEFT, 200, 144);
+ lv_obj_set_style_text_font(s_radiosrc_ta, &lv_font_montserrat_28, 0);
+ ui_theme_style_textarea(s_radiosrc_ta);
+
+ lv_obj_t *nas = radiosrc_btn(p, LV_SYMBOL_WIFI " Use NAS", UI_COLOR_PRIMARY, -280, radiosrc_use_nas_cb);
+ radiosrc_btn(p, LV_SYMBOL_USB " Use USB", 0x3a4149, 0, radiosrc_use_usb_cb);
+ lv_obj_t *cancel = radiosrc_btn(p, "Cancel", 0x3a4149, 280, radiosrc_cancel_cb);
+ ui_kbd_set_buttons(nas, cancel); /* physical keyboard: Enter / Esc */
+
+ s_radiosrc_kb = lv_keyboard_create(s_radiosrc_modal);
+ ui_theme_style_keyboard(s_radiosrc_kb);
+ lv_keyboard_set_mode(s_radiosrc_kb, LV_KEYBOARD_MODE_NUMBER);
+ lv_obj_set_size(s_radiosrc_kb, LV_PCT(100), 280);
+ lv_obj_align(s_radiosrc_kb, LV_ALIGN_BOTTOM_MID, 0, 0);
+ lv_keyboard_set_textarea(s_radiosrc_kb, s_radiosrc_ta);
+ lv_obj_add_event_cb(s_radiosrc_kb, radiosrc_kb_cb, LV_EVENT_ALL, NULL);
+}
+
+static void radiosrc_modal_show(void)
+{
+    radiosrc_modal_build();
+    char host[RADIO_RELAY_HOST_LEN] = "";
+    uint16_t port = 0;
+    radio_relay_get_target(host, sizeof(host), &port);
+    s_radiosrc_port = port ? port : RADIO_RELAY_DEFAULT_PORT;
+    const bool nas = radio_relay_enabled() && host[0];
+
+    char st[RADIO_RELAY_HOST_LEN + 96];
+    if (nas) {
+        snprintf(st, sizeof(st), "Now: QMX on the NAS at %s, port %u - %s.", host,
+            (unsigned)s_radiosrc_port,
+            radio_relay_connected() ? "connected" : "not connected");
+    } else {
+        radiosrc_last_get(host, sizeof(host));
+        snprintf(st, sizeof(st), "Now: QMX on the Tab5's USB port - %s.",
+            strcmp(cat_get_transport(), "usb") == 0 ? "connected" : "no QMX found");
+    }
+    lv_label_set_text(s_radiosrc_state, st);
+    lv_obj_set_style_text_color(s_radiosrc_state, lv_color_hex(0xC8D0D8), 0);
+    lv_textarea_set_text(s_radiosrc_ta, host);
+    lv_obj_clear_flag(s_radiosrc_modal, LV_OBJ_FLAG_HIDDEN);
+    lv_obj_move_foreground(s_radiosrc_modal);
+}
+
+static void drawer_radiosrc_btn_cb(lv_event_t *e)
+{
+    (void)e;
+    drawer_close(); /* the modal and its keyboard need the screen */
+    radiosrc_modal_show();
+}
+
+static lv_obj_t *s_sdeject_modal;
+
+static void sdeject_modal_close_cb(lv_event_t *e)

```

main/wifi/wifi.c (diff vs v1.16.11)

```

diff --git a/main/wifi/wifi.c b/main/wifi/wifi.c
--- a/main/wifi/wifi.c
+++ b/main/wifi/wifi.c
@@ -137,6 +137,18 @@ static void set_sdio_gpio_drive(void)
 }

// SNTP callback -----
+static TaskHandle_t s_sntp_task;
+static volatile time_t s_sntp_pending;
+
+static void sntp_apply_task(void *arg)
+{
+    (void)arg;
+    for (;;) {
+        ulTaskNotifyTake(pdTRUE, portMAX_DELAY);
+        time_sync_notify_sntp(s_sntp_pending);
+    }
+}
+
static void sntp_sync_cb(struct timeval *tv)
{
    struct tm tm_utc;
@@ -147,8 +159,18 @@ static void sntp_sync_cb(struct timeval *tv)
    xEventGroupSetBits(s_events, BIT_TIME_OK);

    // Write to the Tab5 supercap RTC, persist NVS anchor, update system clock,
-    // and push to QMX (unless QMX GPS flag is set).
-    time_sync_notify_sntp(tv->tv_sec);
+    // and push to QMX (unless QMX GPS flag is set) - on sntp_apply_task, NOT
+    // here. □ This callback runs ON lwIP's tcpip thread. The QMX time push is
+    // a CAT write, and with the network radio (QMX on a NAS) a CAT write is a
+    // lwIP socket send(), which posts to the tcpip thread and waits for it -
+    // i.e. for itself. Measured 2026-10-04: every boot on the NAS radio, the
+    // whole network stack (radio stream, web page, TLS feeds) died within a
+    // second of "SNTP sync", for good. Over USB the push never touched lwIP,
+    // which is why this was invisible until now. The RTC (I2C) and NVS writes
+    // do not belong on the tcpip thread either.
+    s_sntp_pending = tv->tv_sec;
+    if (s_sntp_task) xTaskNotifyGive(s_sntp_task);
+    else time_sync_notify_sntp(tv->tv_sec); // worker not up: old behaviour
}

// Replicates the work IDF's default WIFI_EVENT_STA_START handler (wifi_start in
@@ -727,6 +749,11 @@ static void on_ip_event(void *arg, esp_event_base_t base,
    sntp_started = true;
    esp_sntp_config_t cfg = ESP_NETIF_SNTP_DEFAULT_CONFIG("pool.ntp.org");
    cfg.sync_cb = sntp_sync_cb;
+    // Same stack/priority as time_sync_task, which runs this same
+    // time_sync code for its other sources.
+    if (!s_sntp_task)
+        s_sntp_task = psram_task_create(sntp_apply_task, "sntp_apply", 8192,
+                                         NULL, 4, tskNO_AFFINITY);
    esp_netif_sntp_init(&cfg);
    ESP_LOGI(TAG, "SNTP started (pool.ntp.org)");
}

```

tools/patches/apply_esp_hosted_sdio_split_bundles.ps1 (new)

```

<#
.SYNOPSIS
    esp_hosted SDIO RX: deliver every frame of a multi-frame read instead of
    keeping only the first (normal reads) or discarding all of them (oversize
    bursts). Marker: QMX_SDIO_RX_SPLIT.

.DESCRIPTION
    The Tab5's ESP32-C6 runs its SDIO send path in STREAMING mode: when several
    frames are waiting it exposes them back to back, each with its own
    esp_payload_header, and the pending-length register covers all of them.
    This host is built for PACKET mode (RX_NONE + 1536-byte mempool buffers,
    v0.18.2), so the stock read path only ever looked at the first frame:

    - a normal read holding two small frames lost the second one;
    - a burst larger than 1536 bytes was drained and DISCARDED by the
      oversize recovery (apply_esp_hosted_sdio_recovery.ps1) - every frame
      in it lost.

    Invisible under light traffic, fatal for a sustained inbound stream:
    measured 2026-10-04 with the NAS radio relay (1.5 Mbit/s of I/Q) the
    oversize drain fired several times a second, TCP retransmitted the lost
    frames, the stream collapsed to 10-20k of 48k pairs/s and the Tab5 was
    too busy to take touch input until rebooted.

```

The patch walks such a read, copies each frame into its own mempool buffer (no DMA allocation - the v0.18.2 rule stands) and queues it exactly like a single-frame read. The oversize drain keeps its chunked reads and its exact byte accounting unchanged; the drained bytes are also copied into a 96 KB PSRAM buffer and split after the bus is released. A header that does not parse, or an empty mempool, ends the walk: frames already delivered stay delivered, the rest is dropped, nothing aborts. The "SDIO RX oversize" warning (the recovery patch's check_patches marker) is kept, rate-limited to one line per 5 s with the split counters.

Requires apply_esp_hosted_sdio_recovery.ps1 first (its drain code is an anchor here) - this file name sorts after it.

Edits managed_components/ - wiped by fullclean or a dependency refresh; re-run like the other apply_*.ps1 patches. Idempotent.

.HOW TO USE

```
powershell -ExecutionPolicy Bypass -File tools/patches/apply_esp_hosted_sdio_split_bundles.ps1
```

```
#>
```

```
$ErrorActionPreference = "Stop"
```

```
$repo = Split-Path -Parent (Split-Path -Parent $PSScriptRoot)
```

```
$target = Join-Path $repo "managed_components/espressif_esp_hosted/host/drivers/transport/sdio/sdio_drv.c"
```

```
if (-not (Test-Path $target)) {
    Write-Host "sdio_drv.c not found at:" -ForegroundColor Red
    Write-Host " $target"
    Write-Host "Build once so the component is fetched, then re-run."
    exit 1
}
```

```
$content = (Get-Content -Raw -Path $target) -replace "`r`n", "`n"
```

```
if ($content.Contains("QMX_SDIO_RX_SPLIT")) {
    Write-Host "Already patched (SDIO RX split of multi-frame reads) - nothing to do." -ForegroundColor Green
    exit 0
}
```

```
if (-not $content.Contains("SDIO RX oversize")) {
    Write-Host "apply_esp_hosted_sdio_recovery.ps1 has not been applied yet - run it first." -ForegroundColor Red
    exit 1
}
```

```
$pairs = @(
    @{ Old = @'
        g_h.funcs->_h_queue_item(from_slave_queue[pkt_prio], &buf_handle, HOSTED_BLOCK_MAX);
        g_h.funcs->_h_post_semaphore(sem_from_slave_queue);

    return ESP_OK;
    '@; New = @'
        g_h.funcs->_h_queue_item(from_slave_queue[pkt_prio], &buf_handle, HOSTED_BLOCK_MAX);
        g_h.funcs->_h_post_semaphore(sem_from_slave_queue);

    return ESP_OK;
    '@;
}
```

```
/* QMX_SDIO_RX_SPLIT (qmx-panadapter network-radio, 2026-10-04).
```

```
*
```

```
* The C6 slave runs its SDIO send path in STREAMING mode: when several frames
* are waiting it exposes them back to back, each starting with its own
* esp_payload_header, and the pending-length register covers all of them.
* This host is built for PACKET mode (RX_NONE, fixed 1536-byte mempool
* buffers - chosen in v0.18.2 because streaming mode re-allocates DMA memory
* per burst), so stock code only ever looked at the FIRST frame of a read:
*
```

```
* - a read <= 1536 bytes holding two small frames (two TCP ACKs, an ACK
*   plus a CAT reply) silently lost the second one;
* - a read > 1536 bytes (a burst of full-size frames) was drained and
*   DISCARDED by the oversize recovery - every frame in it lost.
*
```

```
* Harmless for light traffic, fatal for a sustained inbound stream: measured
* 2026-10-04 with the NAS radio relay (1.5 Mbit/s of I/Q) - "SDIO RX oversize
* ... draining" several times a second, TCP retransmitting the lost frames and
* the stream collapsing to 10-20 k of the 48 k pairs/s.
*
```

```
* This walks such a byte stream, copies each frame into its own mempool
* buffer (no DMA allocation, the v0.18.2 rule stands) and queues it exactly
* as a single-frame read would be. A header that does not parse, or an empty
* mempool, ends the walk: what was already delivered stays delivered, the
* rest is dropped, and nothing aborts. */
```

```
static uint32_t qmx_split_bundles, qmx_split_frames, qmx_split_drops;
static void qmx_sdio_push_stream(const uint8_t *s, uint32_t s_len)
```

```

{
    while (s_len >= sizeof(struct esp_payload_header)) {
        const struct esp_payload_header *h = (const struct esp_payload_header *)s;
        uint16_t len = le16toh(h->len);
        uint16_t offset = le16toh(h->offset);
        uint32_t psize = (uint32_t)len + offset;
        uint16_t vlen = 0, voff = 0;
        uint8_t *p;

        if (!len && !offset)
            return; /* block padding after the last frame */
        if (!len || len > MAX_PAYLOAD_SIZE ||
            offset != sizeof(struct esp_payload_header) ||
            psize > s_len || psize > MAX_SDIO_BUFFER_SIZE) {
            qmx_split_drops++;
            return;
        }
        p = sdio_buffer_alloc(MEMSET_REQUIRED);
        if (!p) {
            qmx_split_drops++;
            return;
        }
        memcpy(p, s, psize);
        if (!is_valid_sdio_rx_packet(p, &vlen, &voff)) {
            sdio_buffer_free(p);
            qmx_split_drops++;
            return;
        }
        sdio_push_pkt_to_queue(p, vlen, voff);
        qmx_split_frames++;
        s += psize;
        s_len -= psize;
    }
}

'@ {
    ,@{ Old = '@'
    uint16_t len = 0;
    uint16_t offset = 0;

    /* Drop packet if no processing needed */
    if (!is_valid_sdio_rx_packet(buf, &len, &offset)) {
'@; New = '@'
    uint16_t len = 0;
    uint16_t offset = 0;

    /* QMX_SDIO_RX_SPLIT: a read can hold more than one frame (see
     * qmx_sdio_push_stream). Peek at the first header WITHOUT validating it
     * (validation may rewrite it): if another frame follows, copy every frame
     * out in order and release this buffer. */
    {
        const struct esp_payload_header *h0 = (const struct esp_payload_header *)buf;
        uint32_t first = (uint32_t)le16toh(h0->len) + le16toh(h0->offset);
        if (le16toh(h0->offset) == sizeof(struct esp_payload_header) &&
            le16toh(h0->len) && first <= buf_len &&
            buf_len - first >= sizeof(struct esp_payload_header)) {
            const struct esp_payload_header *h1 =
                (const struct esp_payload_header *) (buf + first);
            if (le16toh(h1->len) && le16toh(h1->offset) == sizeof(struct esp_payload_header)) {
                qmx_split_bundles++;
                qmx_sdio_push_stream(buf, buf_len);
                sdio_buffer_free(buf);
                return ESP_OK;
            }
        }
    }

    /* Drop packet if no processing needed */
    if (!is_valid_sdio_rx_packet(buf, &len, &offset)) {
'@ {
    ,@{ Old = '@'
        if (sdio_oversize_drain_buf) {
            uint32_t d_left = len_from_slave;
'@; New = '@'
        /* QMX_SDIO_RX_SPLIT: keep what is drained, in PSRAM (never DMA). */
        if (!qmx_split_stream)
            qmx_split_stream = (uint8_t *)heap_caps_malloc(
                64u * (uint32_t)ESP_RX_BUFFER_SIZE,
                MALLOC_CAP_SPIRAM | MALLOC_CAP_8BIT);
        if (sdio_oversize_drain_buf) {
            uint32_t d_left = len_from_slave;
'@ {
    ,@{ Old = '@'
        d_left -= chunk;

```

```

        drained += chunk;
'@; New = '@'
        if (qmx_split_stream)
            memcpy(qmx_split_stream + drained, sdio_oversize_drain_buf, chunk);
        d_left -= chunk;
        drained += chunk;
'@ }
,@{ Old = '@'
    sdio_rx_byte_count = (sdio_rx_byte_count + drained) % ESP_RX_BYTE_MAX;
    SDIO_DRV_UNLOCK();
    if (!drained)
        g_h.funcs->_h_msleep(1);
    continue;
'@; New = '@'
    sdio_rx_byte_count = (sdio_rx_byte_count + drained) % ESP_RX_BYTE_MAX;
    SDIO_DRV_UNLOCK();
    /* QMX_SDIO_RX_SPLIT: a complete drain is a burst of whole frames -
     * deliver them instead of throwing them away. A partial drain (a
     * read failed part-way) is still discarded: its tail is unknown. */
    if (qmx_split_stream && drained && drained == len_from_slave) {
        qmx_split_bundles++;
        qmx_sdio_push_stream(qmx_split_stream, drained);
    } else if (drained) {
        qmx_split_drops++;
    }
    if (!drained)
        g_h.funcs->_h_msleep(1);
    continue;
'@ }
,@{ Old = '@'
    ESP_LOGW(TAG, "SDIO RX oversize: len=%lu host_cnt=%lu slave_reg=%lu - draining to recover",
        (unsigned long)len_from_slave, (unsigned long)sdio_rx_byte_count,
        (unsigned long)slave_reg);
'@; New = '@'
    /* QMX_SDIO_RX_SPLIT: with the burst now delivered this is routine
     * under a sustained stream, so report it at most every 5 s with
     * running totals rather than per event (it was flooding the log
     * several times a second). */
    {
        static int64_t qmx_last_oversize_log_us;
        static uint32_t qmx_oversize_since_log;
        int64_t now_us = esp_timer_get_time();
        qmx_oversize_since_log++;
        if (now_us - qmx_last_oversize_log_us >= 5000000LL) {
            ESP_LOGW(TAG, "SDIO RX oversize: len=%lu host_cnt=%lu slave_reg=%lu - "
                "draining (x%lu in 5 s); split: %lu bundles, %lu frames delivered, %lu dropped",
                (unsigned long)len_from_slave, (unsigned long)sdio_rx_byte_count,
                (unsigned long)slave_reg, (unsigned long)qmx_oversize_since_log,
                (unsigned long)qmx_split_bundles, (unsigned long)qmx_split_frames,
                (unsigned long)qmx_split_drops);
            qmx_last_oversize_log_us = now_us;
            qmx_oversize_since_log = 0;
        }
    }
'@ }
,@{ Old = '@'
static uint8_t *sdio_oversize_drain_buf = NULL;

'@; New = '@'
static uint8_t *sdio_oversize_drain_buf = NULL;
/* QMX_SDIO_RX_SPLIT: PSRAM copy of a drained burst, split into frames afterwards. */
static uint8_t *qmx_split_stream = NULL;

'@ }
)

foreach ($p in $pairs) {
    $old = $p.Old -replace "`r`n", "`n"
    $new = $p.New -replace "`r`n", "`n"
    $n = ([regex]::Matches($content, [regex]::Escape($old))).Count
    if ($n -ne 1) {
        Write-Host "Anchor found $n times (expected 1) - esp_hosted or the recovery patch changed; patch by hand and u
        < pdate this script:" -ForegroundColor Red
        Write-Host ($old.Substring(0, [Math]::Min(160, $old.Length)))
        exit 1
    }
    $content = $content.Replace($old, $new)
}

[System.IO.File]::WriteAllText($target, $content, (New-Object System.Text.UTF8Encoding $false))
Write-Host "Patched sdio_drv.c: multi-frame SDIO reads are split and delivered, not dropped." -ForegroundColor Green
Write-Host " $target"

```

tools/patches/apply_esp_hosted_sdio_rx_copy_psram.ps1 (new)

```
<#
.SYNOPSIS
    esp_hosted SDIO RX: copy each received WiFi frame into PSRAM instead of
    internal RAM, and drop a frame instead of asserting when no memory is
    left. Marker: QMX_SDIO_RX_COPY_PSRAM.

.DESCRIPTION
    sdio_process_rx task() copies every received frame into a fresh buffer
    that lwIP then holds (zero-copy, L2_T0_L3_COPY off) until TCP has handed
    the data to the application. Stock code used _h_malloc -> malloc, and with
    CONFIG_SPIRAM_MALLOC_ALWAYSINTERNAL=16384 every frame-sized malloc lands in
    INTERNAL RAM, so internal RAM in use grows with the data in flight.

    Measured 2026-10-04 with the NAS radio relay's sustained 1.5 Mbit/s stream:
    TCP window 11520 -> internal low-water 3 KB and DMA-capable free 4 KB;
    touch input (I2C) stopped and the web server stopped answering until a
    reboot. Window 5760 -> the same. Window 2880 -> usable, but only 66% of
    the stream arrives.

    The copy now goes to PSRAM (it is released with free(), which handles
    either heap), falling back to the stock allocation only if PSRAM is
    exhausted. Stock also assert()ed when the allocation failed; the frame is
    now dropped (TCP retransmits) instead of rebooting the device.

    Edits managed_components/ - wiped by fullclean or a dependency refresh;
    re-run like the other apply_*.ps1 patches. Idempotent.

.HOW TO USE
    powershell -ExecutionPolicy Bypass -File tools/patches/apply_esp_hosted_sdio_rx_copy_psram.ps1
#>

$ErrorActionPreference = "Stop"

$repo = Split-Path -Parent (Split-Path -Parent $PSScriptRoot)
$target = Join-Path $repo "managed_components/espressif__esp_hosted/host/drivers/transport/sdio/sdio_drv.c"

if (-not (Test-Path $target)) {
    Write-Host "sdio_drv.c not found at:" -ForegroundColor Red
    Write-Host "  $target"
    Write-Host "Build once so the component is fetched, then re-run."
    exit 1
}

$content = (Get-Content -Raw -Path $target) -replace "`r`n", "`n"

if ($content.Contains("QMX_SDIO_RX_COPY_PSRAM")) {
    Write-Host "Already patched (SDIO RX copy in PSRAM) - nothing to do." -ForegroundColor Green
    exit 0
}

$old = @'
/* TODO : Need to abstract heap_caps_malloc */
uint8_t * copy_payload = (uint8_t *)g_h.funcs->_h_malloc(buf_handle->payload_len);
assert(copy_payload);
assert(buf_handle->payload_len);
assert(buf_handle->payload);
memcpy(copy_payload, buf_handle->payload, buf_handle->payload_len);
'@

$new = @'
/* QMX_SDIO_RX_COPY_PSRAM (qmx-panadapter network-radio, 2026-10-04).
 *
 * Every received WiFi frame is copied into a fresh buffer that
 * lwIP holds (zero-copy, L2_T0_L3_COPY off) until TCP has
 * delivered it to the application. Stock code used _h_malloc ->
 * malloc, and with CONFIG_SPIRAM_MALLOC_ALWAYSINTERNAL=16384 every
 * frame-sized malloc lands in INTERNAL RAM - so internal RAM in
 * use grows with the data in flight. Measured with the NAS radio
 * relay's 1.5 Mbit/s stream: TCP window 11520 -> internal low-water
 * 3 KB, DMA free 4 KB, touch input (I2C) dead and the web server
 * unresponsive until reboot; window 2880 -> usable but only 66% of
 * the stream. The copy goes to PSRAM instead (freed with free(),
 * which handles either heap); internal RAM only if PSRAM is full.
 * Stock also assert()ed on allocation failure - now the frame is
 * dropped (TCP retransmits) instead of rebooting the device. */
uint8_t * copy_payload = (uint8_t *)heap_caps_malloc(buf_handle->payload_len,
    MALLOC_CAP_SPIRAM | MALLOC_CAP_8BIT);
if (!copy_payload)
    copy_payload = (uint8_t *)g_h.funcs->_h_malloc(buf_handle->payload_len);
if (!copy_payload || !buf_handle->payload_len || !buf_handle->payload) {
    if (copy_payload)
        HOSTED_FREE(copy_payload);

```

```

        H_FREE_PTR_WITH_FUNC(buf_handle->free_buf_handle, buf_handle->priv_buffer_handle);
        continue;
    }
    memcpy(copy_payload, buf_handle->payload, buf_handle->payload_len);
'@
$dold = $old -replace "`r`n", "`n"
$new = $new -replace "`r`n", "`n"
$n = ([regex]::Matches($content, [regex]::Escape($dold))).Count
if ($n -ne 1) {
    Write-Host "Anchor found $n times (expected 1) - esp_hosted changed; patch by hand and update this script." -ForegroundColor Red
    exit 1
}
$content = $content.Replace($dold, $new)

[System.IO.File]::WriteAllText($target, $content, (New-Object System.Text.UTF8Encoding $false))
Write-Host "Patched sdio_drv.c: received WiFi frames are copied into PSRAM, not internal RAM." -ForegroundColor Green
Write-Host " $target"

```

tools/patches/apply_esp_hosted_sdio_split_psram.ps1 (new)

```

<#
.SYNOPSIS
    esp_hosted SDIO RX: frames split out of a multi-frame read go into PSRAM,
    not into the DMA mempool. Marker: QMX_SDIO_SPLIT_PSRAM.

.DESCRIPTION
    apply_esp_hosted_sdio_split_bundles.ps1 copies every frame of a bundled
    SDIO read into its own buffer from the SDIO mempool. That mempool NEVER
    gives memory back to the heap: a freed block goes onto its free list and
    stays there. So the pool grows to the largest number of frames ever in
    flight at once, in DMA-capable internal RAM, for the rest of the session.

    With the NAS radio's full-rate stream (~35 bundles/s, up to 7 frames
    each) that peak is 20-30 blocks of ~1.6 KB. Measured 2026-10-04: DMA free
    fell from 38 KB to 1.2 KB within ~20 s of the stream starting, internal
    free bottomed at 1 KB, the touch controller's I2C reads (and the PI4IO
    expander's) started timing out, SD reads failed - and the touchscreen
    stopped responding while the spectrum kept running.

    These frames are already copies; nothing DMAs into them. The patch takes
    them from PSRAM with heap_caps_malloc() and hands sdio_push_pkt_to_queue()
    heap_caps_free() as their free function, so they are really freed when the
    RX task is done with them. The single-frame path (the DMA read buffer
    itself) still uses the mempool, as before.

    STAGE 2 (marker QMX_SDIO_RXQ_PSRAM, added the same day): moving the split
    frames was not enough. A NORMAL single-frame read also parks its mempool
    block in the RX queue (depth 20) until sdio_process_rx_task gets to it, so
    under the stream the pool still grew to ~20 blocks - measured: DMA free
    691 B, touch still dead. So every received frame is now copied into PSRAM
    the moment it is queued and its mempool block goes straight back; the pool
    only ever holds the one or two blocks the SDIO read itself is using. Cost:
    one extra ~1.5 KB memcpy per frame (~1.5 MB/s at the full radio rate).
    If PSRAM is ever exhausted the frame is queued in its mempool block exactly
    as before. Stage 2 is applied on its own to a file that already has stage 1.

    Requires apply_esp_hosted_sdio_split_bundles.ps1 first - this file name
    sorts after it. Edits managed_components/; re-run like the other
    apply_*.ps1 patches. Idempotent.

.HOW TO USE
    pwsh -File tools/patches/apply_esp_hosted_sdio_split_psram.ps1
#>

$ErrorActionPreference = "Stop"

$repo = Split-Path -Parent (Split-Path -Parent $PSScriptRoot)
$target = Join-Path $repo "managed_components/espressif__esp_hosted/host/drivers/transport/sdio/sdio_drv.c"

if (-not (Test-Path $target)) {
    Write-Host "sdio_drv.c not found at:" -ForegroundColor Red
    Write-Host " $target"
    Write-Host "Build once so the component is fetched, then re-run."
    exit 1
}

$content = (Get-Content -Raw -Path $target) -replace "`r`n", "`n"

if ($content.Contains("QMX_SDIO_RXQ_PSRAM")) {
    Write-Host "Already patched (SDIO RX frames queued in PSRAM) - nothing to do." -ForegroundColor Green
    exit 0
}

```

```

if (-not $content.Contains("QMX_SDIO_RX_SPLIT")) {
    Write-Host "apply_esp_hosted_sdio_split_bundles.ps1 has not been applied yet - run it first." -ForegroundColor Red
    exit 1
}

$stage1 = @(
@{ Old = @'
    buf_handle.priv_buffer_handle = rxbuff;
    buf_handle.free_buf_handle    = sdio_buffer_free;
    buf_handle.payload_len        = len;
'@; New = @'
    buf_handle.priv_buffer_handle = rxbuff;
    /* QMX_SDIO_SPLIT_PSRAM: a frame split out of a bundle lives in PSRAM
     * and is released with heap_caps_free, not back into the mempool. */
    buf_handle.free_buf_handle    = qmx_pkt_free ? qmx_pkt_free : sdio_buffer_free;
    buf_handle.payload_len        = len;
'@ },
@{ Old = @'
static esp_err_t sdio_push_pkt_to_queue(uint8_t * rxbuff, uint16_t len, uint16_t offset)
{
'@; New = @'
/* QMX_SDIO_SPLIT_PSRAM: set only around qmx_sdio_push_stream()'s pushes. */
static void (*qmx_pkt_free)(void *);

static esp_err_t sdio_push_pkt_to_queue(uint8_t * rxbuff, uint16_t len, uint16_t offset)
{
'@ },
@{ Old = @'
    p = sdio_buffer_alloc(MEMSET_REQUIRED);
    if (!p) {
        qmx_split_drops++;
        return;
    }
    memcpy(p, s, psize);
    if (!is_valid_sdio_rx_packet(p, &vlen, &voff)) {
        sdio_buffer_free(p);
        qmx_split_drops++;
        return;
    }
    sdio_push_pkt_to_queue(p, vlen, voff);
'@; New = @'
/* QMX_SDIO_SPLIT_PSRAM: PSRAM, not the never-shrinking DMA mempool. */
p = heap_caps_malloc(psize, MALLOC_CAP_SPIRAM | MALLOC_CAP_8BIT);
if (!p) {
    qmx_split_drops++;
    return;
}
memcpy(p, s, psize);
if (!is_valid_sdio_rx_packet(p, &vlen, &voff)) {
    heap_caps_free(p);
    qmx_split_drops++;
    return;
}
qmx_pkt_free = heap_caps_free;
sdio_push_pkt_to_queue(p, vlen, voff);
qmx_pkt_free = NULL;
'@ }
)

# Stage 2 - anchors are stage 1's own output, so it always runs after it.
$stage2 = @(
@{ Old = @'
    buf_handle.free_buf_handle    = qmx_pkt_free ? qmx_pkt_free : sdio_buffer_free;
'@; New = @'
    buf_handle.free_buf_handle    = qmx_free ? qmx_free : sdio_buffer_free;
'@ },
@{ Old = @'
    interface_buffer_handle_t buf_handle;

    h = (struct esp_payload_header *)rxbuff;
'@; New = @'
    interface_buffer_handle_t buf_handle;
/* QMX_SDIO_RXQ_PSRAM: queue a PSRAM copy and give the DMA mempool block
 * back at once - the pool never returns memory to the heap, so blocks
 * parked in the RX queue under a sustained stream ate all DMA RAM. */
void (*qmx_free)(void *) = qmx_pkt_free;
if (!qmx_free) {
    uint8_t *cp = heap_caps_malloc((size_t)offset + len, MALLOC_CAP_SPIRAM | MALLOC_CAP_8BIT);
    if (cp) {
        memcpy(cp, rxbuff, (size_t)offset + len);
        sdio_buffer_free(rxbuff);
        rxbuff = cp;
        qmx_free = heap_caps_free;
    }
}
'@ }
)

```

```

    }

    h = (struct esp_payload_header *)rxbuff;
    '@ }
    )

    $pairs = @()
    if (-not $content.Contains("QMX_SDIO_SPLIT_PSRAM")) { $pairs += $stage1 }
    $pairs += $stage2

    foreach ($p in $pairs) {
        $old = $p.Old -replace "`r`n", "`n"
        $new = $p.New -replace "`r`n", "`n"
        $n = ([regex]::Matches($content, [regex]::Escape($old))).Count
        if ($n -ne 1) {
            Write-Host "Anchor found $n times (expected 1) - esp_hosted or the split patch changed; patch by hand and upda
↪ te this script:" -ForegroundColor Red
            Write-Host ($old.Substring(0, [Math]::Min(160, $old.Length)))
            exit 1
        }
        $content = $content.Replace($old, $new)
    }

[System.IO.File]::WriteAllText($target, $content, (New-Object System.Text.UTF8Encoding $false))
Write-Host "Patched sdio_drv.c: received SDIO frames are queued in PSRAM, not in the DMA mempool." -ForegroundColor Gr
↪ een
Write-Host "    $target"

```

tools/mac-build-flash.sh (new)

```

#!/bin/bash
# Build this firmware on a Mac and flash it onto a Tab5 over USB.
#
#   bash ~/src/qmx-panadapter-v1.16/tools/mac-build-flash.sh          build + flash
#   bash ~/src/qmx-panadapter-v1.16/tools/mac-build-flash.sh --build build only
#
# Uses the ESP-IDF in $IDF_PATH or ~/esp/esp-idf, and an installed PowerShell
# (pwsh, or set PWSH=/path/to/pwsh), because the project's standing patches are
# PowerShell scripts (tools/patches/apply*.ps1); tools/check_patches.py refuses
# the build if any patch is missing. Flashing is a NORMAL flash: settings,
# WiFi, memories and the QSO log are kept. Everything is logged to
# build/mac-build-flash.log.

REPO="$(cd "$(dirname "$0")/.." && pwd)"
mkdir -p "$REPO/build"
LOG="$REPO/build/mac-build-flash.log"
exec > >(tee "$LOG") 2>&1
step() { echo; echo "=== $*"; }
fail() { echo; echo "FAILED: $*"; echo "(log: $LOG)"; exit 1; }
fwver() { sed -n 's/.**project_version: **\[([^\]]*)\].*/\1/p' "$REPO/build/project_description.json" | head -1; }
BUILD_ONLY=0; [ "$1" = "--build" ] && BUILD_ONLY=1

step "Mac and project"
date; sw_vers 2>/dev/null | tr '\n' ' '; echo; uname -m
cd "$REPO" || fail "no project folder"
echo "project: $REPO"
echo "branch: $(git rev-parse --abbrev-ref HEAD)   version: $(git describe --tags --dirty)"

step "PowerShell (for the standing patches)"
# Use the PowerShell that's already installed. Look on PATH, in the usual install
# places, and in your login shell's PATH (Terminal's zsh may set it up there).
PWSH="{PWSH:-}"
for c in pwsh pwsh-preview /usr/local/bin/pwsh /opt/homebrew/bin/pwsh /usr/local/bin/pwsh-preview \
    /opt/homebrew/bin/pwsh-preview /usr/local/microsoft/powershell/7/pwsh \
    /usr/local/microsoft/powershell/7-preview/pwsh "$HOME/.dotnet/tools/pwsh" \
    /Applications/PowerShell.app/Contents/MacOS/pwsh; do
    [ -n "$PWSH" ] && break
    command -v "$c" >/dev/null 2>&1 && PWSH="$(command -v "$c")"
done
[ -n "$PWSH" ] || PWSH="$(zsh -lic 'command -v pwsh || command -v pwsh-preview' 2>/dev/null | tail -1)"
[ -n "$PWSH" ] && [ -x "$PWSH" ] || fail "can't find PowerShell (pwsh). In a PowerShell window run (Get-Process -Id
↪ $PID).Path and re-run this as: PWSH=<that path> bash $0"
echo "PowerShell: $PWSH"
"$PWSH" -NoProfile -Command '$PSVersionTable.PSVersion.ToString()' || fail "pwsh does not run"

step "ESP-IDF"
export IDF_PATH="${IDF_PATH:-$HOME/esp/esp-idf}"
[ -f "$IDF_PATH/export.sh" ] || fail "ESP-IDF not found at $IDF_PATH"
echo "IDF tree: $(git -C "$IDF_PATH" describe --tags --dirty 2>/dev/null)"
# PlatformIO puts its own python3 first on PATH; ESP-IDF must not pick that one.
PATH="$(echo "$PATH" | tr ':' '\n' | grep -v '/.platformio/penv' | paste -sd: -)"
# Use the ESP-IDF Python environment that already exists for this IDF (made by
# install.sh with whichever Python was current then), and that Python itself.

```

```

if [ -z "$IDF_PYTHON_ENV_PATH" ]; then
    IDF_PYTHON_ENV_PATH="$(ls -d "$HOME/.espressif/python_env/idf5.4_py"*_env 2>/dev/null | sort -V | tail -1)"
fi
[ -x "$IDF_PYTHON_ENV_PATH/bin/python" ] || fail "no ESP-IDF Python environment in ~/.espressif/python_env - run $IDF_
PATH/install.sh esp32p4"
export IDF_PYTHON_ENV_PATH
ENVPY="$(cd "$IDF_PYTHON_ENV_PATH/bin" && pwd -P)/python"
BASEPY="$(readlink -f "$IDF_PYTHON_ENV_PATH/bin/python" 2>/dev/null)"
[ -x "$BASEPY" ] && "$BASEPY" -V >/dev/null 2>&1 || fail "the Python behind $IDF_PYTHON_ENV_PATH is gone ($BASEPY) - r
un $IDF_PATH/install.sh esp32p4 once to rebuild it"
mkdir -p build/pyshim && ln -sf "$BASEPY" build/pyshim/python3 && PATH="$REPO/build/pyshim:$PATH"
echo "ESP-IDF Python env: $IDF_PYTHON_ENV_PATH"
echo "python3 for export: $(command -v python3) ($(python3 --version 2>&1))"
. "$IDF_PATH/export.sh" > build/idf-export.log 2>&1
RC=$?
if [ $RC -ne 0 ] || ! command -v idf.py >/dev/null 2>&1; then
    echo "--- ESP-IDF export.sh output (exit code $RC):"; cat build/idf-export.log
    fail "ESP-IDF export.sh did not set up idf.py"
fi
tail -3 build/idf-export.log
idf.py --version
case "$(idf.py --version)" in *v5.4*) ;; *) echo "WARNING: the project and its patches are written for ESP-IDF v5.4.4"
;; esac

step "Fetch components (idf.py reconfigure)"
idf.py reconfigure > build/reconfigure.log 2>&1 || { tail -40 build/reconfigure.log; fail "idf.py reconfigure"; }
tail -3 build/reconfigure.log
ls managed_components

step "Apply the standing patches"
for s in tools/patches/apply_*.ps1; do
    echo "--- $(basename "$s")"
    "$PSWSH" -NoProfile -ExecutionPolicy Bypass -File "$s" || fail "$(basename "$s")"
done
step "Check the standing patches"
python tools/check_patches.py || fail "check_patches.py"

step "Build"
idf.py build > build/build.log 2>&1 || { grep -E "error|Error|FAILED" build/build.log | head -40; tail -30 build/build
.log; fail "idf.py build"; }
tail -8 build/build.log
echo "Firmware version: $(fwver)"
ls -l build/*.bin
[ "$BUILD_ONLY" = 1 ] && { step "Done (build only)"; exit 0; }

step "Flash the Tab5"
PORTS=( $(ls /dev/cu.usbmodem* 2>/dev/null) )
echo "USB serial ports: ${PORTS[*]:-none}"
[ ${#PORTS[@]} -gt 0 ] || fail "no Tab5 found - plug it in with a USB-C DATA cable and run this again"
PORT=${PORTS[0]}
if [ ${#PORTS[@]} -gt 1 ]; then
    echo "More than one USB serial device is plugged in:"
    select PORT in "${PORTS[@]}; do [ -n "$PORT" ] && break; done
fi
echo
echo "About to flash the Tab5 on $PORT (normal flash - settings are kept)."
echo "Nothing should be plugged into the Tab5's own USB port: a flash wedges an attached QMX."
read -r -p "Press Enter to flash, or Ctrl+C to cancel..."
idf.py -p "$PORT" -b 460800 flash || fail "flash (try another USB-C cable, or reboot the Tab5 and run again)"

step "SUCCESS"
echo "Flashed $(fwver). The Tab5 is restarting."

```

tools/mac-capture-serial.sh (new)

```

#!/bin/bash
# Record the Tab5's USB serial output (boot messages, crash reasons) for a while.
# bash ~/src/qmx-panadapter-v1.16/tools/mac-capture-serial.sh [seconds] (default 90)
# Output: build/serial-capture.log in this project. Uses ESP-IDF's Python (pyserial).
# Note: on macOS opening the port restarts the Tab5 (USB-serial-JTAG reset), so
# the capture always starts from a fresh boot. Setting DTR/RTS first does not help.
REPO="$(cd "$(dirname "$0")/.." && pwd)"; mkdir -p "$REPO/build"
OUT="$REPO/build/serial-capture.log"; SECS="${1:-90}"
PY="$(ls -d "$HOME/.espressif/python_env/idf5.4_py"*_env 2>/dev/null | sort -V | tail -1)/bin/python"
[ -x "$PY" ] || { echo "ESP-IDF Python not found"; exit 1; }
PORT="$(ls /dev/cu.usbmodem* 2>/dev/null | head -1)"
[ -n "$PORT" ] || { echo "No Tab5 USB serial port found - is it plugged in with a data cable?"; exit 1; }
echo "Recording $PORT for $SECS s into $OUT ..."
"$PY" - "$PORT" "$SECS" "$OUT" <<'PYEOF'
import sys, time, serial
port, secs, out = sys.argv[1], float(sys.argv[2]), sys.argv[3]
end = time.time() + secs
with open(out, "wb") as f:

```

```
f.write("# capture of %s started %s\n" % (port, time.ctime())).encode())
while time.time() < end:
    try:
        s = serial.Serial(port, 115200, timeout=0.5)
        s.dtr = False; s.rts = False          # never reset the board ourselves
        while time.time() < end:
            d = s.read(4096)
            if d: f.write(d); f.flush(); sys.stdout.buffer.write(d); sys.stdout.flush()
        except (serial.SerialException, OSError) as e:
            # the port disappears while the board reboots - wait and reopen
            f.write("\n# [port gone: %s] %s\n" % (e.__class__.__name__, time.ctime())).encode()); f.flush()
            time.sleep(0.3)
print("\nSaved:", out)
PYEOF
```

12. Application listings (NAS relay)

~/src/qmx-panadapter-v1.16/tools/nas-relay, deployed to the Synology NAS.

tools/nas-relay/qmx_nas_relay.py

```
#!/usr/bin/env python3
"""
```

qmx_nas_relay.py - serve a QMX/QMX+ plugged into a Synology NAS to the QMX-Panadapter (M5Stack Tab5) over the network.

```
QMX --USB--> Synology NAS (this relay, run natively or in Docker)
| QMXR/1 framed TCP, port 7355
v
Tab5 QMX-Panadapter (Radio source: NAS)
```

The relay owns the QMX directly from userspace, so it needs no DSM kernel drivers (no cdc_acm / snd-usb-audio). It uses libusb-1.0 (python-libusb1) when that loads, and otherwise talks to Linux usbfs (/dev/bus/usb) itself through ioctl() - which is what runs on NAS models without libusb-1.0, such as the DS416j (QMX_BACKEND=libusb|usbfs forces one):

- * CAT: CDC-ACM data interface, bulk IN/OUT endpoints (38400 8N1 set for completeness - it's a virtual COM port).
- * I/Q: UAC streaming interface, isochronous IN, 48 kHz stereo 24-bit. Converted to int16 exactly like the firmware's USB path (s24 >> 8).
- * Terminal: the QMX's optional SECOND serial port (radio set to "USB serial ports" = 2 or 3; on USB it is interface 5), opened on request for the Tab5's "Radio menus" and carried as its own stream, never mixed with CAT - exactly why the firmware uses port 2 on USB.
- * Port 3: with "USB serial ports" = 3 the QMX's THIRD serial port is served as a plain TCP serial port (default 7356) for PC software - WSJT-X, rigctld, a terminal - so a PC can control the radio at the same time as the Tab5, on its own port.

Wire format (QMXR/1): every frame = 4-byte header + payload

```
u8 type 1=HELLO 2=IQ 3=CAT 4=PING
        5=TERM_OPEN 6=TERM_STATUS 7=TERM_DATA 8=TERM_CLOSE
u8 flags 0
u16 len payload length, little-endian
```

Tab5 -> relay : HELLO "QMXR/1 ...", then CAT bytes

relay -> Tab5 : HELLO "QMXR/1 rate=48000 ch=2 fmt=s16le term=1", then IQ / CAT / PING

Terminal (second serial port), Tab5-initiated:

```
Tab5 -> relay : TERM_OPEN          relay -> Tab5 : TERM_STATUS u8 ok [+ reason]
both ways    : TERM_DATA bytes      Tab5 -> relay : TERM_CLOSE
relay -> Tab5 : TERM_STATUS 0 unsolicited if the port goes away while open
```

The relay only answers the Tab5's HELLO after it has opened the QMX, and it closes the connection as soon as the QMX goes away, so for the Tab5 an open session means "radio present" - exactly like a USB plug/unplug.

One Tab5 at a time and one port-3 client at a time; a new connection replaces an old one (e.g. after a reboot left a half-dead socket). The QMX is opened when the first of them connects and released when the last one leaves.

Requirements: Python 3.8+. Nothing else on Linux (usbfs backend); on other systems libusb-1.0 + `pip install libusb1` (the Dockerfile installs both).

```
"""
from __future__ import annotations
```

```
import argparse
import collections
import ctypes
import errno
import logging
import math
import os
import queue
import select
import socket
import struct
import sys
import threading
import time
```

```
log = logging.getLogger("qmx-nas-relay")
```

```
PROTO_HELLO, PROTO_IQ, PROTO_CAT, PROTO_PING = 1, 2, 3, 4
```

```
PROTO_TERM_OPEN, PROTO_TERM_STATUS, PROTO_TERM_DATA, PROTO_TERM_CLOSE = 5, 6, 7, 8
```

```
HDR = struct.Struct("<BBH")
```

```

DEFAULT_PORT = 7355
DEFAULT_SERIAL_PORT = 7356      # QMX port 3 as raw TCP serial for PCs
QMX_VID, QMX_PID = 0x0483, 0xA34C
SAMPLE_RATE = 48000
FRAME_PAIRS = 480               # 10 ms of I/Q per IQ frame (1920 bytes)
IQ_QUEUE_MAX = 3                # I/Q frames (30 ms) waiting to be sent; beyond that drop I/Q, never CAT
TAB5_SNDBUF = 8192              # kernel send buffer per Tab5 (Linux doubles it): keeps the
                                # backlog - and so the CAT reply delay - under ~0.2 s
ISO_TRANSFERS = 8               # in flight
ISO_PACKETS = 8                 # 1 ms each -> 8 ms per transfer
CAT_BAUD = 38400

```

```

def frame(ftype: int, payload: bytes = b'') -> bytes:
    if len(payload) > 0xFFFF:
        raise ValueError("payload too large")
    return HDR.pack(ftype, 0, len(payload)) + payload

```

```

def recv_exact(sock: socket.socket, n: int) -> bytes:
    buf = bytearray()
    while len(buf) < n:
        chunk = sock.recv(n - len(buf))
        if not chunk:
            raise ConnectionError("peer closed")
        buf += chunk
    return bytes(buf)

```

```

def to_s16(raw: bytes, subframe: int) -> bytes:
    """Interleaved PCM -> interleaved int16 LE, matching the firmware's USB
    path: 24-bit samples keep (s24 >> 8) = the upper two bytes; 32-bit keep
    (s32 >> 16); 16-bit pass through."""
    if subframe == 2:
        return bytes(raw[: len(raw) // 2 * 2])
    n = len(raw) // subframe
    out = bytearray(n * 2)
    out[0::2] = raw[subframe - 2: n * subframe: subframe]
    out[1::2] = raw[subframe - 1: n * subframe: subframe]
    return bytes(out)

```

```

# -----
# Radio backends
# -----

```

```

class RadioGone(Exception):
    pass

```

```

class QmxUsb:
    """The real QMX over libusb. on_iq(bytes s16) / on_cat(bytes) are called
    from the libusb event thread; on_gone() once if the device disappears."""

    def __init__(self, on_iq, on_cat, on_gone, vid=QMX_VID, pid=QMX_PID):
        import usb1 # imported here so --fake works without libusb
        _load_libusb(usb1)
        self.usb1 = usb1
        self.on_iq, self.on_cat, self.on_gone = on_iq, on_cat, on_gone
        self.vid, self.pid = vid, pid
        self.ctx = usb1.USBContext()
        self.h = None
        self.running = False
        self.claimed: list[int] = []
        self.transfers = []
        self.gone_reported = False
        self.evt_thread = None
        self.cdc_ports = [] # [(comm_if, data_if, bulk_in, bulk_out)] in interface order
        # Extra serial ports, by index into cdc_ports: 1 = port 2 (terminal), 2 = port 3 (PC).
        self.extra: dict[int, dict] = {}

    # -- descriptor discovery -----
    def _discover(self, dev):
        return discover_qmx(dev[0])

    # -- open / close -----
    def open(self):
        usb1 = self.usb1
        h = self.ctx.openByVendorIDAndProductID(self.vid, self.pid, skip_on_error=True)
        if h is None:
            raise RadioGone("QMX (%04x:%04x) not found on the NAS USB bus" % (self.vid, self.pid))
        self.h = h
        dev = h.getDevice()
        self.cdc_ports, uac2, aud = self._discover(dev)

```

```

if not self.cdc_ports or aud is None:
    self.close()
    raise RadioGone("QMX descriptors not recognised (CAT=%s audio=%s)" % (self.cdc_ports, aud))
cdc_ctrl, cdc_data, self.bulk_in, self.bulk_out = self.cdc_ports[0]
as_if, alt, self.iso_ep, self.iso_mps, ch, self.subframe, bits, rates = aud
log.info("QMX found: CAT if%d (IN 0x%02x OUT 0x%02x), I/Q if%d alt%d EP 0x%02x mps=%d %dch %d-bit/%dB %s%s",
        cdc_data, self.bulk_in, self.bulk_out, as_if, alt, self.iso_ep, self.iso_mps,
        ch, bits, self.subframe, rates or "", " UAC2" if uac2 else "")
n = len(self.cdc_ports)
log.info("QMX USB serial ports: %d (%s)", n, ", ".join(
    "port %d = if%s/if%d" % (i + 1, c, d) for i, (c, d, _, _) in enumerate(self.cdc_ports)))
if n < 2:
    log.info("Radio menus need 'USB serial ports' = 2 or 3 on the radio")
if n < 3:
    log.info("PC serial (port 3) needs 'USB serial ports' = 3 on the radio")
if ch != 2:
    log.warning("audio stream has %d channels, expected 2 (I/Q)", ch)
try:
    h.setAutoDetachKernelDriver(True) # if DSM bound cdc_acm / snd-usb-audio
except Exception:
    pass
try:
    for i in [x for x in (cdc_ctrl, cdc_data, as_if) if x is not None]:
        h.claimInterface(i)
        self.claimed.append(i)
except usb1.USBErrorBusy:
    self.close()
    raise RadioGone("QMX is in use by another program on the NAS - stop it, then reconnect")
except usb1.USBErrorAccess:
    self.close()
    raise RadioGone("no permission for USB - run the container privileged")

# CAT: line coding 38400 8N1 + DTR/RTS (same as the firmware's USB path).
if cdc_ctrl is not None:
    try:
        h.controlWrite(0x21, 0x20, 0, cdc_ctrl, struct.pack("<IBBB", CAT_BAUD, 0, 0, 8), timeout=500)
        h.controlWrite(0x21, 0x22, 0x0003, cdc_ctrl, b"", timeout=500)
    except usb1.USBError as e:
        log.warning("CDC line setup: %s (continuing)", e)

# I/Q: select the streaming alt setting, ask for 48 kHz (UAC1 endpoint request).
h.setInterfaceAltSetting(as_if, alt)
self.as_if = as_if
if not uac2:
    try:
        h.controlWrite(0x22, 0x01, 0x0100, self.iso_ep, SAMPLE_RATE.to_bytes(3, "little"), timeout=500)
    except usb1.USBError as e:
        log.info("sample-rate SET_CUR not accepted (%s) - device default used", e)

self.running = True
self.gone_reported = False
# Isochronous I/Q transfers.
for _ in range(ISO_TRANSFERS):
    t = h.getTransfer(iso_packets=ISO_PACKETS)
    t.setIsochronous(self.iso_ep, ISO_PACKETS * self.iso_mps, callback=self._iso_cb)
    t.submit()
    self.transfers.append(t)
# CAT bulk IN.
for _ in range(2):
    t = h.getTransfer()
    t.setBulk(self.bulk_in, 64, callback=self._bulk_cb)
    t.submit()
    self.transfers.append(t)
self.evt_thread = threading.Thread(target=self._events, name="usb-events", daemon=True)
self.evt_thread.start()

def _events(self):
    while self.running:
        try:
            self.ctx.handleEventsTimeout(0.1)
        except self.usb1.USBErrorInterrupted:
            continue
        except Exception as e:
            log.warning("libusb event loop: %s", e)
            self._gone()
            return

def _gone(self):
    if not self.gone_reported:
        self.gone_reported = True
        log.warning("QMX unplugged / powered off")
        self.running = False
        self.on_gone()

def _resubmit_or_gone(self, t):

```

```

        if not self.running:
            return
        try:
            t.submit()
        except self.usbl.USBError as e:
            log.warning("USB resubmit failed: %s", e)
            self._gone()

def _iso_cb(self, t):
    usbl = self.usbl
    st = t.getStatus()
    if st == usbl.TRANSFER_COMPLETED:
        raw = bytearray()
        for pst, buf in t.iterISO():
            if pst == usbl.TRANSFER_COMPLETED and buf:
                raw += buf
        if raw:
            self.on_iq(to_s16(raw, self.subframe))
    elif st == usbl.TRANSFER_NO_DEVICE:
        self._gone()
        return
    elif st == usbl.TRANSFER_CANCELLED:
        return
    self._resubmit_or_gone(t)

def _bulk_cb(self, t):
    usbl = self.usbl
    st = t.getStatus()
    if st == usbl.TRANSFER_COMPLETED:
        n = t.getActualLength()
        if n:
            self.on_cat(bytes(t.getBuffer()[0:n]))
    elif st == usbl.TRANSFER_NO_DEVICE:
        self._gone()
        return
    elif st == usbl.TRANSFER_CANCELLED:
        return
    self._resubmit_or_gone(t)

# -- extra serial ports (index 1 = port 2 terminal, index 2 = port 3 PC) --
def serial_count(self) -> int:
    return len(self.cdc_ports)

def serial_open(self, idx, on_data):
    """Claim the QMX's serial port number idx+1 and start reading it.
    Returns (ok, reason)."""
    usbl = self.usbl
    name = "port %d" % (idx + 1)
    st = self.extra.get(idx)
    if st and st["cb"]:
        st["cb"] = on_data
        return True, ""
    if len(self.cdc_ports) <= idx:
        return False, ("the radio has %d USB serial port%s - set 'USB serial ports' to %d"
                       % (len(self.cdc_ports), "" if len(self.cdc_ports) == 1 else "s", idx + 1))
    comm, data, bin_, bout = self.cdc_ports[idx]
    st = {"ifs": [], "transfers": [], "out": bout, "cb": None}
    self.extra[idx] = st
    try:
        for i in [x for x in (comm, data) if x is not None]:
            self.h.claimInterface(i)
            st["ifs"].append(i)
    except usbl.USBErrorBusy:
        self._serial_release(idx)
        return False, "%s is in use by another program on the NAS" % name
    except usbl.USBError as e:
        self._serial_release(idx)
        return False, "%s claim failed: %s" % (name, e)
    if comm is not None:
        try:
            self.h.controlWrite(0x21, 0x20, 0, comm, struct.pack("<IBBB", CAT_BAUD, 0, 0, 8), timeout=500)
            self.h.controlWrite(0x21, 0x22, 0x0003, comm, b"", timeout=500)
        except usbl.USBError as e:
            log.warning("%s line setup: %s (continuing)", name, e)
    st["cb"] = on_data
    for _ in range(2):
        t = self.h.getTransfer()
        t.setBulk(bin_, 512, callback=self._serial_cb, user_data=idx)
        t.submit()
        st["transfers"].append(t)
    log.info("QMX %s open (if%s/if%d)", name, comm, data)
    return True, ""

def _serial_cb(self, t):

```

```

usb1 = self.usb1
idx = t.getUserData()
st = self.extra.get(idx)
status = t.getStatus()
if status == usb1.TRANSFER_COMPLETED:
    n = t.getActualLength()
    cb = st["cb"] if st else None
    if n and cb:
        cb(bytes(t.getBuffer()[0:n]))
elif status == usb1.TRANSFER_NO_DEVICE:
    self._gone()
    return
elif status == usb1.TRANSFER_CANCELLED:
    return
if st and st["cb"]:
    self._resubmit_or_gone(t)

def serial_send(self, idx, data: bytes):
    st = self.extra.get(idx)
    if not self.h or not st or not st["cb"]:
        return
    try:
        self.h.bulkWrite(st["out"], data, timeout=500)
    except self.usb1.USBErrorNoDevice:
        self._gone()
        raise RadioGone("QMX gone")
    except self.usb1.USBError as e:
        log.warning("port %d write: %s", idx + 1, e)

def serial_close(self, idx):
    st = self.extra.get(idx)
    if not st:
        return
    st["cb"] = None
    for t in st["transfers"]:
        try:
            if t.isSubmitted():
                t.cancel()
        except Exception:
            pass
    deadline = time.time() + 1.0
    while time.time() < deadline and any(_submitted(t) for t in st["transfers"]):
        time.sleep(0.02) # the event thread completes the cancellations
    st["transfers"] = []
    self._serial_release(idx)
    log.info("QMX port %d closed", idx + 1)

def _serial_release(self, idx):
    st = self.extra.pop(idx, None)
    if not st:
        return
    for i in st["ifs"]:
        try:
            self.h.releaseInterface(i)
        except Exception:
            pass

# Terminal = port 2 (kept as names the Tab5 session uses).
def term_open(self, on_data):
    return self.serial_open(1, on_data)

def term_send(self, data: bytes):
    self.serial_send(1, data)

def term_close(self):
    self.serial_close(1)

def send_cat(self, data: bytes):
    if not self.h:
        raise RadioGone("closed")
    try:
        self.h.bulkWrite(self.bulk_out, data, timeout=500)
    except self.usb1.USBErrorNoDevice:
        self._gone()
        raise RadioGone("QMX gone")

def close(self):
    self.running = False
    for st in self.extra.values():
        st["cb"] = None
        self.transfers += st["transfers"]
        self.claimed += st["ifs"]
    self.extra = {}
    for t in self.transfers:

```

```

        try:
            if t.isSubmitted():
                t.cancel()
        except Exception:
            pass
    # Let cancellations complete before freeing.
    deadline = time.time() + 1.0
    while time.time() < deadline and any(_submitted(t) for t in self.transfers):
        try:
            self.ctx.handleEventsTimeout(0.05)
        except Exception:
            break
    if self.evt_thread and self.evt_thread is not threading.current_thread():
        self.evt_thread.join(timeout=1.0)
    self.transfers = []
    if self.h:
        try:
            if getattr(self, "as_if", None) is not None:
                self.h.setInterfaceAltSetting(self.as_if, 0)
        except Exception:
            pass
        for i in self.claimed:
            try:
                self.h.releaseInterface(i)
            except Exception:
                pass
        self.claimed = []
        try:
            self.h.close()
        except Exception:
            pass
        self.h = None

def _load_libusb(usb1):
    """Use QMX_LIBUSB=/path/libusb-1.0.so.0 if set (e.g. a NAS whose libusb is
    not on the default search path); otherwise python-libusb1's own lookup."""
    path = os.environ.get("QMX_LIBUSB")
    if path and not getattr(usb1, "_qmx_loaded", False):
        usb1.loadLibrary(path)
        usb1._qmx_loaded = True

def _submitted(t) -> bool:
    try:
        return t.isSubmitted()
    except Exception:
        return False

def _split_descs(extra) -> list[bytes]:
    """getExtra() may return one blob or a list of descriptor blobs."""
    blobs = extra if isinstance(extra, (list, tuple)) else [extra]
    out = []
    for b in blobs:
        b = bytes(b)
        i = 0
        while i + 2 <= len(b) and b[i] >= 2:
            out.append(b[i:i + b[i]])
            i += b[i]
    return out

def discover_qmx(config):
    """config = list of interfaces, each a list of alt settings (python-libusb1 or
    usbfs shim objects). Returns (cdc_ports, uac2, best audio-in candidate)."""
    comms, datas = [], []
    uac2 = False
    best = None # (iface, alt, ep, mps, channels, subframe, bits, rates)
    for iface in config:
        for s in iface:
            cls, sub, num, alt = s.getClass(), s.getSubClass(), s.getNumber(), s.getAlternateSetting()
            if cls == 0x02 and sub == 0x02 and alt == 0:
                comms.append(num)
            elif cls == 0x0A and alt == 0:
                bin_ = bout = None
                for ep in s:
                    a, attr = ep.getAddress(), ep.getAttributes() & 3
                    if attr == 2:
                        if a & 0x80:
                            bin_ = a
                        else:
                            bout = a
                if bin_ and bout:

```

```

        datas.append((num, bin_, bout))
    elif cls == 0x01 and sub == 0x01 and s.getProtocol() == 0x20:
        uac2 = True
    elif cls == 0x01 and sub == 0x02 and alt > 0:
        iso_in = [ep for ep in s
                   if (ep.getAddress() & 0x80) and (ep.getAttributes() & 3) == 1]
        if not iso_in:
            continue
        ch, subframe, bits, rates = 2, 3, 24, []
        for d in _split_descs(s.getExtra()):
            # UAC1 Format Type I: len, 0x24, 0x02, 0x01, nCh, subframe, bits, nFreq, freqs...
            if len(d) >= 8 and d[1] == 0x24 and d[2] == 0x02 and d[3] == 0x01:
                ch, subframe, bits, nf = d[4], d[5], d[6], d[7]
                rates = [d[8 + 3 * i] | d[9 + 3 * i] << 8 | d[10 + 3 * i] << 16
                        for i in range(nf) if 10 + 3 * i < len(d)]
            # UAC2 Format Type I: len, 0x24, 0x02, 0x01, subslot, bits
            elif len(d) == 6 and d[1] == 0x24 and d[2] == 0x02 and d[3] == 0x01:
                subframe, bits = d[4], d[5]
            # UAC2 AS GENERAL carries bNrChannels at offset 10
            elif len(d) >= 11 and d[1] == 0x24 and d[2] == 0x01 and d[0] == 16:
                ch = d[10]
        ep = iso_in[0]
        cand = (num, alt, ep.getAddress(), ep.getMaxPacketSize() & 0x7FF, ch, subframe, bits, rates)
        # Prefer stereo; the QMX's RX/I-Q stream is the stereo IN one.
        if best is None or (ch == 2 and best[4] != 2):
            best = cand
    # Pair each CDC comm interface with the data interface that follows it.
    # The QMX's functions are NOT contiguous (CAT 0-1, audio 2-4, port 2 at
    # 5-6), so pair by order, not by number+1.
    comms.sort()
    datas.sort()
    ports = []
    for i, (dnum, bin_, bout) in enumerate(datas):
        ports.append((comms[i] if i < len(comms) else None, dnum, bin_, bout))
    return ports, uac2, best

# -----
# Linux usbfs backend - no libusb needed (DSM ships only libusb-0.1)
# Structures and ioctl numbers follow <linux/usbdevice_fs.h>; ioctl numbers
# use the generic _IOC encoding of x86, ARM and arm64.
# -----

c_u8, c_u16, c_u32, c_int, c_void_p = ctypes.c_uint8, ctypes.c_uint16, ctypes.c_uint32, ctypes.c_int, ctypes.c_void_p

# ---- <linux/usbdevice_fs.h> -----

class CtrlTransfer(ctypes.Structure):
    _fields_ = [("bRequestType", c_u8), ("bRequest", c_u8), ("wValue", c_u16), ("wIndex", c_u16),
                ("wLength", c_u16), ("timeout", c_u32), ("data", c_void_p)]

class BulkTransfer(ctypes.Structure):
    _fields_ = [("ep", c_u32), ("len", c_u32), ("timeout", c_u32), ("data", c_void_p)]

class SetInterface(ctypes.Structure):
    _fields_ = [("interface", c_u32), ("altsetting", c_u32)]

class IsoPacketDesc(ctypes.Structure):
    _fields_ = [("length", c_u32), ("actual_length", c_u32), ("status", c_u32)]

_URB_FIELDS = [("type", c_u8), ("endpoint", c_u8), ("status", c_int), ("flags", c_u32),
                ("buffer", c_void_p), ("buffer_length", c_int), ("actual_length", c_int),
                ("start_frame", c_int), ("number_of_packets", c_int), ("error_count", c_int),
                ("signr", c_u32), ("usercontext", c_void_p)]

class Urb(ctypes.Structure):
    _fields_ = _URB_FIELDS

def iso_urb_type(npackets):
    class IsoUrb(ctypes.Structure):
        _fields_ = _URB_FIELDS + [("iso_frame_desc", IsoPacketDesc * npackets)]
    return IsoUrb

class UsbdevfsIoctl(ctypes.Structure):
    _fields_ = [("ifno", c_int), ("ioctl_code", c_int), ("data", c_void_p)]
def _IOC(d, t, nr, size):

```

```

    return (d << 30) | (size << 16) | (ord(t) << 8) | nr

_R, _W = 2, 1
USBDEVFS_CONTROL = _IOC(_R | _W, "U", 0, ctypes.sizeof(CtrlTransfer))
USBDEVFS_BULK = _IOC(_R | _W, "U", 2, ctypes.sizeof(BulkTransfer))
USBDEVFS_SETINTERFACE = _IOC(_R, "U", 4, ctypes.sizeof(SetInterface))
USBDEVFS_SUBMITURB = _IOC(_R, "U", 10, ctypes.sizeof(Urb))
USBDEVFS_DISCARDURB = _IOC(0, "U", 11, 0)
USBDEVFS_REAPURBDELAY = _IOC(_W, "U", 13, ctypes.sizeof(c_void_p))
USBDEVFS_CLAIMINTERFACE = _IOC(_R, "U", 15, ctypes.sizeof(ctypes.c_uint))
USBDEVFS_RELEASEINTERFACE = _IOC(_R, "U", 16, ctypes.sizeof(ctypes.c_uint))
USBDEVFS_IOCTL = _IOC(_R | _W, "U", 18, ctypes.sizeof(UsbdevfsIoctl))
USBDEVFS_DISCONNECT = _IOC(0, "U", 22, 0)
USBDEVFS_CONNECT = _IOC(0, "U", 23, 0)

URB_TYPE_ISO, URB_TYPE_BULK = 0, 3
URB_ISO_ASAP = 0x02

_libc = ctypes.CDLL(None, use_errno=True)
_ioctl = _libc.ioctl
_ioctl.argtypes = [c_int, ctypes.c_ulong, c_void_p]
_ioctl.restype = c_int

class UsbfsError(OSError):
    pass

def ioctl(fd, req, arg):
    """ioctl with a real pointer (fcntl.ioctl copies small buffers, which
    breaks async URBs whose address the kernel keeps)."""
    ptr = None if arg is None else (arg if isinstance(arg, int) else ctypes.addressof(arg))
    r = _ioctl(fd, req, ptr)
    if r < 0:
        e = ctypes.get_errno()
        raise UsbfsError(e, os.strerror(e))
    return r

# ---- device discovery from sysfs + raw descriptors -----
SYS_USB = "/sys/bus/usb/devices"

def usbfs_find_device(vid, pid):
    """Return (sysfs name, /dev/bus/usb path) of the first matching device."""
    try:
        names = sorted(os.listdir(SYS_USB))
    except OSError:
        return None
    for name in names:
        d = os.path.join(SYS_USB, name)
        try:
            v = int(open(os.path.join(d, "idVendor")).read(), 16)
            p = int(open(os.path.join(d, "idProduct")).read(), 16)
            if (v, p) != (vid, pid):
                continue
            bus = int(open(os.path.join(d, "busnum")).read())
            dev = int(open(os.path.join(d, "devnum")).read())
        except (OSError, ValueError):
            continue
        return name, "/dev/bus/usb/%03d/%03d" % (bus, dev)
    return None

def usbfs_list_devices():
    out = []
    try:
        names = sorted(os.listdir(SYS_USB))
    except OSError:
        return out
    for name in names:
        d = os.path.join(SYS_USB, name)
        try:
            v = int(open(os.path.join(d, "idVendor")).read(), 16)
            p = int(open(os.path.join(d, "idProduct")).read(), 16)
            bus = int(open(os.path.join(d, "busnum")).read())
            dev = int(open(os.path.join(d, "devnum")).read())
        except (OSError, ValueError):
            continue
        out.append((bus, dev, v, p, name))
    return out

class UsbfsEndpoint:

```

```

def __init__(self, addr, attrs, mps):
    self.a, self.attrs, self.mps = addr, attrs, mps

def getAddress(self):
    return self.a

def getAttributes(self):
    return self.attrs

def getMaxPacketSize(self):
    return self.mps

class UsbfsSetting(list):
    """An interface alternate setting; iterates over its endpoints. Same
    accessor names as python-libusb1, so the relay's discovery code is shared."""

    def __init__(self, num, alt, cls, sub, proto):
        super().__init__()
        self.v = (num, alt, cls, sub, proto)
        self.extra = b""

    def getNumber(self):
        return self.v[0]

    def getAlternateSetting(self):
        return self.v[1]

    def getClass(self):
        return self.v[2]

    def getSubClass(self):
        return self.v[3]

    def getProtocol(self):
        return self.v[4]

    def getExtra(self):
        return [self.extra] if self.extra else []

def usbfs_parse_config(raw: bytes):
    """Raw descriptors (device descriptor, then the first configuration) ->
    list of interfaces, each a list of Settings."""
    i = 0
    # skip the device descriptor
    if len(raw) >= 2 and raw[1] == 1:
        i = raw[0]
    ifaces: dict = {}
    cur = None
    seen_config = False
    while i + 2 <= len(raw):
        ln, typ = raw[i], raw[i + 1]
        if ln < 2:
            break
        d = raw[i:i + ln]
        if typ == 2:
            # CONFIGURATION
            if seen_config:
                break
            seen_config = True
        elif typ == 4 and ln >= 9:
            # INTERFACE
            cur = UsbfsSetting(d[2], d[3], d[5], d[6], d[7])
            ifaces.setdefault(d[2], []).append(cur)
        elif typ == 5 and ln >= 7 and cur is not None:
            # ENDPOINT
            cur.append(UsbfsEndpoint(d[2], d[3], d[4] | d[5] << 8))
        elif cur is not None and typ in (0x24, 0x25):
            # class-specific
            cur.extra += bytes(d)
        i += ln
    return [ifaces[k] for k in sorted(ifaces)]

# ---- the device -----

class UsbfsDevice:
    """Minimal async usbfs device: control/bulk sync calls plus bulk/iso URBs
    completed on a reaper thread that calls each URB's callback."""

    def __init__(self, path):
        self.path = path
        self.fd = os.open(path, os.O_RDWR)
        self.lock = threading.Lock()
        self.pending: dict = {}
        self.running = True
        self.gone = False

```

```

self.on_gone = None
self.reaper = threading.Thread(target=self._reap_loop, name="usbfs-reaper", daemon=True)
self.reaper.start()

def read_descriptors(self) -> bytes:
    os.lseek(self.fd, 0, os.SEEK_SET)
    chunks = []
    while True:
        b = os.read(self.fd, 4096)
        if not b:
            break
        chunks.append(b)
    return b"".join(chunks)

# interfaces
def detach_kernel_driver(self, ifno):
    req = UsbdevfsIoctl(ifno, USBDEVFS_DISCONNECT, None)
    try:
        ioctl(self.fd, USBDEVFS_IOCTL, req)
    except UsbfsError as e:
        if e.errno not in (errno.ENODATA, errno.ENOENT, errno.EINVAL):
            raise

def attach_kernel_driver(self, ifno):
    req = UsbdevfsIoctl(ifno, USBDEVFS_CONNECT, None)
    try:
        ioctl(self.fd, USBDEVFS_IOCTL, req)
    except UsbfsError:
        pass

def claim(self, ifno):
    self.detach_kernel_driver(ifno)
    ioctl(self.fd, USBDEVFS_CLAIMINTERFACE, ctypes.c_uint(ifno))

def release(self, ifno):
    try:
        ioctl(self.fd, USBDEVFS_RELEASEINTERFACE, ctypes.c_uint(ifno))
    except UsbfsError:
        pass

def set_altsetting(self, ifno, alt):
    ioctl(self.fd, USBDEVFS_SETINTERFACE, SetInterface(ifno, alt))

# synchronous transfers
def control_write(self, rt, req, val, idx, data: bytes, timeout_ms=500):
    buf = ctypes.create_string_buffer(bytes(data), max(1, len(data)))
    ct = CtrlTransfer(rt, req, val, idx, len(data), timeout_ms, ctypes.addressof(buf) if data else None)
    return ioctl(self.fd, USBDEVFS_CONTROL, ct)

def bulk_write(self, ep, data: bytes, timeout_ms=500):
    buf = ctypes.create_string_buffer(bytes(data), max(1, len(data)))
    bt = BulkTransfer(ep, len(data), timeout_ms, ctypes.addressof(buf))
    return ioctl(self.fd, USBDEVFS_BULK, bt)

# asynchronous URBs
def submit_bulk_in(self, ep, length, callback):
    buf = ctypes.create_string_buffer(length)
    urb = Urb()
    urb.type, urb.endpoint, urb.buffer, urb.buffer_length = URB_TYPE_BULK, ep, ctypes.addressof(buf), length
    self._submit(urb, buf, callback)

def submit_iso_in(self, ep, npackets, mps, callback):
    T = iso_urb_type(npackets)
    buf = ctypes.create_string_buffer(npackets * mps)
    urb = T()
    urb.type, urb.endpoint, urb.flags = URB_TYPE_ISO, ep, URB_ISO_ASAP
    urb.buffer, urb.buffer_length, urb.number_of_packets = ctypes.addressof(buf), npackets * mps, npackets
    for k in range(npackets):
        urb.iso_frame_desc[k].length = mps
    self._submit(urb, buf, callback)

def _submit(self, urb, buf, callback):
    addr = ctypes.addressof(urb)
    with self.lock:
        self.pending[addr] = (urb, buf, callback)
    try:
        ioctl(self.fd, USBDEVFS_SUBMITURB, urb)
    except UsbfsError:
        with self.lock:
            self.pending.pop(addr, None)
        raise

def resubmit(self, urb, buf, callback):
    if self.running and not self.gone:

```

```

        # reset per-transfer fields before reuse
        urb.status = 0
        urb.actual_length = 0
        urb.error_count = 0
        if urb.type == URB_TYPE_ISO:
            for k in range(urb.number_of_packets):
                urb.iso_frame_desc[k].actual_length = 0
                urb.iso_frame_desc[k].status = 0
        self._submit(urb, buf, callback)

def discard_all(self, callbacks=None):
    """Cancel pending URBs (all, or only those with the given callbacks)."""
    with self.lock:
        items = [(a, u) for a, (u, _b, cb) in self.pending.items() if callbacks is None or cb in callbacks]
        for _a, u in items:
            try:
                ioctl(self.fd, USBDEVFS_DISCARDURB, u)
            except UsbfsError:
                pass # already completed: the reaper will collect it

def outstanding(self, callbacks=None):
    with self.lock:
        return sum(1 for (_u, _b, cb) in self.pending.values() if callbacks is None or cb in callbacks)

def _reap_loop(self):
    p = select.poll()
    p.register(self.fd, select.POLLOUT | select.POLLERR | select.POLLHUP)
    ptr = c_void_p()
    while self.running or self.outstanding():
        try:
            ev = p.poll(100)
        except InterruptedError:
            continue
        if not ev:
            if not self.running and self.gone:
                break
            continue
        while True:
            try:
                ioctl(self.fd, USBDEVFS_REAPURBNDELAY, ptr)
            except UsbfsError as e:
                if e.errno == errno.EAGAIN:
                    break
                if e.errno == errno.ENODEV:
                    self._mark_gone()
                    with self.lock:
                        self.pending.clear()
                    return
            break
        with self.lock:
            entry = self.pending.pop(ptr.value, None)
        if entry:
            urb, buf, cb = entry
            try:
                cb(urb, buf)
            except Exception:
                pass
        if any(e & (select.POLLERR | select.POLLHUP) for _fd, e in ev):
            self._mark_gone()
            with self.lock:
                self.pending.clear()
        return

def _mark_gone(self):
    if not self.gone:
        self.gone = True
        cb = self.on_gone
        if cb:
            cb()

def close(self):
    self.running = False
    self.discard_all()
    if self.reaper is not threading.current_thread():
        self.reaper.join(timeout=2.0)
    try:
        os.close(self.fd)
    except OSError:
        pass

class UsbfsQmx:
    """The real QMX through Linux usbfs directly (no libusb). Same interface
    and behaviour as QmxUsb; used where libusb-1.0 isn't installed (DSM)."""

```

```

def __init__(self, on_iq, on_cat, on_gone, vid=QMX_VID, pid=QMX_PID):
    self.on_iq, self.on_cat, self.on_gone = on_iq, on_cat, on_gone
    self.vid, self.pid = vid, pid
    self.dev = None
    self.running = False
    self.gone_reported = False
    self.claimed: list[int] = []
    self.cdc_ports = []
    self.extra: dict[int, dict] = {}
    self.as_if = None

def open(self):
    found = usbfs_find_device(self.vid, self.pid)
    if not found:
        raise RadioGone("QMX (%04x:%04x) not found on the NAS USB bus" % (self.vid, self.pid))
    name, path = found
    try:
        self.dev = UsbfsDevice(path)
    except PermissionError:
        raise RadioGone("no permission for %s - run the relay as root" % path)
    except OSError as e:
        raise RadioGone("cannot open %s: %s" % (path, e))
    self.dev.on_gone = self._gone
    cfg = usbfs_parse_config(self.dev.read_descriptors())
    self.cdc_ports, uac2, aud = discover_qmx(cfg)
    if not self.cdc_ports or aud is None:
        self.close()
        raise RadioGone("QMX descriptors not recognised (CAT=%s audio=%s)" % (self.cdc_ports, aud))
    cdc_ctrl, cdc_data, self.bulk_in, self.bulk_out = self.cdc_ports[0]
    as_if, alt, self.iso_ep, self.iso_mps, ch, self.subframe, bits, rates = aud
    log.info("QMX found via usbfs %s (%s): CAT if%d (IN 0x%02x OUT 0x%02x), I/Q if%d alt%d EP 0x%02x "
            "mps=%d %dch %d-bit/%dB %s%s", path, name, cdc_data, self.bulk_in, self.bulk_out, as_if, alt,
            self.iso_ep, self.iso_mps, ch, bits, self.subframe, rates or "", " UAC2" if uac2 else "")
    n = len(self.cdc_ports)
    log.info("QMX USB serial ports: %d (%s)", n, ", ".join(
        "port %d = if%s/if%d" % (i + 1, c, d) for i, (c, d, _, _) in enumerate(self.cdc_ports)))
    try:
        for i in [x for x in (cdc_ctrl, cdc_data, as_if) if x is not None]:
            self.dev.claim(i)
            self.claimed.append(i)
    except UsbfsError as e:
        self.close()
        if e.errno == errno.EBUSY:
            raise RadioGone("QMX is in use by another program on the NAS - stop it, then reconnect")
        raise RadioGone("claiming the QMX failed: %s" % e)
    if cdc_ctrl is not None:
        try:
            self.dev.control_write(0x21, 0x20, 0, cdc_ctrl, struct.pack("<IBBB", CAT_BAUD, 0, 0, 8))
            self.dev.control_write(0x21, 0x22, 0x0003, cdc_ctrl, b"")
        except UsbfsError as e:
            log.warning("CDC line setup: %s (continuing)", e)
    self.dev.set_altsetting(as_if, alt)
    self.as_if = as_if
    if not uac2:
        try:
            self.dev.control_write(0x22, 0x01, 0x0100, self.iso_ep, SAMPLE_RATE.to_bytes(3, "little"))
        except UsbfsError as e:
            log.info("sample-rate SET_CUR not accepted (%s) - device default used", e)
    self.running = True
    self.gone_reported = False
    for _ in range(ISO_TRANSFERS):
        self.dev.submit_iso_in(self.iso_ep, ISO_PACKETS, self.iso_mps, self._iso_cb)
    for _ in range(2):
        self.dev.submit_bulk_in(self.bulk_in, 64, self._bulk_cb)

# -- completions (reaper thread) -----
def _gone(self):
    if not self.gone_reported:
        self.gone_reported = True
        log.warning("QMX unplugged / powered off")
        self.running = False
        self.on_gone()

def _status_ok(self, urb):
    st = urb.status
    if st in (0, -errno.EXDEV, -errno.EREMOTEIO): # EXDEV: some iso packets missed; still usable
        return True
    if st in (-errno.ENODEV, -errno.ESHUTDOWN, -errno.EPROTO) and self.running:
        self._gone() # only while running: closing kills URBs too
        return False # ENOENT / ECONNRESET = discarded

def _resubmit(self, urb, buf, cb):
    if not self.running:
        return

```

```

try:
    self.dev.resubmit(urb, buf, cb)
except UsbfsError as e:
    log.warning("USB resubmit failed: %s", e)
    self._gone()

def _iso_cb(self, urb, buf):
    if not self._status_ok(urb):
        return
    raw = bytearray()
    mps = self.iso_mps
    for k in range(urb.number_of_packets):
        d = urb.iso_frame_desc[k]
        if d.status == 0 and d.actual_length:
            raw += buf.raw[k * mps: k * mps + d.actual_length]
    if raw:
        self.on_iq(to_s16(raw, self.subframe))
    self._resubmit(urb, buf, self._iso_cb)

def _bulk_cb(self, urb, buf):
    if not self._status_ok(urb):
        return
    if urb.actual_length:
        self.on_cat(buf.raw[:urb.actual_length])
    self._resubmit(urb, buf, self._bulk_cb)

# -- extra serial ports -----
def serial_count(self) -> int:
    return len(self.cdc_ports)

def serial_open(self, idx, on_data):
    name = "port %d" % (idx + 1)
    st = self.extra.get(idx)
    if st and st["cb"]:
        st["cb"] = on_data
        return True, ""
    if len(self.cdc_ports) <= idx:
        return False, ("the radio has %d USB serial port%s - set 'USB serial ports' to %d"
                       % (len(self.cdc_ports), "" if len(self.cdc_ports) == 1 else "s", idx + 1))
    comm, data, bin_, bout = self.cdc_ports[idx]
    st = {"ifs": [], "out": bout, "cb": None, "rx": None}
    self.extra[idx] = st
    try:
        for i in [x for x in (comm, data) if x is not None]:
            self.dev.claim(i)
            st["ifs"].append(i)
    except UsbfsError as e:
        self._serial_release(idx)
        return False, ("%s is in use by another program on the NAS" % name if e.errno == errno.EBUSY
                       else "%s claim failed: %s" % (name, e))
    if comm is not None:
        try:
            self.dev.control_write(0x21, 0x20, 0, comm, struct.pack("<IBBB", CAT_BAUD, 0, 0, 8))
            self.dev.control_write(0x21, 0x22, 0x0003, comm, b"")
        except UsbfsError as e:
            log.warning("%s line setup: %s (continuing)", name, e)
    st["cb"] = on_data

    def rx(urb, buf, idx=idx):
        s = self.extra.get(idx)
        if not self._status_ok(urb) or not s or not s["cb"]:
            return
        if urb.actual_length:
            s["cb"](buf.raw[:urb.actual_length])
        self._resubmit(urb, buf, s["rx"])
    st["rx"] = rx
    for _ in range(2):
        self.dev.submit_bulk_in(bin_, 512, rx)
    log.info("QMX %s open (if%s/if%d)", name, comm, data)
    return True, ""

def serial_send(self, idx, data: bytes):
    st = self.extra.get(idx)
    if not self.dev or not st or not st["cb"]:
        return
    try:
        self.dev.bulk_write(st["out"], data)
    except UsbfsError as e:
        if e.errno == errno.ENODEV:
            self._gone()
            raise RadioGone("QMX gone")
        log.warning("port %d write: %s", idx + 1, e)

def serial_close(self, idx):

```

```

    st = self.extra.get(idx)
    if not st:
        return
    st["cb"] = None
    if st["rx"] and self.dev:
        self.dev.discard_all({st["rx"]})
        deadline = time.time() + 1.0
        while time.time() < deadline and self.dev.outstanding({st["rx"]}):
            time.sleep(0.02)
    self._serial_release(idx)
    log.info("QMX port %d closed", idx + 1)

def _serial_release(self, idx):
    st = self.extra.pop(idx, None)
    if st and self.dev:
        for i in st["ifs"]:
            self.dev.release(i)

def term_open(self, on_data):
    return self.serial_open(1, on_data)

def term_send(self, data: bytes):
    self.serial_send(1, data)

def term_close(self):
    self.serial_close(1)

def send_cat(self, data: bytes):
    if not self.dev:
        raise RadioGone("closed")
    try:
        self.dev.bulk_write(self.bulk_out, data)
    except UsbfsError as e:
        if e.errno == errno.ENODEV:
            self._gone()
            raise RadioGone("QMX gone")
        log.warning("CAT write: %s", e)

def close(self):
    self.running = False
    dev, self.dev = self.dev, None
    if not dev:
        return
    for st in self.extra.values():
        st["cb"] = None
        self.claimed += st["ifs"]
    self.extra = {}
    dev.on_gone = None # an orderly close is not an unplug
    dev.discard_all()
    deadline = time.time() + 1.0
    while time.time() < deadline and dev.outstanding():
        time.sleep(0.02)
    if self.as_if is not None and not dev.gone:
        try:
            dev.set_altsetting(self.as_if, 0) # stop the isochronous stream
        except UsbfsError:
            pass
    dev.close() # waits for the reaper, closes the fd (releases interfaces)
    self.claimed = []

def libusb_available() -> bool:
    try:
        from usb1 import _libusb1
        path = os.environ.get("QMX_LIBUSB")
        _libusb1.loadLibrary(path) if path else _libusb1.loadLibrary()
        return True
    except Exception:
        return False

def pick_backend():
    """libusb if it loads (any OS), else Linux usbfs. QMX_BACKEND=libusb|usbfs forces one."""
    want = os.environ.get("QMX_BACKEND", "").lower()
    if want == "usbfs" or (want != "libusb" and not libusb_available()):
        if not sys.platform.startswith("linux"):
            raise RadioGone("libusb-1.0 not found (and usbfs is Linux-only)")
        return UsbfsQmx
    return QmxUsb

class FakeRadio:
    """Bench-test stand-in: a +3 kHz tone and a tiny Kenwood CAT emulator."""
    def __init__(self, on_iq, on_cat, on_gone):

```

```

self.on_iq, self.on_cat, self.on_gone = on_iq, on_cat, on_gone
self.running = False
self.freq, self.mode = 14074000, "2"
self.bufs = {0: b"", 2: b""}
self.cbs = {} # idx -> data callback for extra ports
self.sel = 0

def open(self):
    self.running = True
    threading.Thread(target=self._gen, daemon=True).start()
    log.info("FAKE radio: synthetic I/Q tone at +3000 Hz, CAT emulator")

def _gen(self):
    ph, step = 0.0, 2 * math.pi * 3000 / SAMPLE_RATE
    nxt = time.monotonic()
    while self.running:
        out = bytearray()
        for _ in range(FRAME_PAIRS):
            out += struct.pack("<hh", int(8000 * math.cos(ph)), int(8000 * math.sin(ph)))
            ph = (ph + step) % (2 * math.pi)
        self.on_iq(bytes(out))
        nxt += FRAME_PAIRS / SAMPLE_RATE
        time.sleep(max(0.0, nxt - time.monotonic()))

def serial_count(self) -> int:
    n = getattr(self.args_ns, "fake_ports", 3)
    return 1 if getattr(self.args_ns, "fake_one_port", False) else n

def send_cat(self, data: bytes, idx: int = 0, reply=None):
    reply = reply or self.on_cat
    self.bufs[idx] += data
    while b";" in self.bufs[idx]:
        cmd, self.bufs[idx] = self.bufs[idx].split(b";", 1)
        c = cmd.decode(errors="replace")
        r = None
        if c == "ID":
            r = "ID020;"
        elif c == "FA":
            r = "FA%01ld;" % self.freq
        elif c.startswith("FA") and c[2:].isdigit():
            self.freq = int(c[2:])
        elif c == "MD":
            r = "MD%s;" % self.mode
        elif c.startswith("MD") and len(c) == 3:
            self.mode = c[2]
        elif c == "FW":
            r = "FW2700;"
        elif c == "VN":
            r = "VN1_03_0020QMX;"
        elif c in ("Q9", "Q9 1"):
            r = "Q91;"
        elif c.startswith("MM"):
            r = "MM;"
        if r:
            reply(r.encode())

# Fake extra ports: port 2 answers CR with a tiny ANSI menu (arrows move);
# port 3 is a second, independent CAT port.
def serial_open(self, idx, on_data):
    n = self.serial_count()
    if n <= idx:
        return False, ("the radio has %d USB serial port%s - set 'USB serial ports' to %d"
                        % (n, "" if n == 1 else "s", idx + 1))
    self.cbs[idx] = on_data
    if idx == 1:
        self.sel = 0
    return True, ""

def _paint(self):
    items = ["Band config.", "Mode config.", "System config", "Exit terminal"]
    out = "\x1b[2J\x1b[H QMX menu\r\n"
    for i, it in enumerate(items):
        out += ("\x1b[7m %s \x1b[0m" if i == self.sel else " %s ") % it + "\r\n"
    self.cbs[1](out.encode())

def serial_send(self, idx, data: bytes):
    cb = self.cbs.get(idx)
    if not cb:
        return
    if idx == 2:
        self.send_cat(data, idx=2, reply=cb)
    elif data in (b"\r", b"\x1b[B", b"\x1b[A"):
        if data == b"\x1b[B":
            self.sel = (self.sel + 1) % 4

```

```

        elif data == b"\x1b[A":
            self.sel = (self.sel - 1) % 4
            self._paint()

    def serial_close(self, idx):
        self.cbs.pop(idx, None)

    def term_open(self, on_data):
        return self.serial_open(1, on_data)

    def term_send(self, data: bytes):
        self.serial_send(1, data)

    def term_close(self):
        self.serial_close(1)

    def close(self):
        self.running = False
        self.cbs = {}

# -----
# Shared radio: opened by the first user (Tab5 session or port-3 client),
# released by the last.
# -----

class RadioManager:
    def __init__(self, args):
        self.args = args
        self.lock = threading.Lock()
        self.radio = None
        self.dead = False
        self.users: set = set()
        self.session = None          # the current Tab5 session gets I/Q and CAT

    def acquire(self, user):
        with self.lock:
            if self.radio is not None and self.dead:
                self._close_locked()          # a previous QMX connection died; start fresh
            if self.radio is None:
                cls = FakeRadio if self.args.fake else pick_backend()
                r = cls(self._on_iq, self._on_cat, self._on_gone)
                r.args_ns = self.args
                r.open()                      # raises RadioGone
                self.radio, self.dead = r, False
            self.users.add(user)
            return self.radio

    def release(self, user, radio):
        with self.lock:
            self.users.discard(user)
            if self.session is user:
                self.session = None
            if radio is self.radio and not self.users:
                self._close_locked()

    def _close_locked(self):
        r, self.radio = self.radio, None
        if r:
            r.close()
            log.info("QMX released")

    # radio callbacks (libusb / fake thread)
    def _on_iq(self, b):
        s = self.session
        if s:
            s._on_iq(b)

    def _on_cat(self, b):
        s = self.session
        if s:
            s._on_cat(b)

    def _on_gone(self):
        self.dead = True
        for u in list(self.users):
            u.radio_gone()

# -----
# One Tab5 session
# -----

class Session:

```

```

def __init__(self, sock: socket.socket, peer, args, mgr: RadioManager):
    self.sock, self.peer, self.args, self.mgr = sock, peer, args, mgr
    # Two queues: control frames (CAT, terminal, status, ping) always go
    # out before queued I/Q, so a slow WiFi link delays the spectrum, not
    # the radio's replies. The I/Q queue is short and drops when full.
    self.ctl: collections.deque[bytes] = collections.deque()
    self.iq: collections.deque[bytes] = collections.deque()
    self.wake = threading.Condition()
    self.iq_acc = bytearray()
    self.alive = True
    self.dropped = 0
    self.iq_bytes = 0

def _put_ctl(self, data: bytes):
    with self.wake:
        self.ctl.append(data)
        self.wake.notify()

# radio callbacks (libusb thread)
def _on_iq(self, s16: bytes):
    self.iq_acc += s16
    fb = FRAME_PAIRS * 4
    while len(self.iq_acc) >= fb:
        chunk = bytes(self.iq_acc[:fb])
        del self.iq_acc[:fb]
        with self.wake:
            if len(self.iq) >= IQ_QUEUE_MAX:
                self.dropped += 1 # Tab5/WiFi can't keep up: drop I/Q, keep latency low
                continue
            self.iq_bytes += len(chunk)
            self.iq.append(frame(PROTO_IQ, chunk))
            self.wake.notify()

def _on_cat(self, data: bytes):
    log.debug("QMX -> Tab5 CAT %r", data)
    self._put_ctl(frame(PROTO_CAT, data))

def _on_term(self, data: bytes):
    self._put_ctl(frame(PROTO_TERM_DATA, data))

def radio_gone(self):
    self.stop("QMX disconnected")

def stop(self, why: str):
    if self.alive:
        self.alive = False
        with self.wake:
            self.wake.notify()
        log.info("Tab5 session %s:%d ending: %s", self.peer[0], self.peer[1], why)
        try:
            self.sock.shutdown(socket.SHUT_RDWR)
        except OSError:
            pass

def run(self):
    sock = self.sock
    sock.setsockopt(socket.IPPROTO_TCP, socket.TCP_NODELAY, 1)
    sock.setsockopt(socket.SOL_SOCKET, socket.SO_KEEPALIVE, 1)
    # Small kernel send buffer: otherwise Linux autotunes it to megabytes,
    # seconds of I/Q pile up there instead of being dropped above, and every
    # CAT reply waits behind them (the Tab5 then times out IQ-mode setup).
    try:
        sock.setsockopt(socket.SOL_SOCKET, socket.SO_SNDBUF, TAB5_SNDBUF)
    except OSError:
        pass
    radio = None
    try:
        sock.settimeout(5)
        t, _, n = HDR.unpack(recv_exact(sock, 4))
        hello = recv_exact(sock, n) if n else b""
        if t != PROTO_HELLO or not hello.startswith(b"QMXR/1"):
            raise ConnectionError("not a QMXR/1 client")
        log.info("Tab5 %s:%d hello: %s", self.peer[0], self.peer[1], hello.decode(errors="replace"))

        radio = self.mgr.acquire(self) # raises RadioGone -> we close without answering
        self.mgr.session = self

        sock.sendall(frame(PROTO_HELLO, b"QMXR/1 rate=%d ch=2 fmt=s16le src=nas term=1" % SAMPLE_RATE))
        sock.settimeout(None)
        sender = threading.Thread(target=self._send_loop, daemon=True)
        sender.start()
        while self.alive:
            t, _, n = HDR.unpack(recv_exact(sock, 4))
            payload = recv_exact(sock, n) if n else b""

```

```

        if t == PROTO_CAT and payload:
            log.debug("Tab5 -> QMX CAT %r", payload)
            radio.send_cat(payload)
        elif t == PROTO_TERM_DATA and payload:
            radio.term_send(payload)
        elif t == PROTO_TERM_OPEN:
            ok, why = radio.term_open(self._on_term)
            if not ok:
                log.warning("terminal: %s", why)
                self._put_ctl(frame(PROTO_TERM_STATUS, (b"\x01" if ok else b"\x00" + why.encode())[:200]))
            elif t == PROTO_TERM_CLOSE:
                radio.term_close()
    except RadioGone as e:
        log.warning("%s", e)
    except (ConnectionError, OSError, socket.timeout) as e:
        if self.alive:
            log.info("Tab5 %s:%d disconnected: %s", self.peer[0], self.peer[1], e)
    finally:
        self.alive = False
        if radio:
            try:
                radio.term_close()
            except Exception:
                pass
            if self.dropped:
                log.info("%d I/Q frames were dropped for a slow link", self.dropped)
            self.mgr.release(self, radio)
        try:
            sock.close()
        except OSError:
            pass

def _send_loop(self):
    last_ping = time.monotonic()
    while self.alive:
        with self.wake:
            if not self.ctl and not self.iq:
                self.wake.wait(0.25)
            # all pending control frames first, then at most one I/Q frame,
            # so a CAT reply never waits behind more than one I/Q frame here
            out = list(self.ctl)
            self.ctl.clear()
            if self.iq:
                out.append(self.iq.popleft())
            if time.monotonic() - last_ping >= 1.0:
                out.insert(0, frame(PROTO_PING))
                last_ping = time.monotonic()
            try:
                for data in out:
                    self.sock.sendall(data)
            except OSError as e:
                self.stop("send failed: %s" % e)
                return

# -----
# QMX port 3 as raw TCP serial for PC software
# -----

PORT3 = 2 # index into the QMX's serial ports (0 = CAT, 1 = terminal, 2 = port 3)

class SerialClient:
    """One PC on the QMX's third serial port. Bytes are passed through
    unchanged in both directions - no framing, like a serial cable."""

    def __init__(self, sock: socket.socket, peer, mgr: RadioManager):
        self.sock, self.peer, self.mgr = sock, peer, mgr
        self.q: queue.Queue[bytes] = queue.Queue()
        self.alive = True

    def radio_gone(self):
        self.stop("QMX disconnected")

    def stop(self, why: str):
        if self.alive:
            self.alive = False
            log.info("port 3 client %s:%d ending: %s", self.peer[0], self.peer[1], why)
            try:
                self.sock.shutdown(socket.SHUT_RDWR)
            except OSError:
                pass

    def _on_data(self, data: bytes):

```

```

        self.q.put(data)                # libusb thread: never block on the PC's socket

    def _send_loop(self):
        while self.alive:
            try:
                data = self.q.get(timeout=0.25)
            except queue.Empty:
                continue
            try:
                self.sock.sendall(data)
            except OSError as e:
                self.stop("send failed: %s" % e)
                return

    def run(self):
        sock = self.sock
        sock.setsockopt(socket.IPPROTO_TCP, socket.TCP_NODELAY, 1)
        sock.setsockopt(socket.SOL_SOCKET, socket.SO_KEEPALIVE, 1)
        radio = opened = None
        try:
            radio = self.mgr.acquire(self)
            ok, why = radio.serial_open(PORT3, self._on_data)
            if not ok:
                log.warning("port 3 client %s refused: %s", self.peer[0], why)
                return
            opened = True
            log.info("port 3 client %s:%d connected", self.peer[0], self.peer[1])
            threading.Thread(target=self._send_loop, daemon=True).start()
            while self.alive:
                data = sock.recv(4096)
                if not data:
                    log.info("port 3 client %s:%d disconnected", self.peer[0], self.peer[1])
                    break
                radio.serial_send(PORT3, data)
            except RadioGone as e:
                log.warning("port 3: %s", e)
            except OSError as e:
                if self.alive:
                    log.info("port 3 client %s:%d disconnected: %s", self.peer[0], self.peer[1], e)
            finally:
                self.alive = False
                if radio:
                    if opened:
                        try:
                            radio.serial_close(PORT3)
                        except Exception:
                            pass
                    self.mgr.release(self, radio)
                try:
                    sock.close()
                except OSError:
                    pass

    def serve(listen, port, make, label):
        """Accept loop: one client at a time; a new connection replaces the old one."""
        srv = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        srv.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
        srv.bind((listen, port))
        srv.listen(2)
        log.info("listening on %s:%d (%s)", listen, port, label)
        current: list = [None, None] # [client, thread]
        while True:
            sock, peer = srv.accept()
            old, th = current
            if old and old.alive:
                old.stop("replaced by new connection from %s" % peer[0])
                th.join(timeout=3) # let it release its port before the new one opens it
            c = make(sock, peer)
            t = threading.Thread(target=c.run, daemon=True)
            current[:] = [c, t]
            t.start()

# -----
# -----
# Self-test against the real radio (read-only: never keys the transmitter)
# -----

class _SelfTestUser:
    def __init__(self):
        self.pairs = 0
        self.sumsq = 0

```

```

        self.nsq = 0
        self.peak = 0
        self.cat = bytearray()
        self.gone = False

    def radio_gone(self):
        self.gone = True

    def _on_iq(self, b: bytes):
        from array import array
        a = array("h")
        a.frombytes(b[: len(b) // 2 * 2])
        self.pairs += len(a) // 2
        sub = a[::8]
        self.sumsq += sum(x * x for x in sub)
        self.nsq += len(sub)
        m = max((abs(x) for x in sub), default=0)
        if m > self.peak:
            self.peak = m

    def _on_cat(self, b: bytes):
        self.cat += b

def selftest(mgr) -> bool:
    """Open the QMX, measure the I/Q stream, ask read-only CAT questions on
    port 1, check ports 2 and 3, then release it. Results go to the log."""
    log.info("SELFTEST: starting (read-only - nothing is transmitted)")
    u = _SelfTestUser()
    try:
        radio = mgr.acquire(u)
    except RadioGone as e:
        log.error("SELFTEST FAIL: %s", e)
        return False
    ok_all = True
    mgr.session = u
    try:
        # 1. I/Q stream
        t0 = time.monotonic()
        time.sleep(2.0)
        dt = time.monotonic() - t0
        rate = u.pairs / dt if dt else 0
        rms = math.sqrt(u.sumsq / u.nsq) if u.nsq else 0.0
        dbfs = 20 * math.log10(rms / 32768) if rms > 0 else float("-inf")
        iq_ok = 40000 <= rate <= 56000
        ok_all &= iq_ok
        log.info("SELFTEST %s: I/Q %.0f pairs/s (expect ~48000), level %.1f dBFS rms, peak %d",
            "PASS" if iq_ok else "FAIL", rate, dbfs, u.peak)
        if iq_ok and u.peak == 0:
            log.warning("SELFTEST note: I/Q is all zeros - is the radio receiving? (IQ mode is set by the Tab5)")

        # 2. CAT on port 1 - queries only
        for q in (b"ID;", b"FA;", b"MD;", b"VN;"):
            u.cat.clear()
            radio.send_cat(q)
            deadline = time.monotonic() + 1.0
            while time.monotonic() < deadline and b";" not in u.cat:
                time.sleep(0.02)
            time.sleep(0.05)
            reply = bytes(u.cat).decode(errors="replace").strip()
            good = bool(reply) and reply.endswith(";")
            ok_all &= good if q == b"ID;" else True
            log.info("SELFTEST %s: port 1 CAT %s -> %s", "PASS" if good else "FAIL",
                q.decode(), reply or "(no reply)")

        # 3. serial ports 2 and 3
        n = radio.serial_count()
        log.info("SELFTEST: radio reports %d USB serial port(s)", n)
        if n >= 2:
            ok, why = radio.serial_open(1, lambda b: None) # open/close only: no bytes sent,
            if ok: # so the radio never enters terminal mode
                radio.serial_close(1)
            ok_all &= ok
            log.info("SELFTEST %s: port 2 (Radio menus) %s", "PASS" if ok else "FAIL", "opens" if ok else why)
        else:
            log.info("SELFTEST skip: port 2 - set 'USB serial ports' to 2 or 3 for Radio menus")
        if n >= 3:
            got = bytearray()
            ok, why = radio.serial_open(2, lambda b: got.extend(b))
            reply = ""
            if ok:
                radio.serial_send(2, b"FA;")
                deadline = time.monotonic() + 1.0
                while time.monotonic() < deadline and b";" not in got:

```

```

        time.sleep(0.02)
        radio.serial_close(2)
        reply = bytes(got).decode(errors="replace").strip()
        ok = bool(reply)
        ok_all &= ok
        log.info("SELFTTEST %s: port 3 (PC serial) FA; -> %s", "PASS" if ok else "FAIL",
                reply or why or "(no reply)")
    else:
        log.info("SELFTTEST skip: port 3 - set 'USB serial ports' to 3 for PC software on TCP 7356")
    if u.gone:
        ok_all = False
        log.error("SELFTTEST FAIL: the QMX disconnected during the test")
except RadioGone as e:
    ok_all = False
    log.error("SELFTTEST FAIL: %s", e)
finally:
    if mgr.session is u:
        mgr.session = None
    mgr.release(u, radio)
log.info("SELFTTEST %s", "PASSED" if ok_all else "FAILED - see the lines above")
return ok_all

def probe_usbfs():
    print("(usbfs backend - libusb-1.0 not available)")
    found = False
    for bus, devn, vid, pid, name in usbfs_list_devices():
        mark = " <== QMX" if (vid, pid) == (QMX_VID, QMX_PID) else ""
        print("%03d/%03d %04x:%04x %s%s" % (bus, devn, vid, pid, name, mark))
        if mark:
            found = True
            with open("/dev/bus/usb/%03d/%03d" % (bus, devn), "rb") as f:
                cfg = usbfs_parse_config(f.read())
            for iface in cfg:
                for st in iface:
                    eps = ", ".join("0x%02x/%s/%d" % (e.getAddress(), "CIBI"[e.getAttributes() & 3],
                                                    e.getMaxPacketSize()) for e in st)
                    print("    if%d alt%d class %02x/%02x/%02x eps: %s" % (
                        st.getNumber(), st.getAlternateSetting(), st.getClass(), st.getSubClass(),
                        st.getProtocol(), eps or "-"))
                ports, _uac2, aud = discover_qmx(cfg)
                print("    USB serial ports: %d (Radio menus need 2+, PC serial on port 3 needs 3)" % len(ports))
                print("    I/Q input: %s" % (aud,))
    if not found:
        print("QMX not found. Is it plugged into the NAS and powered on?")
    return 0 if found else 1

def probe(args):
    if os.environ.get("QMX_BACKEND", "").lower() == "usbfs" or not libusb_available():
        return probe_usbfs()
    import usb1
    _load_libusb(usb1)
    with usb1.USBContext() as ctx:
        found = False
        for dev in ctx.getDeviceIterator(skip_on_error=True):
            vid, pid = dev.getVendorID(), dev.getProductID()
            mark = " <== QMX" if (vid, pid) == (QMX_VID, QMX_PID) else ""
            print("%03d/%03d %04x:%04x%s" % (dev.getBusNumber(), dev.getDeviceAddress(), vid, pid, mark))
            if mark:
                found = True
                for iface in dev[0]:
                    for s in iface:
                        eps = ", ".join("0x%02x/%s/%d" % (e.getAddress(), "CIBI"[e.getAttributes() & 3],
                                                            e.getMaxPacketSize()) for e in s)
                        print("    if%d alt%d class %02x/%02x/%02x eps: %s" % (
                            s.getNumber(), s.getAlternateSetting(), s.getClass(), s.getSubClass(),
                            s.getProtocol(), eps or "-"))
                    ncdc = sum(1 for iface in dev[0] for s in iface
                               if s.getClass() == 0x0A and s.getAlternateSetting() == 0)
                    print("    USB serial ports: %d (Radio menus need 2+, PC serial on port 3 needs 3)" % ncdc)
    if not found:
        print("QMX not found. Is it plugged into the NAS and powered on?")
    return 0 if found else 1

def main():
    ap = argparse.ArgumentParser(description=__doc__.split("\n\n")[0],
                                formatter_class=argparse.RawDescriptionHelpFormatter)
    ap.add_argument("--listen", default="0.0.0.0", help="bind address (default all)")
    ap.add_argument("--port", type=int, default=DEFAULT_PORT, help="Tab5 TCP port (default 7355)")
    ap.add_argument("--serial-port", type=int, default=DEFAULT_SERIAL_PORT,
                    help="TCP port for the QMX's 3rd serial port, for PC software "
                          "(default 7356, 0 = off; needs 'USB serial ports' = 3 on the radio)")

```

```

ap.add_argument("--fake", action="store_true", help="synthetic radio for bench tests (no QMX needed)")
ap.add_argument("--fake-one-port", action="store_true", help=argparse.SUPPRESS)
ap.add_argument("--fake-ports", type=int, default=3, choices=(1, 2, 3), help=argparse.SUPPRESS)
ap.add_argument("--probe", action="store_true", help="list USB devices / QMX interfaces and exit")
ap.add_argument("--selftest", action="store_true",
                help="at startup, test the QMX (I/Q, read-only CAT, ports 2/3), log the result, then serve")
ap.add_argument("--selftest-only", action="store_true", help="run the self-test and exit (0 = passed)")
ap.add_argument("-v", "--verbose", action="store_true")
args = ap.parse_args()
logging.basicConfig(level=logging.DEBUG if args.verbose else logging.INFO,
                    format="%(asctime)s %(levelname)s %(message)s")

if args.probe:
    return probe(args)

mgr = RadioManager(args)
if args.selftest or args.selftest_only:
    passed = selftest(mgr)
    if args.selftest_only:
        return 0 if passed else 1
if args.fake:
    log.info("FAKE radio with %d USB serial port(s)", 1 if args.fake_one_port else args.fake_ports)
if args.serial_port:
    threading.Thread(target=serve, daemon=True, args=(
        args.listen, args.serial_port, lambda s, p: SerialClient(s, p, mgr),
        "QMX port 3, raw serial for PC software")).start()
serve(args.listen, args.port, lambda s, p: Session(s, p, args, mgr), "QMXR/1 for the Tab5")

if __name__ == "__main__":
    try:
        sys.exit(main())
    except KeyboardInterrupt:
        pass

```

tools/nas-relay/nettest.py

```

#!/usr/bin/env python3
"""End-to-end network test of a running relay, from the NAS itself.
Acts as the Tab5 on TCP 7355 (QMXR/1) and as a PC on TCP 7356 (port 3).
Read-only: CAT queries only, the terminal port is opened and closed without
sending anything, so the radio never transmits or enters terminal mode."""
import socket, struct, sys, time

HOST = sys.argv[1] if len(sys.argv) > 1 else "127.0.0.1"
HDR = struct.Struct("<BBH")
ok_all = True

def say(ok, msg):
    global ok_all
    ok_all &= ok
    print("NETTEST %s: %s" % ("PASS" if ok else "FAIL", msg), flush=True)

def frame(t, p=b''):
    return HDR.pack(t, 0, len(p)) + p

def rx(s, n):
    b = b''
    while len(b) < n:
        c = s.recv(n - len(b))
        if not c:
            raise ConnectionError("closed by relay")
        b += c
    return b

def read_frames(s, secs, want=None):
    """Collect frames for `secs` seconds (or until `want(type, payload)` is true)."""
    out, end = [], time.monotonic() + secs
    while time.monotonic() < end:
        s.settimeout(max(0.05, end - time.monotonic()))
        try:
            t, _f, n = HDR.unpack(rx(s, 4))
            p = rx(s, n) if n else b''
        except socket.timeout:
            break
        out.append((t, p))
        if want and want(t, p):
            break
    return out
try:

```

```

tab5 = socket.create_connection((HOST, 7355), 5)
tab5.sendall(frame(1, b"QMXR/1 nettest"))
tab5.settimeout(8)
t, _f, n = HDR.unpack(rx(tab5, 4))
hello = rx(tab5, n)
say(t == 1 and hello.startswith(b"QMXR/1"), "Tab5 handshake on 7355: %s" % hello.decode(errors="replace"))

fr = read_frames(tab5, 3.0)
iq = sum(len(p) // 4 for t, p in fr if t == 2)
pings = sum(1 for t, _ in fr if t == 4)
say(40000 * 3 <= iq <= 56000 * 3, "I/Q over the network: %.0f pairs/s, %d pings" % (iq / 3.0, pings))

for q in (b"ID;", b"FA;", b"VN;"):
    tab5.sendall(frame(3, q))
    cat = b""
    for t, p in read_frames(tab5, 2.0, lambda t, p: t == 3 and p.endswith(b";")):
        if t == 3:
            cat += p
    say(cat.endswith(b";"), "Tab5 CAT %s -> %s" % (q.decode(), cat.decode(errors="replace") or "(nothing)"))

tab5.sendall(frame(5)) # TERM_OPEN (Radio menus port) - nothing is sent on it
status = [p for t, p in read_frames(tab5, 3.0, lambda t, p: t == 6) if t == 6]
if status:
    okt = status[0][1:] == b"\x01"
    say(okt, "Radio menus port over the network: %s" % ("opens" if okt else status[0][1:].decode(errors="replace")))
else:
    say(False, "Radio menus port: no TERM_STATUS reply")
tab5.sendall(frame(8)) # TERM_CLOSE
time.sleep(0.3)

pc = socket.create_connection((HOST, 7356), 5)
pc.settimeout(3)
pc.sendall(b"FA;")
reply = b""
end = time.monotonic() + 2
while time.monotonic() < end and not reply.endswith(b";"):
    try:
        reply += pc.recv(256)
    except socket.timeout:
        break
say(reply.endswith(b";"), "PC on port 3 (7356) while the Tab5 is connected: FA; -> %s" % reply.decode(errors="replace"))
pc.close()

tab5.sendall(frame(3, b"FA;"))
cat = b"".join(p for t, p in read_frames(tab5, 2.0, lambda t, p: t == 3 and p.endswith(b";")) if t == 3)
say(cat.endswith(b";"), "Tab5 still answered after the PC left: %s" % cat.decode(errors="replace"))
tab5.close()
except Exception as e:
    say(False, "error: %r" % (e,))

print("NETTEST %s" % ("PASSED" if ok_all else "FAILED"), flush=True)
sys.exit(0 if ok_all else 1)

```

tools/nas-relay/start-relay.sh

```

#!/bin/sh
# Start the QMX relay in the background and keep it running (restarts if it
# exits). DSM: Task Scheduler > Create > Triggered Task > User-defined script,
# user root, event "Boot-up". Log: qmx-relay.log in this folder.
. "$(cd "$(dirname "$0")" && pwd)/env.sh"
LOG="$DIR/qmx-relay.log"
PIDF="$DIR/relay.pid"
[ -n "$PY" ] || { echo "$(date) no python3 found" >>"$LOG"; exit 1; }
# python-libusb1 is optional: without libusb-1.0 the relay uses Linux usbfs.
"$PY" "$DIR/get_libusb1.py" >>"$LOG" 2>&1 || echo "$(date) python-libusb1 not installed - using usbfs" >>"$LOG"
if [ -f "$PIDF" ] && kill -0 "$(cat "$PIDF")" 2>/dev/null; then
    echo "$(date) already running (pid $(cat "$PIDF"))" >>"$LOG"; exit 0
fi
(
    trap 'kill $CHILD 2>/dev/null; exit 0' TERM INT
    while :; do
        if [ -f "$LOG" ] && [ "$(wc -c <"$LOG")" -gt 2000000 ]; then mv -f "$LOG" "$LOG.1"; fi
        "$PY" -u "$DIR/qmx_nas_relay.py" --selftest >>"$LOG" 2>&1 &
        CHILD=$!
        wait $CHILD
        echo "$(date) relay exited (code $?), restarting in 5 s" >>"$LOG"
        sleep 5
    done
) </dev/null >/dev/null 2>&1 &
echo $! >"$PIDF"
echo "$(date) started (supervisor pid $!)" >>"$LOG"

```

tools/nas-relay/stop-relay.sh

```
#!/bin/sh
# Stop the QMX relay started by start-relay.sh.
. "$(cd "$(dirname "$0")" && pwd)/env.sh"
PIDF="$DIR/relay.pid"
[ -f "$PIDF" ] && kill "$(cat "$PIDF")" 2>/dev/null
rm -f "$PIDF"
echo "$(date) stopped" >>"$DIR/qmx-relay.log"
```

tools/nas-relay/env.sh

```
# Shared settings for the QMX relay scripts (sourced, not run).
DIR="$(cd "$(dirname "$0")" && pwd)"
export PYTHONPATH="$DIR/lib"
PY=""
for c in python3 /usr/bin/python3 /bin/python3 /usr/local/bin/python3 \
    /var/packages/Python3.9/target/usr/bin/python3.9 /var/packages/Python3/target/usr/bin/python3; do
    if command -v "$c" >/dev/null 2>&1; then PY="$c"; break; fi
done
# If libusb is not found automatically, set its full path here (or put
# libusb-1.0.so.0 into this folder):
if [ -z "$QMX_LIBUSB" ] && [ -f "$DIR/libusb-1.0.so.0" ]; then export QMX_LIBUSB="$DIR/libusb-1.0.so.0"; fi
```

tools/nas-relay/check.sh

```
#!/bin/sh
# One-off check: environment + QMX probe + read-only self-test (or, when the
# relay is running, a network test against it from the NAS itself).
# DSM: Control Panel > Task Scheduler > Create > Scheduled Task > User-defined
# script, user root, then "Run". Result: check.log in this folder.
. "$(cd "$(dirname "$0")" && pwd)/env.sh"
OUT="$DIR/check.log"
# Restart on request: put a file named "restart.request" in this folder and run
# this task. The relay process is stopped; start-relay.sh's supervisor starts the
# (updated) relay again 5 s later. Avoids re-creating root tasks after an update.
if [ -f "$DIR/restart.request" ]; then
    rm -f "$DIR/restart.request"
    # /proc rather than ps: DSM's "ps w" only lists processes on the current
    # terminal, and a Task Scheduler job has none.
    for d in /proc/[0-9]*; do
        if tr '\0' ' ' < "$d/cmdline" 2>/dev/null | grep -q 'qmx_nas_relay\.py'; then kill "${d#/proc/}" 2>/dev/null; fi
    done
    echo "$(date) restart requested via check.sh" >> "$DIR/qmx-relay.log"
    sleep 12
fi
{
    echo "=== QMX relay check $(date)"
    echo "--- system"; uname -a; cat /etc.defaults/VERSION 2>/dev/null | grep -E 'productversion|buildnumber'
    echo "--- python"; echo "PY=$PY"; [ -n "$PY" ] && "$PY" -V 2>&1
    echo "--- libusb"; ls -l /lib/libusb* /usr/lib/libusb* /lib64/libusb* /usr/lib64/libusb* /usr/local/lib/libusb* 2>/
    dev/null; echo "QMX_LIBUSB=$QMX_LIBUSB"
    echo "--- usb devices"; for d in /sys/bus/usb/devices/*; do [ -f "$d/idVendor" ] && echo "$(basename $d) $(cat $d/id
    Vendor):$(cat $d/idProduct) $(cat $d/product 2>/dev/null)"; done
    echo "--- kernel drivers bound to the QMX"; for i in /sys/bus/usb/devices/*.*; do [ -L "$i/driver" ] && case "$(cat
    $i/..idVendor 2>/dev/null)" in 0483) echo "$(basename $i) -> $(basename $(readlink $i/driver))";; esac; done
    echo "--- relay already running?"; for d in /proc/[0-9]*; do tr '\0' ' ' < "$d/cmdline" 2>/dev/null | grep -q 'qmx_n
    as_relay\.py' && echo "yes, pid ${d#/proc/}"; done
    if [ -n "$PY" ]; then
        echo "--- libusb1 (python); "$PY" "$DIR/get_libusb1.py" 2>&1
        echo "--- probe"; "$PY" -u "$DIR/qmx_nas_relay.py" --probe 2>&1
        if [ -f "$DIR/relay.pid" ] && kill -0 "$(cat "$DIR/relay.pid")" 2>/dev/null; then
            echo "--- relay is running: network test (as Tab5 on 7355 and PC on 7356)"
            "$PY" -u "$DIR/nettest.py" 127.0.0.1 2>&1; echo "nettest exit code: $?"
            echo "--- relay log (last 25 lines); tail -25 "$DIR/qmx-relay.log"
        else
            echo "--- selftest"; "$PY" -u "$DIR/qmx_nas_relay.py" --selftest-only 2>&1; echo "selftest exit code: $?"
        fi
    else
        echo "NO PYTHON 3 FOUND"
    fi
    echo "=== done $(date)"
} > "$OUT" 2>&1
```

tools/nas-relay/get_libusb1.py

```
#!/usr/bin/env python3
"""Fetch python-libusb1 3.4.0 (pure Python) from PyPI into ./lib if it isn't
there yet. The download is checked against its published SHA-256, so the
result is the exact file PyPI lists, even if TLS verification has to fall
back on a NAS without a CA bundle."""
import hashlib, io, os, ssl, sys, urllib.request, zipfile
```

```

URL = ("https://files.pythonhosted.org/packages/d4/64/d4b59444e4d3b6979aa5eb58840634465a24b41a9ab03dcf8434c9b89551/"
      "libusb1-3.4.0-py3-none-any.whl")
SHA256 = "e83d034e44c3efe1c4599c6281d34bca50a38c12cab3b7b6217d583161a01ffd"
LIB = os.path.join(os.path.dirname(os.path.abspath(__file__)), "lib")

if os.path.isfile(os.path.join(LIB, "usb1", "__init__.py")):
    print("libusb1: already present in", LIB)
    sys.exit(0)
try:
    data = urllib.request.urlopen(URL, timeout=60).read()
except ssl.SSLError as e:
    print("libusb1: TLS verification failed (%s); retrying unverified - the SHA-256 check still applies" % e)
    data = urllib.request.urlopen(URL, timeout=60, context=ssl._create_unverified_context()).read()
except Exception as e:
    if "CERTIFICATE_VERIFY_FAILED" not in str(e):
        raise
    print("libusb1: TLS verification failed; retrying unverified - the SHA-256 check still applies")
    data = urllib.request.urlopen(URL, timeout=60, context=ssl._create_unverified_context()).read()
digest = hashlib.sha256(data).hexdigest()
if digest != SHA256:
    sys.exit("libusb1: SHA-256 mismatch (%s) - not installed" % digest)
with zipfile.ZipFile(io.BytesIO(data)) as z:
    for n in z.namelist():
        if n.startswith("usb1/") or n == "libusb1.py" or "/licenses/" in n:
            target = os.path.join(LIB, "usb1", os.path.basename(n)) if "/licenses/" in n else os.path.join(LIB, n)
            os.makedirs(os.path.dirname(target), exist_ok=True)
            with open(target, "wb") as f:
                f.write(z.read(n))
print("libusb1: installed 3.4.0 into", LIB)

```

tools/nas-relay/Dockerfile

```

# QMX network relay for a Synology NAS (Container Manager / Docker).
# Talks to the QMX with libusb in userspace - no DSM kernel drivers needed.
FROM python:3.12-alpine
RUN apk add --no-cache libusb \
    && pip install --no-cache-dir libusb1
COPY qmx_nas_relay.py /app/qmx_nas_relay.py
EXPOSE 7355
ENTRYPOINT ["python", "-u", "/app/qmx_nas_relay.py"]

```

tools/nas-relay/docker-compose.yml

```

# Synology: Container Manager > Project > Create, path = the folder holding
# this file (e.g. /volume1/docker/qmx-relay). Or over SSH: sudo docker compose up -d --build
services:
  qmx-relay:
    build: .
    image: qmx-nas-relay:latest
    container_name: qmx-relay
    restart: unless-stopped
    network_mode: host           # Tab5 -> <NAS IP>:7355, PC software -> <NAS IP>:7356 (QMX port 3)
    privileged: true             # raw USB access, incl. the QMX being power-cycled/replugged
    volumes:
      - /dev/bus/usb:/dev/bus/usb
    command: ["--selftest"]      # test the QMX at startup (read-only) and log the result, then serve
    # all options: ["--selftest", "--port", "7355", "--serial-port", "7356", "-v"]
    # --serial-port 0 = off; -v logs every CAT exchange

```

tools/nas-relay/README.md

QMX on the Synology NAS → Tab5 over the network

The QMX stays plugged into the Synology. A small relay runs **on the NAS itself**, and the Tab5 connects to it directly over WiFi. No other computer is involved.

```

...
QMX —USB→ Synology NAS (qmx_nas_relay.py)
    |
    |→ TCP 7355 (QMXR/1: I/Q 48 kHz + CAT + Radio menus)
    |   → Tab5 QMX-Panadapter  Radio → Radio source: NAS
    |
    |→ TCP 7356 (QMX serial port 3, raw bytes)
    |   → PC software (WSJT-X CAT, rigctld, a terminal ...)
...

```

The QMX can present up to three USB serial ports (**System config → GPS & Ser. ports → USB serial ports**). The relay uses them like this:

QMX port	Used for	Needs "USB serial ports"
1	CAT for the Tab5 (panadapter, FT8, WSPR ...)	1 or more
2	Tab5 **Radio menus** (the QMX's own menu terminal)	2 or 3

| 3 | PC software over the network, TCP 7356 | 3 |

The relay talks to the QMX from userspace, so it doesn't need any DSM kernel drivers (DSM doesn't ship USB audio drivers). On the NAS it uses Linux usbfs (`/dev/bus/usb`) directly and needs nothing but DSM's own Python 3; where libusb-1.0 is available it uses that instead. It opens the QMX when the Tab5 or a PC on port 3 connects, and releases it when the last one disconnects.

1. Install the relay on the NAS

This runs on any DSM 7 model, including ARM models such as the DS416j that have no Container Manager. Tested on a DS416j (DSM 7.4.1) with a QMX+.

1. Copy this folder to a shared folder on the NAS with File Station, e.g. `/volume1/File Share/qmx-relay`.
2. **Control Panel** → **Task Scheduler** → **Create** → **Triggered Task** → **User-defined script**. Task: ``QMX relay``, User: ``root``, Event: ``Boot-up``. On **Task Settings**, Run command:

```
```
sh "/volume1/File Share/qmx-relay/start-relay.sh"
```
```

DSM warns about running scripts as root and asks for your password. Root is needed to open the USB device.

3. Select the task and click **Run** (it also starts on every boot). ``start-relay.sh`` keeps the relay running and restarts it if it exits. ``stop-relay.sh`` stops it.
4. Open ``qmx-relay.log`` in that folder. You should see ``SELFTEST PASSED``, ``listening on 0.0.0.0:7355 (QMXR/1 for the Tab5)`` and ``listening on 0.0.0.0:7356 (QMX port 3, raw serial for PC software)``.

On every start the relay runs a short **self-test** of the QMX and logs it. It measures the I/Q stream, asks read-only CAT questions (``ID``; ``FA``; ``MD``; ``VN``; nothing that transmits), and checks serial ports 2 and 3.

Check from the NAS. Create a second user-defined script task (root; a Scheduled Task set to run once is fine) with ``sh "/volume1/File Share/qmx-relay/check.sh"`` and click **Run**. It writes ``check.log``: system info, the USB devices, a probe of the QMX's interfaces, and then either

- * the relay is running: ``nettest.py`` connects to it like a Tab5 (handshake, I/Q rate, CAT, Radio menus port) and like a PC on port 3, and prints ``NETTEST PASSED``; or
- * it isn't: a one-off self-test of the QMX (``SELFTEST PASSED``).

If the probe says ``QMX not found``, check that the radio is on and try another USB cable: a charge-only cable powers the QMX but shows no device.

``get_libusb1.py`` (run by the scripts) fetches python-libusb1 from PyPI, checked against its published SHA-256. It is only used where libusb-1.0 exists; the relay works without it.

Updating the relay later

Copy the new files over the old ones with File Station, then create an empty file named ``restart.request`` in the same folder and **Run** the check task. ``check.sh`` sees the file, stops the running relay (the supervisor in ``start-relay.sh`` starts the new version 5 s later, with its self-test), deletes the file and then runs the network test. No new root task or password is needed.

Docker (Container Manager) instead

On x86 models with Container Manager (DSM 7.2+) you can run it as a container: **Container Manager** → **Project** → **Create**, Path = this folder, using the existing ``docker-compose.yml``. Or over SSH:

```
```
cd /volume1/docker/qmx-relay
sudo docker compose up -d --build
sudo docker logs -f qmx-relay
```
```

2. Point the Tab5 at the NAS

On the Tab5 itself: open the settings drawer and tap **Radio source** (top of the Radio group). The button shows where the radio is and whether it is connected, e.g. ``NAS 10.0.0.137 ✓``, ``NAS 10.0.0.137 (offline)`` or ``Radio: USB``. Type the NAS's IP address and tap **Use NAS**, or tap **Use USB** to go back to a QMX plugged into the Tab5. The last NAS address is remembered, so switching back is one tap.

Or on the Tab5's web page, open the **Radio** menu, click **Radio source**, and

enter the NAS's IP address. It reads ****Radio source: NAS (network)**** once connected, and the panadapter, FT8 and CAT work exactly as they do over USB. Leave the field blank to go back to the QMX on the Tab5's own USB port.

You can also set ``relay_host`` / ``relay_port`` in the Tab5 config file, or call ``POST /api/relay {"host":"192.168.1.x","port":7355}``. ``GET /api/relay`` shows the link state, the I/Q sample count and the last error.

The Tab5 firmware needs the network-radio WiFi patches

The stream is a steady ~1.5 Mbit/s into the Tab5, far more than its WiFi link (an ESP32-C6 on SDIO, driven by `esp_hosted`) normally receives. Two of the project's standing patches make that work, and ``tools/check_patches.py`` refuses a build without them:

- * ``apply_esp_hosted_sdio_split_bundles.ps1`` - the C6 delivers several frames per SDIO transfer; stock code kept only the first, or discarded a large burst whole. Every frame is now delivered.
- * ``apply_esp_hosted_sdio_rx_copy_psr.am.ps1`` - every received frame was copied into the Tab5's small internal RAM; under the stream that ran out and touch input and the web page stopped. The copy now goes to PSRAM.
- * ``apply_esp_hosted_sdio_split_psr.am.ps1`` - received frames waited in the RX queue in blocks of a buffer pool that never gives memory back, so the pool grew until the Tab5's DMA-capable RAM was gone. Frames are now queued as PSRAM copies and the pool block is returned at once.

With these (and the TCP receive window at 11520 in ``sdkconfig``) a Tab5 on WiFi receives ~47,700 of the 48,000 I/Q pairs per second, measured on a DS416j and a QMX+. Without them it gets 10-20k and touch input freezes.

Building the Tab5 firmware on a Mac

```
...
bash tools/mac-build-flash.sh          # build + flash (asks before flashing)
bash tools/mac-build-flash.sh --build  # build only
bash tools/mac-capture-serial.sh 90    # record the Tab5's USB serial output
...
```

It uses the ESP-IDF in ``~/esp/esp-idf`` (v5.4.4) and an installed PowerShell (``pwsh``, or ``PWSH=/path/to/pwsh``) for the standing patches, keeps PlatformIO's own Python out of ESP-IDF's way, applies and checks all patches, builds, and does a normal flash (settings kept). Logs go to ``build/mac-build-flash.log``. The firmware reports itself as the release it is based on plus ``-NAS``, e.g. ``v1.16.11-NAS``.

3. PC software on QMX port 3 (optional)

Set ****USB serial ports = 3**** on the radio. The relay then serves the QMX's third serial port as a plain TCP port on the NAS, ****7356****. Whatever a PC sends there goes to the radio unchanged, and the radio's replies come straight back, just like a serial cable. Use it for CAT control from WSJT-X, Hamlib/rigctld, logging programs, or a terminal. Configure the program for the QMX exactly as you would for a USB-connected QMX.

Most programs expect a COM port, so map one to the TCP port on the PC:

- * ****macOS / Linux:**** ``socat pty,link=$HOME/qmx-port3,raw,echo=0 tcp:<NAS IP>:7356`` and point the program at ``~/qmx-port3``.
- * ****Windows:**** any "virtual serial port over TCP" tool, pointed at ``<NAS IP>:7356``.
- * Quick check: ``nc <NAS IP> 7356``, then type ``FA``; and the radio answers with its frequency.

Notes:

- * Port 3 carries ****serial data only, not audio****. For digital modes on the PC you still need the radio's audio there by other means. The Tab5 keeps its own I/Q stream either way.
- * The PC and the Tab5 can use the radio at the same time, each on its own port. Changes made from the PC (frequency, mode) show up on the Tab5 within a poll cycle.
- * One PC at a time on port 3; a new connection replaces the old one.
- * With the radio set to 1 or 2 ports, a connection to 7356 is closed straight away and the log says why.
- * Change the port with ``--serial-port N``, or turn it off with ``--serial-port 0`` (add it to the ``qmx_nas_relay.py`` line in ``start-relay.sh``, or to ``command:`` in ``docker-compose.yml``).

Behaviour

- * If the QMX is powered off or unplugged, the relay closes the connection and the Tab5 treats it like a USB unplug. The Tab5 retries every ~2 s and reconnects on its own when the QMX comes back.
- * The relay must be the only program on the NAS that uses the QMX. If the

```

log says `QMX is in use by another program`, stop whatever else holds it.
* One Tab5 at a time. A new connection replaces the old one, so a rebooted
  Tab5 doesn't get locked out by its own stale session. A PC on port 3 is
  independent of the Tab5: either can come and go without affecting the other.
* **Radio menus** (the QMX's own menu system, on the web page's Radio menu)
  work over the network too. Like on USB, they use the QMX's *second* serial
  port, so CAT is never disturbed. Set **System config → GPS & Ser. ports →
  USB serial ports = 2** (or 3) on the radio. With one port, the Tab5 says so
  instead of opening.
* While a NAS address is set, a QMX plugged into the Tab5's USB port is ignored.
* If WiFi can't keep up, the relay drops I/Q frames rather than build up
  delay. It never drops CAT: radio replies are always sent ahead of queued
  I/Q, the I/Q queue holds only 3 frames (30 ms), and the NAS's kernel send
  buffer is kept small (8 KB), so CAT replies are never stuck behind seconds
  of spectrum data. Drops are counted in the log.
* Bandwidth is about 200 kB/s (48 kHz × 2 × 16-bit).
* Tested: switching NAS → USB → NAS from the drawer, and powering the QMX off
  and on (power lead pulled) while the Tab5 stays connected - the stream
  resumes by itself.
* The Tab5 sets the QMX's clock when it gets network time. Over the network
  that CAT write must not run on the network stack's own thread, or every
  connection on the Tab5 freezes for good about 20-50 s after boot (fixed in
  `wifi.c`; `radio_relay.c` refuses such a send instead of hanging).

## Troubleshooting

* **Touch dead, but the screen and stream keep running, even after a restart:**
  the touch chip itself is stuck, and the yellow button does not remove its
  power. Unplug the Tab5's USB-C (and switch off or remove its battery) for
  ~15 s, then power it on.
* `bash tools/mac-capture-serial.sh 120` records the Tab5's serial log. On a
  Mac, opening the port restarts the Tab5, so the log starts from boot.

## Bench test without the radio

...
python3 qmx_nas_relay.py --fake          # synthetic +3 kHz tone, CAT emulator on ports 1 and 3,
...                                     # and a small menu terminal on port 2

```